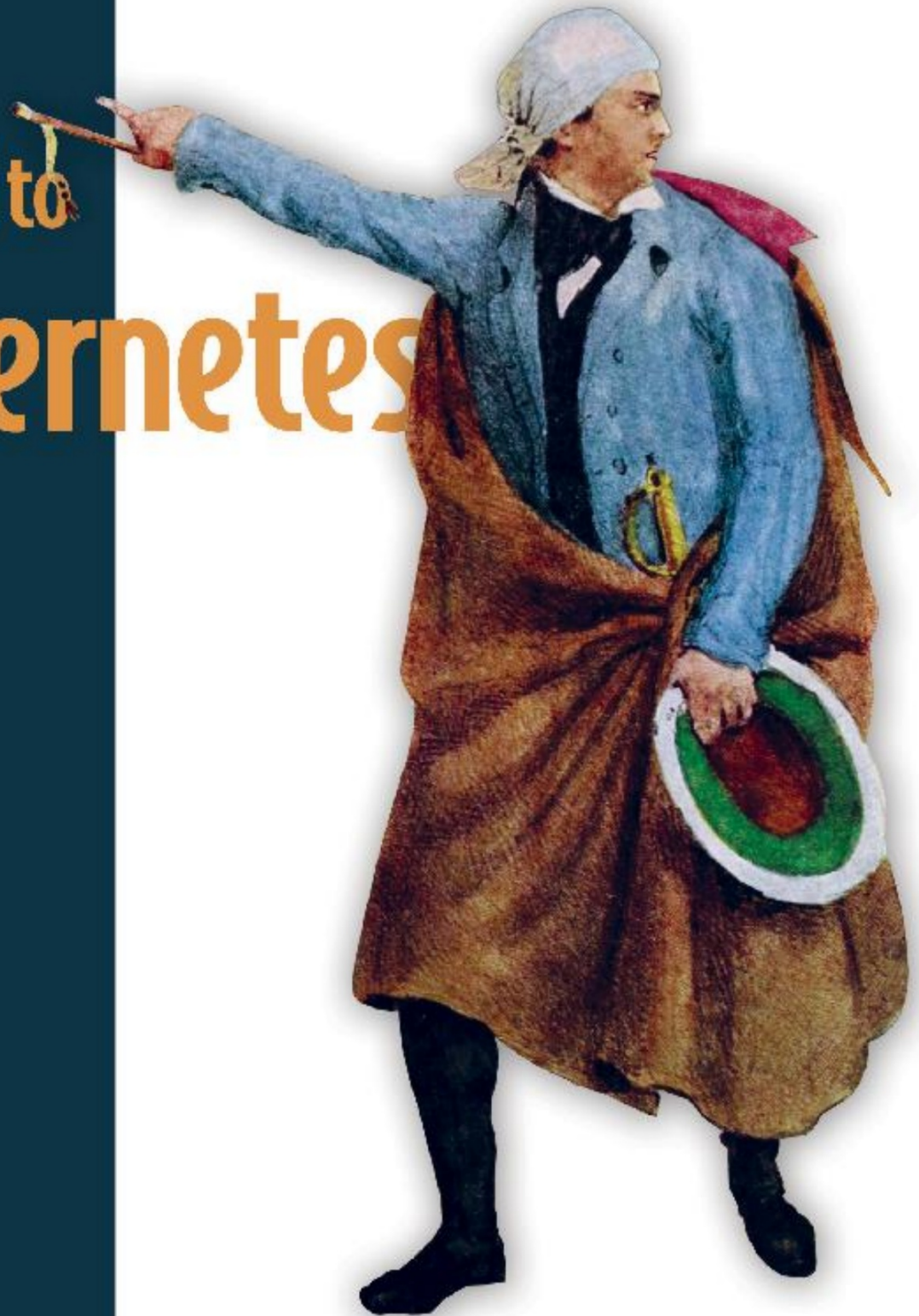


Road to Kubernetes

Justin Mitchel





Camino a Kubernetes

Justin Mitchel

Para comentar ir a [livebook](#).



Manning
Isla Shelter

Para obtener más información sobre este y otros títulos de Manning,
[visite manning.com](#).

derechos de autor

Para obtener información en línea y solicitar este y otros libros de Manning, visite www.manning.com. La editorial ofrece descuentos en este libro al comprar en grandes cantidades.

Para obtener más información, póngase en contacto con

Departamento de Ventas Especiales

Compañía de publicaciones Manning

20 Baldwin Road

Apartado postal 761

Shelter Island, Nueva York 11964

Correo electrónico: orders@manning.com

©2025 por Manning Publications Co. Todos los derechos reservados.

Ninguna parte de esta publicación podrá ser reproducida, almacenada en un sistema de recuperación o transmitida en ninguna forma o por medios electrónicos, mecánicos, de fotocopia o de otro tipo, sin el permiso previo por escrito del editor.

Muchas de las designaciones utilizadas por fabricantes y vendedores para distinguir sus productos se reclaman como marcas comerciales. Cuando esas

Las designaciones aparecen en el libro y Manning Publications estaba al tanto de un reclamo de marca registrada; las designaciones se han impreso en mayúsculas iniciales o en mayúsculas completas.

Reconociendo la importancia de preservar lo que se ha escrito, la política de Manning es que los libros que publicamos se imprimen en papel libre de ácido y hacemos nuestros mejores esfuerzos para lograr ese fin.

Reconociendo también nuestra responsabilidad de conservar los recursos de nuestro planeta, los libros de Manning se imprimen en papel que es al menos un 15 por ciento reciclado y procesado sin el uso de cloro elemental.

Los autores y la editorial han hecho todo lo posible para garantizar que la información de este libro fuera correcta al momento de la publicación. Ni los autores ni la editorial asumen ni se eximen de ninguna responsabilidad ante terceros por pérdidas, daños o interrupciones causadas por errores u omisiones, ya sean resultado de negligencia, accidente o cualquier otra causa, o por el uso de la información aquí contenida.

Compañía de publicaciones Manning
20 Baldwin Road,
Apartado Postal
761, Shelter Island, NY 11964

Editor de desarrollo: Connor O'Brien

Editor técnico: Billy Yue

Editor de producción: Aleksandar Dragosavljević

Editora: Katie Petito

Compositor: Bojan Stojanović

Diseñador de portada: Marija Tudor

ISBN: 9781633438200

Impreso en los Estados Unidos de América

dedicación

A mi esposa Emilee: gracias por tu infinito apoyo y aliento.

Soy muy afortunado de tenerte en mi vida. Gracias por darme todo.

A mis hijas McKenna, Dakota y Emerson: gracias por ser quienes son.

Cada una de ustedes es exactamente lo que nuestra familia necesita, y estoy muy orgullosa de las mujeres en las que se están convirtiendo.

Ser tu papá ha sido el regalo más grande y gratificante del mundo.

Los amo a cada uno de ustedes más de lo que se puedan imaginar y estoy muy emocionado por vivir muchas más aventuras juntos.

¡Los amo a todos!

contenido

[prefacio](#)

[expresiones de gratitud](#)

[Acerca de este libro](#)

[Sobre el autor](#)

[sobre la ilustración de la portada](#)

[1 Kubernetes y el camino hacia la implementación moderna](#)

[1.1 Cualquiera puede implementar](#)

[1.2 Nuestro camino hacia la implementación](#)

[1.3 El desafío de las dependencias](#)

[1.4 ¿Qué hacen los contenedores?](#)

[1.5 Los desafíos de construir y ejecutar contenedores sin Kubernetes](#)

[1.6 ¿Cuáles son los desafíos fundamentales al utilizar Kubernetes?](#)

[1.7 ¿Cómo enseña este libro la tecnología?](#)

[2 Creación de aplicaciones web en Python y JavaScript](#)

[2.1 Diseño de una aplicación web FastAPI básica en Python](#)

[2.1.1 Configuración de un proyecto Python con entornos virtuales](#)

[2.1.2 Hola mundo con FastAPI](#)

[2.1.3 Manejo de múltiples rutas con FastAPI](#)

2.2 Creación de una aplicación web JavaScript con Node.js y Express.js

— — — — —

2.2.1 Hola mundo desde Express.js

2.2.2 Ejecutar la aplicación Node.js + Express.js

2.2.3 Crear una respuesta JSON con Express.js

2.3 Seguimiento de cambios de código con Git

2.3.1 Usando Git en nuestros proyectos

2.3.2 Ignorar archivos

2.3.3 Archivos de seguimiento

2.3.4 Reinicio completo del repositorio Git

2.3.5 Registros y visualización de Git

2.3.6 Descripción general de los comandos fundamentales de Git

2.4 Enviar código a GitHub

2.4.1 Creación de una cuenta de GitHub

2.4.2 Crear un repositorio Git en GitHub

2.4.3 Configurando nuestro repositorio git local para GitHub

2.4.4 Enjuagar y repetir

3 Implementación manual con máquinas virtuales

3.1 Creación y conexión a un servidor remoto

3.1.1 Aprovisionamiento de una máquina virtual

3.1.2 Conexión mediante SSH

3.2 Servir sitios web estáticos con NGINX

3.2.1 Adoptar máquinas virtuales efímeras

3.2.2 Modificar la página HTML predeterminada de NGINX

[3.3 Repositorios Git remotos autoalojados](#)

[3.3.1 Iniciar sesión en el servidor con SSH](#)

[3.3.2 Creación de los repositorios Git básicos](#)

[3.3.3 Enviar código local al repositorio remoto](#)

[3.3.4 Git hook para extraer código](#)

[3.4 Instalación de las dependencias de las aplicaciones](#)

[3.4.1 Instalación de Python 3 y nuestro proyecto FastAPI](#)

[3.4.2 Instalación de Node.js y nuestra aplicación Express.js](#)

[3.4.3 Ejecución de la aplicación Express.js](#)

[3.5 Ejecutar múltiples aplicaciones en segundo plano con Supervisor](#)

[3.5.1 Instalación de Supervisor](#)

[3.5.2 Configurar Supervisor para aplicaciones](#)

[3.6 Sirva múltiples aplicaciones con NGINX y un firewall](#)

[3.6.1 Configuración de NGINX como proxy inverso](#)

[3.6.2 Instalación de un firewall sencillo](#)

[4 Implementación con GitHub Actions](#)

[4.1 Introducción a las canalizaciones de CI/CD con GitHub Actions](#)

[4.1.1 Su primer flujo de trabajo de GitHub Actions](#)

[4.1.2 Creando tu primer secreto de GitHub Actions](#)

[4.1.3 Instalación de la clave SSH pública en Akamai Connected Cloud](#)

[4.1.4 Creación de una nueva máquina virtual en Akamai Linode](#)

[4.1.5 Instalación de NGINX en un servidor remoto con un flujo de trabajo de GitHub Actions](#)

[4.2 Automatización de máquinas virtuales con Ansible](#)

[4.2.1 Flujo de trabajo de GitHub Actions para Ansible](#)

[4.2.2 Creación de su primer Playbook de Ansible](#)

[4.2.3 Ansible para nuestra aplicación Python](#)

[4.2.4 Ansible para nuestra aplicación Node.js](#)

[5 aplicaciones de contenedores](#)

[5.1 Hola mundo con Docker](#)

[5.1.1 Ejecución de su primer contenedor](#)

[5.1.2 Exposición de puertos](#)

[5.1.3 Entrada a un contenedor](#)

[5.1.4 Contenedores sin estado](#)

[5.2 Diseño de nuevas imágenes de contenedores](#)

[5.2.1 Creación de un Dockerfile](#)

[5.2.2 Ejecución de su primera imagen de contenedor](#)

[5.3 Contenerización de aplicaciones Python](#)

[5.3.1 Etiquetas de versión y un Dockerfile de Python](#)

[5.3.2 Creación de un Dockerfile de Python](#)

[5.3.3 Ignorar archivos no deseados en compilaciones de Docker](#)

[5.3.4 Creación y ejecución de un script de punto de entrada](#)

[5.4 Contenerización de aplicaciones Node.js](#)

[5.4.1 El script de punto de entrada de Node.js y el archivo dockerignore](#)

[5.4.2 El Dockerfile de Node.js](#)

[5.5 Enviar imágenes de contenedores a Docker Hub](#)

[5.5.1 Su primer envío a Docker Hub](#)

6 contenedores en acción

6.1 Creación y envío de contenedores con GitHub Actions

6.1.1 Inicio de sesión en Docker y herramientas de acción de GitHub de terceros

6.1.2 Un flujo de trabajo de GitHub Actions para crear contenedores

6.2 Administrar un contenedor con Docker Compose

6.2.1 Por qué necesitamos Docker Compose

6.2.2 Docker Compose para nuestra aplicación Python

6.2.3 Docker Compose para nuestra aplicación Node.js

6.3 Contenedores con estado y volúmenes

6.3.1 Uso de volúmenes con Docker

6.3.2 Uso de volúmenes con Docker Compose

6.3.3 Gestión de volúmenes integrada con Docker Compose

6.4 Fundamentos de redes en Docker Compose

6.4.1 Comunicación de contenedor a contenedor simplificada

6.4.2 Bases de datos y Docker Compose

6.4.3 Variables de entorno con dotenv en Docker y Docker Compose

7 Implementación de aplicaciones en contenedores

7.1 Hola prod, soy Docker

7.1.1 Aprovisionamiento de una nueva máquina virtual

7.1.2 Instalación de Docker en Ubuntu

7.1.3 Instalación de Docker Compose en Ubuntu

7.1.4 Implementación de NGINX

7.2 Contenedores de preparación

7.2.1 Dockerfiles para múltiples entornos

7.2.2 Docker Compose para múltiples entornos

7.3 Acciones de GitHub para implementar contenedores de producción

7.3.1 Creación y alojamiento de un contenedor de producción

7.3.2 Instalación de Docker y Docker Compose en una máquina virtual con GitHub Actions

7.3.3 Uso de GitHub Actions para ejecutar Docker Compose en producción

7.4 Las limitaciones del uso de Docker Compose para producción

7.4.1 Escalado de contenedores con Docker Compose

8 Implementación administrada de Kubernetes

8.1 Introducción a Kubernetes

8.1.1 ¿Kubernetes autoservicio o administrado?

8.1.2 Aprovisionamiento de un clúster de Kubernetes

8.1.3 Conceptos y componentes básicos

8.2 Conexión a Kubernetes

8.2.1 La interfaz gráfica de usuario del panel de Kubernetes

8.2.2 Instalación de kubectl

8.2.3 Configuración de kubectl

8.3 Implementar contenedores en Kubernetes

8.3.1 Tu primer Pod y manifiesto

8.3.2 Su primer Servicio

8.3.3 De los pods a las implementaciones

[8.3.4 Personalizar NGINX con ConfigMaps](#) _____

[8.3.5 Variables de entorno con ConfigMaps y secretos](#) _____

[8.4 Volúmenes y contenedores con estado](#) _____

[8.4.1 Volúmenes e implementaciones](#) _____

[8.4.2 Conjuntos con estado](#) _____

[8.4.3 Comunicación de contenedor a contenedor dentro de Kubernetes](#) _____

[8.4.4 Espacios de nombres para administrar recursos del clúster](#) _____

[8.5 Implementar aplicaciones en producción con Kubernetes](#) _____

[8.5.1 Servicios de balanceador de carga](#) _____

[8.5.2 Implementación en Kubernetes con GitHub Actions](#) _____

[9 herramientas de orquestación alternativas](#) _____

[9.1 Orquestación de contenedores con Docker Swarm](#) _____

[9.1.1 Preparación del Docker Swarm Manager](#) _____

[9.1.2 Docker Compose para Docker Swarm](#) _____

[9.1.3 Iniciar Docker Swarm](#) _____

[9.1.4 Implementar una pila Docker Swarm](#) _____

[9.2 Nómada de HashiCorp](#) _____

[9.2.1 Preparación de nuestro clúster Nomad](#) _____

[9.2.2 Instalación de Nomad](#) _____

[9.2.3 Configuración del servidor Nomad](#) _____

[9.2.4 Configuración de los clientes Nomad](#) _____

[9.2.5 Ejecución de contenedores con trabajos Nomad](#) _____

Apéndice A Instalación de Python en macOS y Ventanas

A.1 Instalación de Python para macOS

Versión A.2 de Python

A.2.1 Procesador: Intel o Apple

A.2.2 Descargar Python

A.3 Instalación de Python para Windows

A.3.1 Verificar la información del sistema

A.3.2 Instalar Python

A.4 Instalación de Python para Linux

A.5 Entornos virtuales y Python

A.5.1 Creación de una carpeta de proyecto de prueba

A.5.2 Creación de un entorno virtual

A.5.3 Activación de un entorno virtual

A.5.4 Instalación de paquetes de Python

A.5.5 Desactivación y reactivación de entornos virtuales

A.5.6 Recreación de entornos virtuales

Apéndice B Instalación de Node.js en macOS y Windows

B.1 Instalación de Node.js para macOS

B.2 Instalación de Node.js para Windows

B.2.1 Verificación de la información del sistema

B.2.2 Descarga e instalación de Node.js

Apéndice C Configuración de claves SSH para ingresar al servidor sin contraseña

C.1 Generación de claves SSH multiplataforma

C.1.1 Navegación a su directorio raíz SSH

C.1.2 Generación manual de claves SSH con ssh-keygen

C.1.3 Automatización de ssh-keygen

C.1.4 Verificación de las claves

C.2 Sobrescritura de claves SSH

C.3 Copia de la clave pública

C.4 Instalación de la clave pública SSH en un host remoto

C.4.1 Instalación manual de claves SSH con copiar y pegar

C.4.2 Instalación automática de claves SSH con el comando ssh-copy-id

C.4.3 Instalación automática en máquinas virtuales en Linode

C.5 Conectarse al host remoto con una clave SSH específica

C.6 Actualización o creación de configuración SSH

Apéndice D Instalación y uso de ngrok

D.1 Instalación de ngrok

D.2 Verificar la instalación

D.3 Ejecución de ngrok

índice

prefacio

Creo que en la próxima década se lanzarán más aplicaciones que en la década anterior, multiplicadas por diez. Este crecimiento exponencial estará impulsado por tecnologías de implementación modernas como los contenedores y la orquestación de contenedores, junto con el auge en el uso de herramientas de IA generativa. Si bien las herramientas y las tecnologías pueden cambiar, estoy seguro de que se implementarán más aplicaciones.

He visto cómo el software ha evolucionado de los disquetes a los CD, a los cartuchos (para consolas), a los DVD y, finalmente, a internet. A medida que cambiaba el formato de distribución, también lo hacía la cantidad de aplicaciones lanzadas; estas aumentaron exponencialmente. Aunque cada cambio de formato físico no generó este crecimiento exponencial, internet sí lo hizo. Crecí en una época en la que nunca tuve que escribir una sola línea de código que requiriera un disco o cartucho para que la aplicación funcionara. Mis habilidades de programación y desarrollo web se desarrollaron en internet.

Cuando tenía 16 años, decidí que iba a hacer fortuna vendiendo camisetas. Un amigo estuvo de acuerdo con la idea, así que unimos fuerzas e hicimos lo que cualquier adolescente inteligente haría: pedir dinero prestado a nuestros padres. Aunque ese dinero fue suficiente para imprimir camisetas, no para crear una página web, o eso creíamos. En aquel entonces, MySpace era la red social más popular y el comercio electrónico estaba en pleno auge. Queríamos dar la impresión más legítima posible a los posibles minoristas, así que pensamos que crear una página web era una gran idea. ¿El problema? Contratar una empresa para crear una página web costaba al menos 800 dólares. Como gastábamos todo el dinero de nuestros padres, teníamos que hacerlo nosotros mismos.

Fue entonces cuando conocí a Bobby Kim, el fundador de una empresa de ropa en auge, The Hundreds, y me dio un consejo que nunca olvidaré: "¡Caramba! No necesitas contratar una empresa de diseño web, ¡compra un libro y hazlo tú mismo! Eso fue lo que hicimos; no es tan difícil".

Aunque no estaba del todo de acuerdo con la idea de "No es tan difícil", compré el libro. Para mi sorpresa, crear un sitio web HTML básico fue fácil, gracias a su carácter práctico y práctico. Aunque el sitio web que creé era, sin duda, amateur desde cualquier punto de vista, pude publicarlo en internet con la ayuda de un amigo de la familia.

Logro desbloqueado y \$800 ahorrados.

Quizás no sorprenda saber que un par de jóvenes de 16 años no lograron competir con marcas como Quiksilver y Billabong justo antes de una gran recesión mundial. Si bien esa frase podría reflejar la situación económica actual y tal vez sería un buen chiste para una película, guardo un recuerdo muy especial de esa época porque esa empresa de camisetas me impulsó al maravilloso mundo de crear productos para internet.

Unos años después, me encontré en una situación difícil creando software real. Aunque HTML pueda parecer "software real", lo considero más bien un documento de texto complejo (ejem, porque lo es). Una vez más, me equivoqué al pensar que escribir código era demasiado complejo y, por lo tanto, no era para mí. Por suerte, durante otro de mis muchos proyectos, mi cofundador técnico me sugirió probar esta herramienta llamada Python. Y así lo hice. Seguro que ya saben cómo sigue la historia.

Parecía que era cuestión de meses, y lanzaba regularmente mis aplicaciones web basadas en Python al mundo más rápido de lo que jamás hubiera imaginado. Estas habilidades me llevaron a...

Muchas oportunidades de trabajo freelance y muchas más oportunidades para aprender y crecer. Eso fue exactamente lo que hice.

Con el tiempo, me di cuenta de que necesitaba ayudar a otros a superar sus propias dudas al escribir, aprender y lanzar software al mundo. Unos meses después, un amigo me invitó a un taller de fin de semana llamado Startup Weekend, donde varios emprendedores y programadores se reunirían para crear negocios en tan solo dos días. Decidí que, en lugar de conducir más de 10 horas para un taller en otro estado, me concentraría en crear mi primer curso. Después de un fin de semana repleto de cafeína, terminé mi primer curso de cuatro horas y estaba listo para publicar.

Aquí hay otro punto de inflexión en mi camino hacia ti, que lees estas palabras. ¿Estaba listo? ¿Fue bueno el curso? ¿Descubrirán que soy malo en esto? Las preguntas me rondaban la cabeza, haciéndome replantear mi curso por completo. Durante este tiempo, dos citas resonaron en mi mente:

“Los negocios son sólo dos cosas: innovación y marketing”.
—Peter Drucker

“Simplemente hazlo” —Nike

Así que, a pesar de mis dudas y pensamientos contraproducentes, decidí lanzar el curso, lo cual me llevó más de dos semanas. Decidí llamarlo "Programación para Emprendedores" porque refleja exactamente quién soy y con quién me identifico: un emprendedor programador. También pensé que si mi yo de 16 años hubiera visto el curso "Programación para Emprendedores", lo habría comprado en dos segundos.

El curso despegó y rápidamente reunió a decenas de miles de estudiantes, lo que condujo a una exitosa campaña de Kickstarter y, finalmente, a cientos de miles de estudiantes en todo el mundo.

Todo el mundo a través de plataformas como Udemy, YouTube, CodingForEntrepreneurs.com y muchas otras. Hasta la fecha, tengo más de 800.000 estudiantes inscritos en Udemy, más de 235.000 suscriptores en YouTube y más de 15.000 seguidores en GitHub.

Creé este libro como resultado de mis múltiples experiencias con el software y, aún más importante, con su implementación. Creo que el software implementado es el único que te impulsará a ti o a tu equipo. Escucho constantemente afirmaciones sobre cuál es el mejor lugar para implementar software, y, sinceramente, no creo que exista nunca un mejor. Mejor implica terminado, hecho, finalizado, etc., mientras que mejor implica que existe la posibilidad de algo peor. En un momento dado, el protocolo de transferencia de archivos (FTP) se consideró la mejor opción para implementar código, y, ¡alerta de spoiler!, ya no lo es.

Aunque acabo de quejarme del término "mejor", diré que creo que los contenedores y la orquestación de contenedores son, a finales de 2023, las mejores maneras de implementar código en producción. Sospecho que estas herramientas eventualmente cambiarán hacia algo mejor.

El camino hacia Kubernetes trata sobre aprender diversas opciones para implementar tus aplicaciones en producción y así poder hacerlo continuamente. Escribí este libro como un recorrido paso a paso para que obtengas resultados prácticos y tangibles en cada paso del proceso, lo que te ayudará a implementar con mayor frecuencia y, si hice bien mi trabajo, a implementar mejor. Cuanto más puedas implementar tu software, mayor será el impulso que tu proyecto pueda tener y, por lo tanto, ayudarás a otros en el proceso. Cuanto más puedas implementar software, mejor será.

expresiones de gratitud

Primero, quiero agradecerle, querido lector. Creé este libro para ti y espero que lo disfrutes. No dudes en contactarme para compartir cualquier idea que tengas.

Me gustaría agradecer a mis estudiantes y seguidores por todo su mirar, leer, escuchar, compartir y hacer el trabajo; no estaría donde estoy hoy si no fuera por ustedes.

Gracias por permitirme ser una voz en este grupo de personas excepcionales.

Quisiera agradecer a Akamai por su apoyo en mis proyectos educativos y tecnológicos. Akamai ha sido un gran aliado en mi misión de ayudar a otros a desarrollar software y lanzarlo al mundo.

Quisiera agradecer a Timothy Ryan por su apoyo para hacer posible este libro y por escuchar mis largas conversaciones sobre tecnología. También quiero agradecer a Andrew, Justin, Jamie, Hillary, Kerterika, Maddie, Talia y otros miembros del equipo de Akamai Connected Cloud por su ayuda en este y muchos otros proyectos.

Si bien esta lista no es exhaustiva, le agradezco a usted y todo lo que ha hecho durante el tiempo que hemos trabajado juntos.

Por último, me gustaría agradecer a Andy, Ian y Connor de Manning por ayudarnos a darle vida a este proyecto.

Acerca de este libro

Con Road to Kubernetes, comenzamos creando aplicaciones web sencillas en Python y Node.js. Estas aplicaciones nos acompañarán durante todo el proceso para que podamos aprender sobre las diversas herramientas y técnicas para implementarlas. A partir de ahí, comenzamos a usar shells seguros (SSH) y un control de versiones moderno mediante Git con entornos, repositorios e implementaciones autogestionados.

Tras comprender el método de implementación autogestionado, pasamos a la automatización con GitHub, GitHub Actions e incluso Ansible. GitHub es un popular repositorio Git gestionado por terceros y servicio de alojamiento de código que también nos permite ejecutar flujos de trabajo de código puntuales llamados GitHub Actions. Estos flujos de trabajo son entornos informáticos de corta duración que nos ayudan a compilar, probar e implementar nuestro código y nos ayudan a integrar y entregar (CI/CD) continuamente nuestras aplicaciones. Ansible nos ayuda a configurar automáticamente nuestros entornos de implementación (por ejemplo, máquinas virtuales) al indicar el software necesario para ejecutar nuestras aplicaciones.

Si bien Ansible es excelente para configurar nuestros entornos a posteriori, comenzamos a adoptar una forma de preconfigurarlos en entornos de ejecución portátiles y manejables, llamados contenedores. Los contenedores y el proceso de contenerización fueron pioneros de Docker y se conocen como contenedores Docker. Estos contenedores son, en esencia, aplicaciones que incluyen un pequeño sistema operativo para ejecutar nuestro código; imagínelo como un pequeño sistema operativo Linux que ejecuta nuestra aplicación Python o Node.js y que se puede mover fácilmente de un sistema a otro sin necesidad de configuración adicional.

Tras aprender sobre la creación y ejecución de contenedores, aprendimos a gestionar la implementación de diferentes tipos de contenedores, ya que las aplicaciones rara vez existen de forma aislada: las aplicaciones web suelen necesitar bases de datos para ejecutarse. Ejecutar y gestionar más de un contenedor a la vez se denomina orquestación de contenedores y es exactamente lo que hacen Docker Compose, Docker Swarm, HashiCorp Nomad y, por supuesto, Kubernetes. Si bien cada herramienta gestiona los contenedores de forma diferente, analizaremos diversas formas en que estas herramientas se intersecan y cuándo y dónde se pueden usar.

Kubernetes es una de las herramientas de orquestación de contenedores más utilizadas y, en muchos sentidos, es superior a las alternativas gracias a su ecosistema de terceros y al soporte gestionado por numerosos proveedores de nube. En algunos casos, las empresas ofrecen servicios para ejecutar y gestionar sus contenedores, donde la tecnología subyacente es, en realidad, únicamente Kubernetes.

Aunque implementar en Kubernetes puede parecer complejo, este libro busca descifrar esta complejidad centrándose en las diversas tecnologías y técnicas de implementación disponibles para que pueda determinar cuál es la mejor opción para usted y sus proyectos. En la mayoría de los casos, Kubernetes es la mejor opción.

¿Quién debería leer este libro?

Si no está seguro de si Kubernetes es adecuado para usted o su proyecto, este libro es ideal. Idealmente, tiene conocimientos básicos de escritura de código Python o JavaScript, lo que le permitirá empezar con los ejemplos prácticos de este libro.

Si no está seguro de si los contenedores o los contenedores Docker son adecuados para sus aplicaciones, este libro también es para usted.

Los contenedores son un elemento esencial de Kubernetes, ya que permiten crear aplicaciones portátiles que pueden ejecutarse en cualquier ordenador, la nube, servidores locales, Raspberry Pis o una combinación de todos ellos. Los contenedores pueden ayudar a escalar tus aplicaciones. Kubernetes es una de las herramientas que garantiza la escalabilidad, pero no es la única. También aprendemos sobre Docker Compose, Docker Swarm y HashiCorp Nomad para ejecutar contenedores.

Cómo está organizado este libro: una hoja de ruta

El nombre de este libro implica que estás a punto de emprender un viaje con un propósito. Dependiendo de tu experiencia, puedes retomar casi cualquier parte de este recorrido. Si bien cada capítulo se basa en el anterior, el objetivo es que puedas avanzar en este libro a medida que tu proyecto crece o tu curiosidad lo hace.

El capítulo 1 sienta las bases de lo que haremos en este libro. Este capítulo está dirigido tanto a lectores con conocimientos técnicos como a aquellos que no lo tienen, y busca ayudar a comprender la situación actual de la implementación y qué se puede hacer al respecto.

En el capítulo 2, creamos ejemplos de aplicaciones web en Python y JavaScript mediante Node.js. Estas aplicaciones sirven como complemento para prácticamente cualquier tipo de aplicación que se desee crear. Dos entornos de ejecución diferentes, Python y Node.js, nos ayudan a comprender los desafíos que enfrentamos al implementar diversos tipos de software; estos desafíos son casi idénticos, pero lo suficientemente diferentes como para causar numerosos problemas.

El capítulo 3 es la primera incursión en la implementación de aplicaciones. Este capítulo trata sobre el trabajo manual que se realizará para implementar código en un servidor utilizando tecnologías modernas como el control de versiones mediante Git y tecnologías consolidadas como shells seguros y firewalls. El carácter manual de este capítulo es un rito de iniciación para muchos desarrolladores, ya que suele ser la forma más rudimentaria de implementar una aplicación.

El capítulo 4 convierte una implementación manual en una automatizada aprovechando GitHub y GitHub Actions para ejecutar varios comandos en nuestro nombre mediante un solo proceso computacional.

Flujo de trabajo. Este flujo de trabajo informático nos permite usar el software de terceros Ansible para automatizar la configuración de nuestros entornos de implementación en lugar de hacerlo manualmente.

En el capítulo 5, comenzamos a aprender a agrupar nuestras aplicaciones en contenedores. Los contenedores nos ayudan a preconfigurar entornos para nuestras aplicaciones, lo que nos permite moverlas con mayor facilidad con muy poca configuración adicional.

Aunque esto pueda parecer más complejo al principio, las aplicaciones contenedorizadas pueden ejecutarse dondequiera que haya un entorno de ejecución de contenedores, independientemente de si la aplicación usa Python, Node.js, Ruby, Java, etc. Los entornos de ejecución de contenedores significan que no necesitamos que nuestros sistemas de implementación tengan entornos de ejecución de aplicaciones instalados para funcionar; es decir, podemos omitir la instalación de Python, Node.js, Ruby, Java, etc., y usar solo un contenedor y un entorno de ejecución de contenedores.

En el capítulo 6, usamos la automatización para crear y enviar nuestros contenedores a un lugar de almacenamiento llamado Docker Hub. A partir de aquí, aprenderemos sobre nuestra primera forma de gestión de múltiples contenedores, llamada orquestación de contenedores, con una herramienta llamada Docker Compose.

El capítulo 7 es donde implementamos nuestros primeros contenedores en un servidor de producción, lo que requerirá que configuremos el servidor y ejecutemos nuestras imágenes de contenedores a través de Docker Compose, todo orquestado por GitHub Actions.

El capítulo 8 es donde implementamos nuestros contenedores en Kubernetes. En este capítulo, aprenderemos a aprovisionar un clúster de Kubernetes administrado en un conjunto de máquinas virtuales. Este clúster de máquinas, junto con Kubernetes, nos permite escalar y administrar contenedores, a diferencia de cualquier herramienta que hayamos utilizado hasta ahora. Kubernetes administrado ofrece funciones de producción muy necesarias, como una dirección IP estática.

Balanceadores de carga y volúmenes persistentes. También aprendemos a diseñar manifiestos para poder definir con precisión qué deben hacer los contenedores en Kubernetes y cómo.

El capítulo 9 es donde implementamos contenedores en dos alternativas de Kubernetes llamadas Docker Swarm y HashiCorp Nomad.

Ambas herramientas organizan contenedores como Kubernetes, pero su enfoque y funcionalidades son diferentes. Docker Swarm es una extensión natural de Docker Compose, mientras que HashiCorp Nomad ofrece una visión única de la gestión de contenedores que se integra a la perfección con el ecosistema de herramientas de HashiCorp como Terraform y Vault.

Como puede ver, cada capítulo se basa en los conceptos presentados en capítulos anteriores y finaliza con dos capítulos que explican cómo gestionar contenedores con herramientas modernas de orquestación de contenedores. El componente fundamental de Kubernetes es un contenedor y, por lo tanto, los contenedores también son un concepto fundamental en este libro.

Los primeros capítulos le ayudarán a comprender tanto conceptual como prácticamente la necesidad de los contenedores, mientras que el resto del libro le ayudará a comprender y aprovechar mejor los contenedores en la implementación.

Acerca del código

Este libro contiene numerosos ejemplos de código fuente, tanto en listas numeradas como en formato de texto normal. En ambos casos, el código fuente se presenta con una fuente de ancho fijo como esta para diferenciarlo del texto normal. En ocasiones, el código también aparece en negrita para resaltar el código que ha cambiado con respecto a pasos anteriores del capítulo, como cuando se añade una nueva función a una línea de código existente.

En muchos casos, se ha reformateado el código fuente original; hemos añadido saltos de línea y rediseñado la sangría para adaptarla al espacio disponible en el libro. En raras ocasiones, ni siquiera esto fue suficiente, y los listados incluyen marcadores de continuación de línea (`>`). Además, los comentarios en el código fuente se han eliminado con frecuencia de los listados cuando el código se describe en el texto.

Las anotaciones de código acompañan a muchos de los listados, resaltando conceptos importantes.

Puede obtener fragmentos de código ejecutables de la versión liveBook (en línea) de este libro en [https://](https://livebook.manning.com/book/road-to-kubernetes)

livebook.manning.com/book/road-to-kubernetes.

El código completo de los ejemplos del libro está disponible para descargar desde el sitio web de Manning en [https://www.manning.com/](https://www.manning.com/books/road-to-kubernetes)

[books/road-to-kubernetes](https://www.manning.com/books/road-to-kubernetes).

Cada capítulo, con la excepción del capítulo 1, utiliza código y listados de códigos. Hay algunos repositorios públicos disponibles en

- <https://github.com/jmitchel3/roadtok8s-py> [https://github.com/](https://github.com/jmitchel3/roadtok8s-py)
- [jmitchel3/roadtok8s-js](https://github.com/jmitchel3/roadtok8s-js) [https://github.com/jmitchel3/](https://github.com/jmitchel3/roadtok8s-js)
- [roadtok8s-kube](https://github.com/jmitchel3/roadtok8s-kube)

Los capítulos específicos harán referencia a estos repositorios de código según sea necesario. El código disponible complementa el libro y se utiliza como material de aprendizaje. El código puede o no estar listo para producción, y el autor no puede garantizar que se mantenga más allá de lo que ya está disponible.

Foro de discusión de liveBook

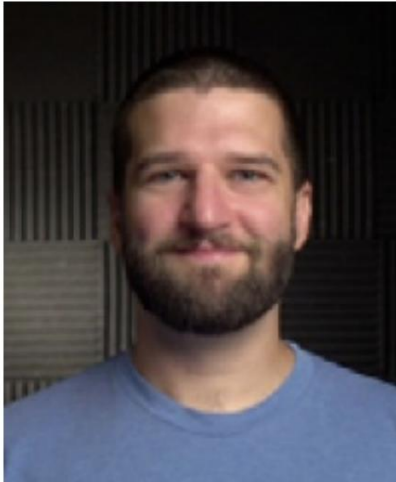
La compra de Road to Kubernetes incluye acceso gratuito a liveBook, la plataforma de lectura en línea de Manning. Gracias a las funciones exclusivas de discusión de liveBook, puede adjuntar comentarios al libro de forma global o a secciones o párrafos específicos. Es muy fácil tomar notas, hacer y responder preguntas técnicas, y recibir ayuda del autor y otros usuarios. Para acceder al foro, visite <https://livebook.manning.com/book/road-to-kubernetes/discussion>. También puede obtener más información sobre los foros de Manning y las normas de conducta en <https://livebook.manning.com/discussion>.

El compromiso de Manning con nuestros lectores es brindar un espacio donde se pueda entablar un diálogo significativo entre lectores individuales y entre lectores y el autor. No implica un compromiso de participación específico por parte del autor, cuya contribución al foro es voluntaria (y no remunerada). Le sugerimos que intente hacerle algunas preguntas interesantes al autor para que no se desvíe de su interés. El foro y los archivos de discusiones previas estarán disponibles desde el sitio web de la editorial mientras el libro esté impreso.

Otros recursos en línea

En mi blog "Coding For Entrepreneurs", escribo regularmente guías y tutoriales sobre desarrollo de aplicaciones, contenedores, infraestructura como código, Kubernetes y otros temas relacionados con el software. El blog es gratuito y está disponible en <https://www.codingforentrepreneurs.com/blog/>.

Sobre el autor



Justin Mitchel es esposo, padre de tres hijos, emprendedor, programador y fundador de CodingforEntrepreneurs.com. Lleva más de 10 años enseñando software web como Python, JavaScript, aprendizaje automático, DevOps y más. Ha impartido clases a más de 800.000 estudiantes en Udemy, a más de 235.000 en YouTube, a más de 15.000 en GitHub y ha sido profesor adjunto durante un semestre en la Universidad del Sur de California.

SOBRE EL EDITOR TÉCNICO

Billy Yuen es arquitecto sénior en Atlassian. Fue el autor principal del libro sobre Gitops y Kubernetes y ponente en Kubecon, Velocity y Java One.

sobre la ilustración de la portada

La figura de la portada de Camino a Kuberentes, un “Le Curé Morelos” o “padre Morelos”, está tomada de un libro de Claudio Linati publicado en 1828. El libro de Linati incluye litografías coloreadas a mano que representan una variedad de trajes civiles, militares y religiosos de la sociedad mexicana de la época.

En aquella época, era fácil identificar dónde vivía la gente y cuál era su oficio o posición social simplemente por su vestimenta. Manning celebra la inventiva y la iniciativa de la industria informática con portadas basadas en la rica diversidad de la cultura regional de siglos atrás, revividas con imágenes de colecciones como esta.

1 Kubernetes y el camino hacia la implementación moderna

Este capítulo cubre

- Entendiendo nuestro camino hacia la implementación
- Implementación del control de versiones y la implementación como documentos
- Trabajando con el desafío de las dependencias
- Preguntando qué hacen los contenedores
- Uso de contenedores sin orquestación automatizada
- Uso de Kubernetes con orquestación automatizada de contenedores

Antes de poder usar Kubernetes y sus maravillosas funciones, debemos comprender el camino que lleva hasta él. Esto no se refiere a la historia detrás de la tecnología, sino a las habilidades que sustentan los fundamentos de la implementación de software desde cero hasta Kubernetes.

Este libro está diseñado para personas con al menos un nivel básico de codificación de aplicaciones de software modernas.

Si temas como funciones, clases, variables y sustitución de cadenas son nuevos para usted, este libro puede no ser para usted.

En ese caso, considere tomar un curso de programación orientada a objetos para principiantes con Python o JavaScript. Esta recomendación se debe a tres razones:

- Python y JavaScript son lenguajes increíblemente amigables para principiantes (Python lo es aún más, en mi opinión).

- Python y JavaScript son posiblemente los lenguajes de programación más utilizados.
- En este libro, implementaremos una aplicación Python y una aplicación JavaScript.

1.1 Cualquiera puede implementar

Si sabes leer inglés, puedes implementar software. Hay muchísimos libros que hacen que esto parezca más complejo de lo que debería ser. El objetivo de este libro es facilitar la implementación para ti y tu equipo. Este libro está diseñado para que puedas desarrollar tus habilidades con cada capítulo, de modo que puedas dedicarle el tiempo que necesites antes de continuar.

Una vez que comprendamos los conceptos básicos de la implementación, comenzaremos a adoptar tecnologías cada vez más avanzadas, manteniendo un nivel de simplicidad que, suponiendo que siga el proceso paso a paso, podrá comprender.

Kubernetes y Docker son términos que podrían asustarte, pero no deberían. Ambas son herramientas declarativas . Aquí tienes ejemplos de declaraciones:

- Utilice esta aplicación Python `insert_your_app_name`.
- Utilice Python 3.10.
- Ejecútelo en el puerto 80 y 443.
- Permitir el tráfico desde Internet.
- Utilice `example.com`.

Estas declaraciones indican lo que quiero que se haga , no cómo debería hacerse. El cómo lo gestiona la propia herramienta.

Nos preocupa el resultado, no el proceso para lograrlo (figura 1.1).

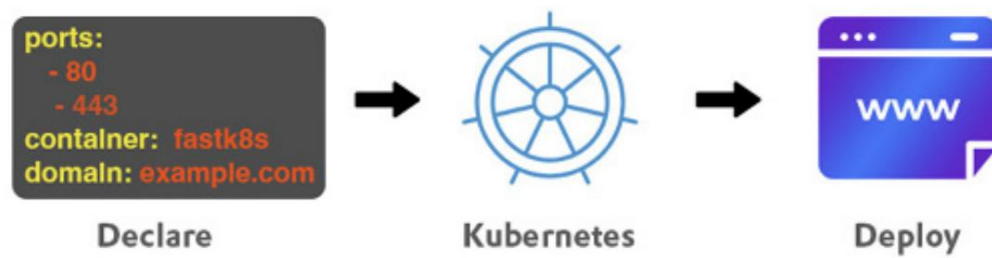


Figura 1.1 Declarar a Kubernetes para implementar

Escribir el contenido de tus archivos declarativos puede ser complicado, pero una vez que lo hagas correctamente, podrás usarlo repetidamente en muchas de tus aplicaciones implementadas. Además, puedes reutilizar los archivos para reimplementar en servidores y hosts completamente nuevos.

En Kubernetes, estos archivos declarativos se denominan manifiestos.

En Docker, se denominan Dockerfiles. Servidores web como Apache (httpd) y NGINX se ejecutan con archivos declarativos. Herramientas populares de infraestructura como código, como Ansible y Terraform, también se ejecutan con archivos declarativos.

1.2 Nuestro camino hacia la implementación

El software se está comiendo el mundo. —Marc Andreessen, 2011

El software implementado es el único tipo de software que puede devorar el mundo; si no se implementa, ¿cómo puede aportar valor al mundo?

El software se puede implementar en:

- Dispositivos personales, como teléfonos inteligentes, tabletas, computadoras de escritorio, portátiles, relojes, televisores, automóviles, refrigeradores, termostatos, tostadoras, vehículos recreativos, etc.

- La nube (servidores virtuales) con proveedores como Amazon Web Services (AWS), Linode, Google Cloud Platform (GCP), Microsoft Azure, DigitalOcean y muchos otros.
- Servidores físicos locales, como servidores Linux, servidores Windows, computadoras de escritorio antiguas, Raspberry Pis, Turing Pi y muchos otros.
- Implementaciones híbridas en cualquiera o todos los anteriores.

Existen demasiadas opciones para implementar software como para que este libro las cubra, por lo que nos centraremos en el software basado en la nube y con conexión a internet. Independientemente de dónde se implemente, el software tiene el potencial de llegar a todos los demás software implementados gracias al poder de las redes e internet.

Nuestro camino de implementación a futuro se muestra en la figura 1.2.

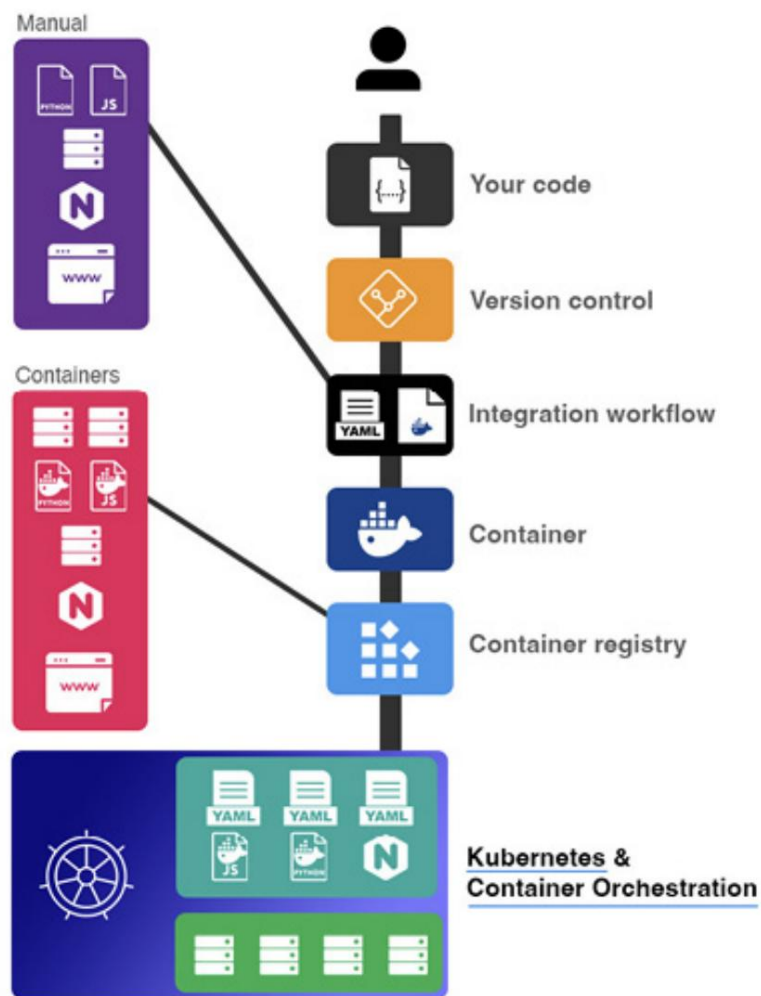


Figura 1.2 Ruta del código a Kubernetes

A lo largo de este libro, aprenderemos varios métodos de implementación para apreciar plenamente el poder y la simplicidad que ofrece Kubernetes.

Comenzamos desarrollando dos aplicaciones web diferentes en Python y JavaScript como herramientas que podemos implementar y en las que podemos iterar continuamente. Estas aplicaciones serán en su mayoría rudimentarias, pero nos ayudarán a comprender de forma tangible cada una de las fases de implementación.

Una vez escritas nuestras aplicaciones, usaremos Git, un sistema de control de versiones, para rastrear y compartir todos los cambios en nuestro código. Con Git, implementaremos un repositorio de GitHub para almacenar nuestro código, junto con un flujo de trabajo de integración con GitHub Actions como base para una automatización completa de la implementación lista para producción.

A continuación, adoptaremos contenedores, comúnmente conocidos como contenedores Docker, para gestionar mejor las dependencias de las aplicaciones y empaquetar, compartir y ejecutar nuestro código en prácticamente cualquier máquina. Implementaremos un flujo de trabajo de integración que generará automáticamente nuestros contenedores en función de nuestras actividades en Git. Una vez generados, los subiremos a un centro de almacenamiento y distribución de contenedores llamado registro de contenedores.

Con nuestros contenedores almacenados en un registro de contenedores, podremos avanzar a la siguiente fase: implementaciones basadas en contenedores. Pronto descubriremos que este tipo de implementaciones puede ser mucho más sencillo que nuestro proceso de implementación sin contenedores, gracias a la forma en que los contenedores gestionan las dependencias de las aplicaciones y a la eficacia de las herramientas modernas para la extracción (es decir, la descarga) y la ejecución de aplicaciones contenedorizadas.

Ahora que sentimos que hemos descubierto un nuevo mundo de implementación, nos damos cuenta rápidamente de que las implementaciones basadas en contenedores también tienen sus límites. Implementar más de un contenedor a la vez es posible con herramientas como Docker Compose y Watchtower, pero gestionar las actualizaciones y el tráfico adicional (carga) se vuelve rápidamente insostenible.

Podría pensar que añadir recursos computacionales adicionales para gestionar el tráfico es una buena idea, y a menudo lo es, pero una pregunta fundamental es: ¿estamos ejecutando eficientemente nuestras aplicaciones basadas en contenedores? ¿Estamos asignando bien los recursos?

Preguntas y muchas más se resuelven añadiendo la orquestación automatizada de contenedores a nuestro flujo de trabajo. Una de esas opciones es Kubernetes.

Implementar una implementación simple de Kubernetes es, como veremos, bastante sencillo cuando se utiliza un servicio de Kubernetes administrado (como veremos). Lo que se vuelve más complejo es agregar servicios adicionales que pueden, o no, requerir tráfico de internet.

El propósito de este libro es ayudarle a comprender y resolver muchos de los desafíos del camino trazado aquí.

1.3 El desafío de las dependencias

Como sabrás, el software viene en muchas formas y tamaños, ya sea Python, JavaScript, Java, Swift, Objective C, C++, C#, Rust, Ruby, PHP y muchos otros. Tan solo enumerar estos lenguajes de programación puede generar muchas preguntas:

- ¿Funcionará en mi dispositivo?
- ¿Funciona en un servidor?
- ¿Qué versión del lenguaje X estás usando?
- ¿Desde dónde descargas o instalas el idioma X?
- ¿Es una versión beta o una versión estable?
- ¿Quisiste decir <versión madura> o <nueva versión bifurcada>?
- ¿Es esta una aplicación web, un proceso de trabajo, una aplicación nativa o algo más?
- ¿Qué paquetes de terceros estás utilizando?
- ¿Es este software de código abierto o necesitamos comprar una licencia?

- ¿En qué sistema operativo se ejecutará?

Todas estas preguntas surgen de un concepto fundamental: las dependencias.

Independientemente del lenguaje de programación, los desarrolladores de software deben gestionar las diversas dependencias para que su software funcione correctamente (compilación, ejecución, etc.). Las dependencias pueden ser

- Sistema operativo (por ejemplo, los juegos de Xbox One se ejecutan en Xbox One)
- Específico de la aplicación (por ejemplo, las aplicaciones web FastAPI requieren Python para ejecutarse)
- Específico de la versión (por ejemplo, la versión 4.1 de Django requiere al menos la versión 3.8 de Python y ciertamente no Python 2.7)
- Dependencias de terceros (por ejemplo, el paquete Python MoviePy requiere que FFmpeg esté instalado en el sistema operativo)
- Específico del hardware (por ejemplo, los modelos de aprendizaje profundo de PyTorch requieren una GPU, el cable de red conectado, el enrutador encendido, CPU y RAM, etc.)

Es decir, no puedo simplemente conectar un reproductor de Blu-ray a un iPhone y ejecutar un juego de Xbox One sin modificar seriamente alguno o todos estos elementos. Incluso si el juego tiene una versión disponible en la App Store, sin duda ha sido modificado para funcionar específicamente en un iPhone. Para aclarar el punto, podrías preguntar: "¿Pero qué versión para iPhone?".

No es de extrañar que el escenario anterior se haya diseñado a propósito para favorecer la dependencia del proveedor con las respectivas plataformas de hardware y software. Este escenario también destaca cómo las dependencias de las aplicaciones y el hardware respectivo

Los requisitos pueden tener un efecto serio en el éxito del software en sí.

Kubernetes y gran parte del software que usaremos en este libro son de código abierto y, por lo tanto, se pueden usar, modificar o cambiar a voluntad. Los proyectos completamente de código abierto no están sujetos a la dependencia de un proveedor, porque ese es precisamente el objetivo.

1.4 ¿Qué hacen los contenedores?

Los contenedores empaquetan su software y sus dependencias en un paquete que facilita su ejecución, uso compartido e implementación en la mayoría de los dispositivos informáticos. En otras palabras, los contenedores hacen que su software sea portátil. El siguiente listado es un ejemplo sencillo del archivo de configuración de un contenedor.

Listado 1.1 Dockerfile

```
DESDE python:3.10
COPIAR . /app
WORKDIR /app
EJECUTAR python3 -m pip install pip --upgrade && \
    python3 -m pip install --no-cache-dir -r requisitos.txt
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "main:app"]
```

En menos de 10 líneas de código, podemos definir el aislamiento necesario para una aplicación Python casi lista para producción.

Este es el poder de los contenedores. Esta simplicidad se aplica a una gran cantidad de aplicaciones, independientemente de su lenguaje de programación.

El proceso de creación de un archivo de configuración también es sencillo. Usamos un generador de contenedores para compilar y agrupar el código de la aplicación en una imagen de contenedor desde un archivo de configuración llamado Dockerfile. Una vez que la imagen está...

Una vez construida, podemos compartir o ejecutar la imagen del contenedor usando un registro de contenedor o un entorno de ejecución de contenedor, respectivamente. A menudo se hace referencia a la creación, el uso compartido o la ejecución de contenedores simplemente como Docker, pero eso no nos cuenta toda la historia.

Docker es el software y la empresa que impulsan el concepto y la tecnología de los contenedores. Docker y muchas de las tecnologías subyacentes son ahora de código abierto, lo que significa que Docker ya no es la única forma de crear, compartir y ejecutar contenedores. Aquí hay algunos ejemplos de aplicaciones que pueden ejecutarse, y de hecho lo hacen, con contenedores:

- Lenguajes de programación: Python, Ruby, Node.js, Go, Rust, PHP Tecnología de servidor web :
- NGINX, HTTPD, Kong Bases de datos: MySQL,
- PostgreSQL, MongoDB, Cassandra, Redis
- Colas de mensajes : RabbitMQ, Kafka

Los contenedores tienen muchas propiedades que analizaremos a lo largo de este libro, pero la idea general es que, si el software es de código abierto, es muy probable que se puedan usar contenedores eficientemente con él. Si se pueden usar contenedores, se puede usar Kubernetes.

1.5 Los desafíos de construir y ejecutar contenedores sin Kubernetes

Iniciar el proceso de compilación de una imagen de contenedor es realmente sencillo. Lo que no es tan sencillo es determinar todas las dependencias que tiene cualquier aplicación: a nivel de sistema operativo, a nivel de aplicación, etc. Una vez que tenga las dependencias,

Puede empezar a crear el archivo de configuración del contenedor, conocido como Dockerfile. La complejidad de los Dockerfiles puede ser enorme y varía desde muy simples (como se mostró anteriormente) hasta los más complejos (algo que dejaremos para otro libro).

El Dockerfile a menudo puede dar la falsa impresión de que no se detectarán errores hasta compilar y ejecutar la imagen del contenedor. Incluso si la compilación se realiza correctamente, la ejecución puede fallar. Incluso si la ejecución inicial es correcta, también puede fallar.

En un mundo ideal, estos fallos se evitarían en producción. Esto, por supuesto, no siempre es posible.

Para complicar aún más las cosas, actualizar el código de una aplicación a una imagen de contenedor ya en ejecución rara vez es sencillo y puede requerir tiempo de inactividad sin la ayuda de otras herramientas. Este es otro problema que queremos evitar en producción en la medida de lo posible.

A continuación, se presentan algunos aspectos destacados de los desafíos que probablemente enfrentaremos al usar solo contenedores sin Kubernetes:

- Actualización o mejora de versiones de contenedores
- Gestionar tres o más contenedores a la vez
- Escalado de aplicaciones y cargas de trabajo basadas en contenedores
- Recuperación de fallos
- Comunicación de contenedor a contenedor (interoperabilidad)

Todos estos desafíos y muchos más a menudo se resuelven mediante el uso de Kubernetes, y estos son temas que exploramos a lo largo de este libro.

1.6 ¿Cuáles son los desafíos fundamentales al utilizar Kubernetes?

A estas alturas, podrías creer que Kubernetes es la solución definitiva para la implementación. No lo es. Los principales desafíos en la adopción de Kubernetes se reducen a...

- Instalación y actualización
- Sobrecarga de configuración
- Aplicaciones con estado y sin estado Seguridad
- y permisos Comunicación de
- contenedor a contenedor
- Almacenamiento
- Recuperación y depuración

Toda esta fricción podría hacer que los desarrolladores busquen otros lugares para implementar aplicaciones.

Kubernetes suele ejecutar contenedores en un clúster o grupo de máquinas virtuales. De forma predeterminada, Kubernetes decidirá por nosotros en qué máquina virtual o nodo se ejecutarán nuestros contenedores. La instalación y actualización del software de Kubernetes es una de las muchas razones por las que en este libro optaremos por Kubernetes administrado.

Una vez que Kubernetes esté en funcionamiento, nos enfrentaremos a una cantidad considerable de configuración, incluso para ejecutar una sola aplicación web basada en contenedores. Para actualizar incluso una sola aplicación, tendremos que aprender a implementar herramientas de integración y entrega continuas (CI/CD) y control de acceso basado en roles (RBAC) para realizar cambios en nuestro clúster de Kubernetes.

Los contenedores son apátridas por diseño, lo que significa que las aplicaciones con estado (como las bases de datos) presentan una capa adicional de complejidad para la que los desarrolladores podrían no estar preparados. Muchas aplicaciones con estado no fueron diseñadas para operaciones basadas en contenedores o incluso en clústeres, por lo que su uso plantea otros desafíos únicos.

La comunicación entre contenedores, la recuperación y la depuración son otras capas de complejidad que exploraremos en este libro.

1.7 ¿Cómo enseña este libro la tecnología?

Este libro enseñará la tecnología con ejemplos de código paso a paso y guías para implementar prácticamente dos aplicaciones web de nivel de producción escritas en Python y JavaScript, respectivamente, utilizando los siguientes enfoques:

- Implementación manual
- Despliegue de contenedores
- Implementación de Kubernetes

Cada tipo de implementación acumula habilidades y herramientas de la implementación anterior.

El beneficio de Kubernetes se completa al intentar implementar aplicaciones manualmente de forma continua en máquinas virtuales. La implementación manual es importante porque proporciona el contexto exacto de lo que hay detrás de las implementaciones automatizadas. Automatizar algo que no comprende lo dejará vulnerable ante errores o fallos inevitables, ya que todo falla constantemente.

Usamos Python y JavaScript simplemente por su popularidad entre los desarrolladores (y el autor). Si te interesa otro lenguaje de programación, considera nuestras demostraciones de aplicaciones web en el GitHub relacionado con este libro (<https://github.com/roadtokubernetes>). Recuerde, una vez que su aplicación esté en contenedor, es casi seguro que podrá utilizar dos de estas tres opciones de implementación.

La segunda mitad de este libro se dedicará a analizar algunas de las complejidades de la gestión y el mantenimiento de una Implementación de Kubernetes para que podamos integrar continuamente

Nuevas funciones y aplicaciones en nuestra implementación de Kubernetes.

Resumen

- La implementación puede ser compleja, pero cualquiera puede realizarla.
- Para realizar actualizaciones continuas, necesitamos un proceso automatizado para la implementación.
- Las aplicaciones Python y JavaScript se encuentran entre las aplicaciones web más fáciles de implementar.
- Las dependencias de la aplicación pueden ser obvias o no durante la primera implementación.
- Los contenedores ayudan a aislar nuestras aplicaciones para compartirlas y ejecutarlas.
- A menudo, la solución para gestionar los aumentos de tráfico es contar con más servidores, no con servidores más robustos.
- Agregar más aplicaciones en contenedores se vuelve cada vez más difícil.
- Kubernetes reduce la complejidad al administrar la implementación de contenedores y al distribuir los recursos computacionales de manera eficaz.

2 Creando Python y Aplicaciones web en JavaScript

Este capítulo cubre

- Diseño de una aplicación web básica FastAPI en
- Python Aislamiento de proyectos Python con entornos virtuales
- Diseño de una aplicación web básica Express.js en Java Script y Node.js
- Inicialización de nuevos proyectos Node.js usando
- npm Seguimiento de cambios de
- código con Git Envío de código a GitHub

El objetivo de este libro es comprender cómo implementar software mediante Kubernetes. Este capítulo sienta las bases al presentarnos dos aplicaciones diferentes para implementar. Estas aplicaciones serán referenciadas continuamente a lo largo de este libro y están pensadas para ser un sustituto de las aplicaciones reales que pretende implementar.

Utilizaremos Node.js, el entorno de ejecución de JavaScript, y el lenguaje de programación Python como base para estas aplicaciones. Estos dos lenguajes son increíblemente populares y fáciles de usar. También comparten varias características, aunque no son idénticas, lo que nos obliga a ajustar nuestra perspectiva sobre los diversos matices que intervienen en la implementación de cada uno. Si bien disfruto trabajando con estos lenguajes, este libro no pretende ser la guía definitiva para crear e implementar aplicaciones en Node.js o Python.

Pronto aprenderemos que implementar aplicaciones, independientemente de su lenguaje de programación o tiempo de ejecución, puede ser una tarea compleja independientemente de la complejidad de la aplicación en sí.

A lo largo de este libro, aprenderemos que los contenedores ayudan a simplificar esta complejidad al proporcionar una forma consistente de empaquetar e implementar aplicaciones. Antes de aprender sobre los contenedores, debemos aprender a crear aplicaciones básicas.

2.1 Diseño de una aplicación web

FastAPI básica en Python

FastAPI es un marco de aplicaciones web que simplifica la creación de sitios web basados en HTML o API con Python.

Utilizamos frameworks web como FastAPI, lo que nos permite evitar replantearnos muchas de las características comunes que necesitan los sitios web.

FastAPI es simplemente un paquete de Python de terceros que ya implementa muchas de estas características, lo que nos permite centrarnos en desarrollar algunos decoradores y funciones de Python para impulsar una aplicación web completamente funcional. Desarrollaremos nuestra aplicación web basada en Python con las siguientes herramientas:

- Versión 3.8 o superior de Python: si necesita ayuda para instalar Python, revise el apéndice A. venv: un módulo de
- Python para crear y administrar entornos virtuales para aislar código.
- FastAPI: un popular marco web de Python que viene con una cantidad mínima de características para crear una aplicación altamente funcional.
- uvicorn: el servidor web que
- usaremos para manejar el tráfico web con FastAPI.

Cada una de estas herramientas se configura e instala en el orden presentado. Para continuar, asumiré que ya tienes

Python está instalado en tu sistema y estás listo para crear un entorno virtual para tu proyecto. Si tienes experiencia con Python, quizás quieras usar herramientas diferentes a las que mencioné, que es exactamente lo que quiero que hagas. Si eres nuevo en Python, te recomiendo que sigas mi configuración hasta que aprendas más sobre él.

2.1.1 Configuración de un proyecto Python con entornos virtuales

Como muchos lenguajes de programación, Python cuenta con un rico ecosistema de desarrollo con numerosos paquetes y bibliotecas de terceros y, como resultado, numerosas versiones. Para facilitar el seguimiento de estas versiones y aislar nuestros proyectos de Python entre sí, utilizamos un concepto llamado entornos virtuales. Los entornos virtuales no son un aislamiento real, pero sí ayudan a mantener nuestros proyectos organizados y a evitar conflictos de versiones.

En este libro, nuestras aplicaciones Python usarán el gestor de entornos virtuales integrado, conocido como venv. Configuremos una carpeta para nuestro proyecto Python y creemos un entorno virtual.

Abra la interfaz de línea de comandos (CLI) de su sistema y navegue hasta la carpeta donde desee almacenar su proyecto de Python. Para este libro, usaré la carpeta `~/Dev/roadtok8s/py/` (listado 2.1). Si usa macOS o Linux, abra la Terminal. Si usa Windows, puede usar PowerShell o el Subsistema de Windows para Linux (WSL).

Listado 2.1 Creación de una carpeta de desarrollo de proyecto de Python

```
# Crea la carpeta del proyecto Python mkdir -p ~/
Dev/roadtok8s/py/

# Crea una carpeta para nuestro código fuente de Python mkdir -p ~/
Dev/roadtok8s/py/src

# Navegue a nuestra carpeta de proyecto Python cd ~/Dev/
roadtok8s/py/
```

Cada vez que crees un nuevo proyecto de Python, te recomiendo seguir estos mismos pasos para aislar el código de los diferentes proyectos. Para aislar aún más tu código, crearemos un entorno virtual para nuestro proyecto de Python.

Naturalmente, a estas alturas supongo que ya sabes qué intérprete de Python usarás. Si usas macOS o Linux, probablemente usarás python3 en lugar de python , y si usas Windows, probablemente usarás python en lugar de python3 (listado 2.2). Puedes obtener más información sobre esto en el apéndice A para usuarios de Mac o en el apéndice B para usuarios de Windows.

Listado 2.2 Creación de un entorno virtual de Python

```
# En nuestra carpeta de proyecto Python cd ~/
Dev/roadtok8s/py/

# Usuarios de Windows
python -m venv venv

# Usuarios de Linux o macOS
python3 -m venv venv
```

Una vez completado este comando, verá un nuevo directorio llamado venv, que contendrá todos los paquetes y bibliotecas de Python que instalaremos para este proyecto. Para usar este nuevo directorio...

Necesitamos activar el entorno virtual y todas las herramientas relacionadas. El entorno virtual se activará cada vez que se desee usar, y algunos editores de texto (como VSCode) pueden activarlo automáticamente. Activemos ahora nuestro entorno virtual, como se muestra en la siguiente lista.

Listado 2.3 Activación de un entorno virtual de Python

```
# navegue a nuestra carpeta de proyecto Python cd ~/Dev/roadtok8s/py/

# usuarios de Windows .
\venv\Scripts\activate

# usuarios de Linux o Mac
obtienen la fuente venv/bin/activate
```

Ahora que está activado, podemos empezar a instalar las dependencias de terceros de nuestro proyecto mediante el Gestor de Paquetes de Python (pip). Tenga en cuenta que pip podría estar instalado globalmente en su sistema, pero necesitamos usar la versión de pip de nuestro entorno virtual. Para garantizarlo, usamos el intérprete de Python del entorno virtual para ejecutar pip. La siguiente lista muestra cómo se hace.

Listado 2.4 Instalación de paquetes de Python

```
# Con el entorno virtual activado $(venv) python --version#1

# Actualizar pip
python -m pip install --upgrade pip

# Instalar FastAPI python -m
pip install fastapi
```

#1 En este ejemplo, \$(venv) significa que el entorno virtual está activado. La línea de comandos puede tener un aspecto diferente.

Instalar cada paquete uno a la vez es suficiente para un proyecto pequeño, pero a medida que tu proyecto crezca, necesitarás instalar muchos más. Para ello, usaremos otra función de pip que nos permite referenciar un archivo con todos los paquetes para instalar; este archivo se llama archivo de requisitos y suele llamarse requirements.txt. En la carpeta del código fuente de tu proyecto de Python (p. ej., ~/Dev/roadtok8s/py/src), agrega lo siguiente a un archivo llamado requirements.txt, como se muestra en la siguiente lista.

Listado 2.5 Requisitos de FastAPI.txt

```
fastapi
jinja2
uvicornio
gunicornio
```

Con este archivo, podemos usar la opción -f integrada en pip para instalar todos los paquetes que contiene. Este archivo de requisitos también puede declarar versiones específicas del código, pero lo dejaremos para otra ocasión. Consulte la siguiente lista para instalar paquetes de terceros.

Listado 2.6 Instalación de todos los paquetes de terceros

```
# navegue a la raíz de nuestro proyecto donde se encuentra la carpeta venv cd ~/Dev/roadtok8s/py/

# activa o reactiva tu entorno virtual source venv/bin/activate # o .
\venv\Scripts\activate

# instalar todos los paquetes a través del archivo de requisitos recién creado
mi
python -m pip install -r src/requisitos.txt
```

Si bien este archivo de requisitos puede parecer trivial, es importante tener uno para que podamos intentar recrear las condiciones necesarias para ejecutar nuestro código en otras máquinas.

Los archivos de requisitos pueden volverse mucho más complejos e incluir elementos como números de versión, rangos de versiones o incluso repositorios Git. Los archivos de requisitos complejos quedan fuera del alcance de este libro, pero si te interesa, te recomiendo revisar el paquete de código abierto de Python llamado pip-tools (<https://github.com/jazzband/pip-tools>).

Ahora que hemos establecido los requisitos en nuestro entorno, tenemos dos opciones: (1) Destruir el entorno virtual e intentarlo de nuevo para que veas lo fácil que es o (2) empezar a escribir nuestro código FastAPI. Dependiendo de tu experiencia, te dejo decidir.

2.1.2 Hola mundo con FastAPI

Creo que FastAPI se popularizó en la comunidad Python gracias a su enfoque API-first, combinado con programación asíncrona avanzada, sin dejar de ser familiar y fácil de usar. Ambas características son excelentes para los desarrolladores que desean crear una API REST capaz de gestionar un gran número de solicitudes y respuestas.

Para crear nuestra aplicación FastAPI, basta con unas pocas importaciones, inicializar una clase y definir funciones que asignen rutas URL HTTP y métodos HTTP. Para ello, crearemos un módulo de Python llamado main.py en la carpeta src de nuestro proyecto de Python, que creamos anteriormente. La siguiente lista muestra el código.

```
# Importa la clase FastAPI desde el módulo fastapi. from fastapi import FastAPI

# Declarar una instancia de la clase FastAPI. app = FastAPI()

# usa la instancia de la aplicación como decorador para manejar una ruta HTTP y un
método HTTP. @app.get("/") def read_index():

    """
    Devuelve un diccionario de Python que admite la serialización JSON
    en.
    """
    devolver {"Hola": "Mundo"}
```

Como vemos lo sencillo que es escribir este código, presenta una característica clave que muchos otros frameworks web no tienen: la función devuelve un valor de diccionario. La mayoría de los frameworks web no son tan sencillos por defecto. Este valor de diccionario se convierte en datos JSON, un formato de datos que se utiliza a menudo en el desarrollo de software de API REST. El diccionario de Python debe ser serializable en JSON, lo cual suele ser sencillo, pero puede complicarlo en ocasiones. Para probar la serialización de diccionarios con Python, es tan sencillo como ejecutar `python -c 'import json; json.dumps({"your": "dictionary_values"})'`.

Ahora que tenemos nuestro primer módulo de Python, ejecutémoslo como servidor web con uvicorn. uvicorn es una interfaz de servidor de puerta de enlace ligera que permite gestionar solicitudes y respuestas asíncronas. En capítulos posteriores, usaremos uvicorn junto con la interfaz de puerta de enlace de servidor web (WSGI) gunicorn, lista para producción, para crear un servidor web de alto rendimiento para nuestra aplicación FastAPI.

A continuación se muestra un formato estándar para su uso en la siguiente lista.

uvicorn para ejecutar aplicaciones FastAPI durante la fase de desarrollo.

Listado 2.8 sintaxis de uvicorn

```
uvicorn <módulo>:<variable> --reload --port <puerto>
```

Para ajustar esto a nuestro proyecto, el <module> corresponde a src.main porque el archivo main.py se encuentra en src/main.py. La variable <variable> corresponderá a la instancia de FastAPI(), que es la aplicación en src/main.py. El indicador --reload reiniciará la aplicación al guardar cualquier archivo relacionado con ella. El indicador --port nos permite especificar un número de puerto para que nuestra aplicación se ejecute. Veamos un ejemplo práctico de esta sintaxis en acción en la siguiente lista.

Listado 2.9 Iniciando FastAPI con uvicorn

```
# navegar al proyecto de Python cd
~/Dev/roadtok8s/py/

# activa o reactiva tu entorno virtual source venv/bin/activate # o .
\venv\Scripts\activate

# ejecutar la aplicación FastAPI con Uvicorn uvicorn
src.main:app --reload --port 8011
```

El puerto (p. ej., --port 8011) que se utiliza para cualquier aplicación cobra mayor importancia a medida que se ejecutan más proyectos. Con uvicorn ejecutando nuestra aplicación FastAPI en el puerto 8011, podemos abrir nuestro navegador web en <http://127.0.0.1:8011> y revisar los resultados (figura 2.1).



Figura 2.1 Uvicorn ejecutando FastAPI en el puerto 8011

Para algunos, llegar a este punto es un gran logro. Para otros, es un poco mundano. Independientemente de su posición en el espectro, le reto a detenerse un momento y agregar algunas nuevas rutas URL HTTP a su aplicación FastAPI antes de continuar.

2.1.3 Manejo de múltiples rutas con FastAPI

La mayoría de las aplicaciones web tienen muchas más rutas que la que acabamos de crear. Con Python y FastAPI, es increíblemente fácil agregar nuevas rutas y controladores de métodos HTTP. Agreguemos una nueva ruta a nuestra aplicación FastAPI que responda al método GET en la ruta `/api/v1/hello-world/`, como se muestra en la siguiente lista.

```
@app.get("/api/v1/hola-mundo/")#1 def
leer_hola_mundo():#2
```

Devuelve una respuesta similar a una API.

```
devolver {"qué": "carretera", "dónde": "kubernetes", "versión": "v1"}#3
```

#1 Define una nueva ruta para el método HTTP GET

#2 Muestra el nombre único de la función

#3 Devuelve un valor de diccionario de Python que se convertirá a JSON

Suponiendo que todavía tienes uvicorn ejecutándose, abre tu navegador web y visita <http://127.0.0.1:8011/api/v1/hello-world/> y deberías ver algo similar a la figura 2.2.



Figura 2.2 Uvicorn ejecutando FastAPI en el puerto 8011

Como era de esperar, esta respuesta es idéntica a la del listado 2.7 con datos diferentes, y ese es precisamente el punto. Crear nuevas rutas y controladores de métodos HTTP es así de fácil con FastAPI.

Al crear aplicaciones web, gran parte del diseño se basa en la ruta URL (p. ej., `/api/v1/hello-world/`) y los métodos HTTP (p. ej., GET, POST, PUT, DELETE, etc.). FastAPI facilita el manejo de ambas funciones.

Ahora que tenemos una comprensión básica de cómo crear una aplicación FastAPI, pasemos a crear una aplicación Node.js.

2.2 Creación de una aplicación web JavaScript con Node.js y Express.js

Express.js es un framework web para Node.js que facilita la creación de aplicaciones web. Express.js es prácticamente idéntico a FastAPI, salvo que, en lugar de usar Python, usaremos JavaScript. Pero ¿de qué tipo de JavaScript estamos hablando? Hay dos tipos: JavaScript basado en navegador y JavaScript del lado del servidor.

El JavaScript basado en navegador es el que se ejecuta en tu navegador. Es la forma en que podemos crear interfaces de usuario dinámicas y la razón por la que existen herramientas como React.js, Vue.js y Angular. Node.js y, por extensión, Express.js, no son herramientas de JavaScript basadas en navegador.

JavaScript del lado del servidor se conoce como Node.js y se ejecuta en el servidor. Al igual que Python es un entorno de ejecución para el lenguaje de programación Python, Node.js es un entorno de ejecución para el lenguaje de programación JavaScript.

Express.js, al igual que FastAPI, es un marco web del lado del servidor que nos ayuda a crear aplicaciones web con una sobrecarga mínima. Para utilizar Express.js, necesitaremos tener instalado lo siguiente:

- Node.js versión 16.14 o superior: si necesita una referencia para instalar Node.js, revise el apéndice B.
- El paquete Node npm: este es un paquete Node.js integrado para instalar y administrar paquetes de terceros (muy similar al paquete pip de Python).

Suponiendo que tiene Node.js en su máquina, creemos una carpeta en la siguiente lista para nuestro proyecto JavaScript e instalemos Express.js.

Listado 2.11 Creación de una carpeta para el código JavaScript de este capítulo

```
mkdir -p ~/Dev/roadtok8s/js/src/ cd ~/Dev/  
roadtok8s/js/
```

El Gestor de Paquetes de Node (npm) aísla el código en el directorio local por defecto. En otras palabras, no necesita activar nada para instalar paquetes en su directorio local. Además, npm registrará automáticamente los paquetes que instale en un archivo llamado package.json.

Para iniciar un nuevo proyecto de Node.js, lo habitual es ejecutar npm init en la carpeta raíz del proyecto, ya que esto facilitará la configuración inicial. Responda las preguntas que le solicite según sus necesidades. Comience con la siguiente lista.

Listado 2.12 Inicialización de un nuevo proyecto Node.js

```
npm init
```

Después de completar este proceso, verá el resultado que se muestra en la siguiente lista.

Listado 2.13 Salida de npm init

A punto de escribir en /Users/justin/Dev/roadtok8s/js/package.js

norle:

```
{
  "nombre": "js",
  "versión": "1.0.0", "descripción":
  "", "principal": "index.js",
  "scripts": { "prueba": "echo
  \"Error: no se
    especificó ninguna prueba\" && salida 1" }, "autor": "", "licencia": "ISC"
}
```

En este caso, normalmente cambio "name": "js" por "name": "roadtok8s" porque js es un nombre demasiado genérico.

También puedes ver estos datos en package.json ejecutando:

```
paquete cat.json
```

A partir de ahora, al ejecutar npm install <package-name>, npm agregará automáticamente el paquete a package.json, lo que le ahorrará la molestia de agregarlo manualmente. Instalemos Express.js en la siguiente lista.

Listado 2.14 Instalación de Express.js

```
npm instalar express
```

Una vez que esto se complete, podemos revisar package.json y ver que npm ha agregado Express.js a nuestro proyecto, como se muestra en la siguiente lista.

Listado 2.15 Salida de package.json

```
...  
  "dependencias":  
    { "express": "^4.18.2"#1  
    }  
...
```

#1 Tu versión de Express.js puede ser diferente

Como vemos, npm cuenta con un método estandarizado para instalar y rastrear las dependencias de los paquetes. Este método es un poco más rígido que el de pip de Python, pero sin duda reduce la complejidad de gestionar entornos de desarrollo uno mismo o de depender de una herramienta externa. Ahora, verifique el contenido de `~/Dev/roadtok8s/js/`, como se muestra en la siguiente lista.

Listado 2.16 Enumere el contenido del directorio node-web-app

```
ls ~/Dev/roadtok8s/js/
```

Como mínimo, lo siguiente debe aparecer en nuestro directorio:

- `node_modules/`—Contiene todos nuestros paquetes de terceros. Considere esto como una carpeta que puede eliminar en cualquier momento y reinstalar con npm
- `install. package-lock.json`: este es un archivo generado automáticamente a partir de `package.json` con información más detallada sobre nuestro proyecto.
- `package.json`: contiene las versiones de todos los paquetes de terceros instalados, el nombre de nuestro paquete (o proyecto), la versión de nuestro paquete y cualquier script personalizado que podamos agregar.

Ahora que nuestro entorno está configurado para nuestro proyecto Node.js, creemos nuestro primer módulo para nuestra aplicación web.

2.2.1 Hola mundo desde Express.js

Al igual que con FastAPI, comenzaremos con una aplicación sencilla de "Hola Mundo" con Express.js. Para ello, necesitaremos crear un archivo llamado `main.js` en el directorio de código fuente (`src`) de la carpeta de nuestro proyecto JavaScript (`~/dev/roadtok8s/js/`).

A diferencia de FastAPI, Express.js no utiliza un servidor web externo como `uvicorn`, por lo que usaremos una función integrada de Express.js para escuchar en un puerto específico de nuestra máquina. El proceso es el siguiente:

1. Importe los módulos necesarios para Express.js, el sistema de archivos y la ruta.
2. Cree una instancia del módulo Express.js.
3. Defina un puerto predeterminado para que la aplicación web escuche o vincúlelo a una variable de entorno para `PUERTO`.
4. Cree una función de devolución de llamada para manejar una ruta URL HTTP y método HTTP específico.
5. Defina una función de inicio de la aplicación para escuchar un puerto, enviar el ID del proceso a un archivo local y enviar un mensaje a la consola.

La siguiente lista muestra cómo se ve ese código.

Listado 2.17 Módulo Express.js en main.js

```

const express = require('express'); const fs =           // #1
require('fs'); const path =
require('path') const app = express();
const port = process.env.PORT
|| 3000; // #2

app.get('/', (req, res) =>
  { res.send("<h1>¡Hola Mundo Express!</h1>");
})

aplicación.listen(puerto, () => {
  const appPid = path.resolve(__dirname, 'app.pid')
  fs.writeFileSync(appPid, `${process.pid}`); console.log(`Servidor
ejecutándose en el puerto http://127.0.0.1:${port}`); })

```

#1 Usa require() y no import. Esto es propio de Node.js.

#2 Se requiere un valor de PUERTO predeterminado para ejecutar esta aplicación.

El formato que Express.js utiliza para manejar cualquier ruta URL HTTP, método HTTP y función de devolución de llamada es el que se muestra en la siguiente lista.

Listado 2.18 Manejo de HTTP de Express.js

```

aplicación.<métodohttp>(<cadenaDeRuta>, <devoluciónDeLlamada>)

```

- <httpMethod>: Este es el método HTTP que gestionará la función de devolución de llamada correspondiente. En main.js, este es simplemente el método GET. <pathString>:
- Esta es una cadena que representa la ruta URL que gestionará la función de devolución de llamada correspondiente. En el listado 2.17, solo gestionamos la ruta de índice (/).

- `<callback>`—Esta es otra función que se llamará cuando se realice una solicitud HTTP a la ruta URL correspondiente (por ejemplo, `/`).

Ahora que tenemos nuestro módulo `main.js` configurado y listo para aceptar una solicitud de URL y ejecutarse en un puerto específico, ejecutémolo y veamos qué sucede.

2.2.2 Ejecutar la aplicación Node.js + Express.js

Ejecutar JavaScript en la línea de comandos del backend es tan sencillo como ejecutar `node <module>`. En este punto, accederemos al código fuente de `Express.js` y ejecutaremos `node main.js` para iniciar nuestra aplicación. Por defecto, nuestra aplicación `Express.js` escuchará en el puerto 3000.

Listado 2.19 Ejecución del servidor web `Express.js`

```
cd ~/Dev/roadtok8s/js/src node  
main.js
```

Este comando debería mostrar en la terminal el siguiente mensaje:
«Servidor ejecutándose en el puerto `http://127.0.0.1:3000`». Abramos esa URL en nuestro navegador web y veamos (figura 2.3).



Figura 2.3 `Express.js` ejecutándose en `http://127.0.0.1:3000`

Para ver el código fuente HTML, en un navegador web, busque "ver código fuente" de la página en las herramientas para desarrolladores para ver el resultado, como en la figura 2.4.

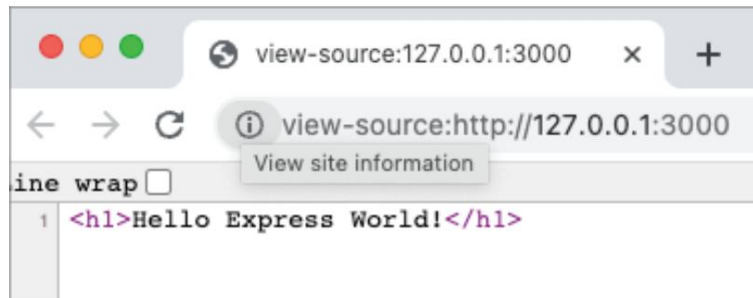


Figura 2.4 Código fuente HTML de `http://127.0.0.1:3000`

El código fuente nos muestra dos cosas sobre nuestra aplicación Express.js:

- Express.js puede devolver HTML fácilmente.
- El navegador seguirá procesando HTML con un formato incorrecto.

El puerto en el que ejecutamos nuestras aplicaciones cobra cada vez más importancia al comenzar a implementarlas. Con esto en mente, cambiemos el puerto en el que se ejecuta nuestra aplicación Express.js.

Presione Control + C y agregue la variable de entorno PORT de 3011 para probar un nuevo puerto, como se muestra en la siguiente lista.

Listado 2.20 Ejecución del servidor web Express.js en un nuevo puerto

```
# macOS o Linux
PUERTO=3011 nodo main.js

# Windows
establece el puerto=3011 y el nodo main.js
```

La salida de la consola debería reflejar ahora el nuevo puerto: Servidor ejecutándose en el puerto `http://127.0.0.1:3011`. Abra su navegador en esta nueva URL y observe qué sucede (figura 2.5).

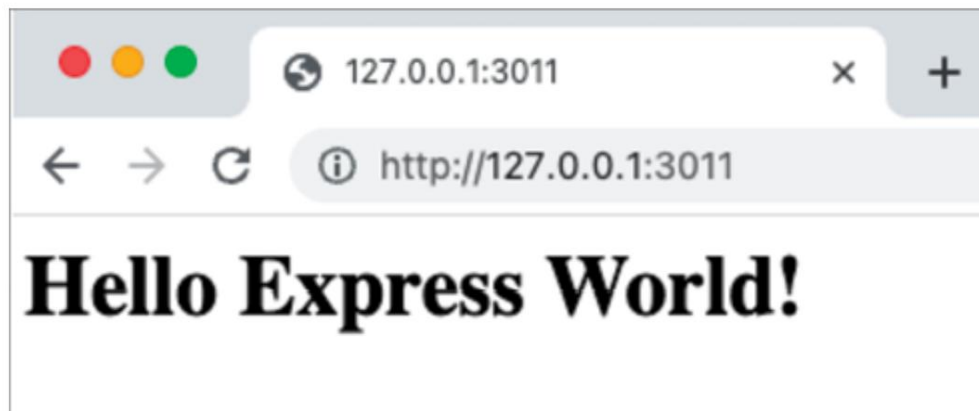


Figura 2.5 Express.js ejecutándose en <http://127.0.0.1:3011>

¿Qué pasaría si intentaras visitar <http://127.0.0.1:3000> de nuevo? ¿Sucede algo? Estas son las primeras etapas para ver cuánto afecta el puerto que elijas a nuestras aplicaciones y a nuestra capacidad de acceder a él.

Opciones para detener/cancelar la ejecución del servidor Node.js:

- Salir de la línea de comandos / ventana de terminal (lo más fácil).
- Utilice Control+C en la terminal donde se ejecuta el programa (recomendado).
- Utilice `kill -9 <PID>` donde `<PID>` es el ID del proceso de la instancia Node.js en ejecución (avanzado).

Ahora que podemos representar respuestas HTML con Express.js, creemos una respuesta JSON utilizando las funciones integradas de JavaScript.

2.2.3 Crear una respuesta JSON con Express.js

Express.js se usa a menudo como microservicio, lo que significa que es una excelente manera de que el software se comunique con el software, y para eso sirven precisamente los datos JSON. Hay dos maneras de devolver datos JSON con Express.js:

- Usando `res.json()`
- Uso de encabezados

Dejaremos el uso de encabezados para otra ocasión y optaremos por la versión mucho más sencilla de `res.json()`. Para devolver datos JSON en `Express.js`, necesitamos hacer lo siguiente:

- Declarar un objeto JavaScript (o recuperar uno de una base de datos).
- Convierte el objeto en JSON usando `JSON.stringify()`.
Curiosamente, los objetos JavaScript no son lo mismo que las cadenas JSON.
- Devuelve la cadena JSON usando `res.json()`.

Aquí está la función de devolución de llamada resultante para nuestra nueva ruta en la siguiente lista.

Listado 2.21 Agregar una ruta en `main.js`

```
aplicación.get('/api/v2/rocket-man1/', (req, res) => {  
  const myObject = {quién: "hombre cohete", dónde: "marte"}; const  
  jsonString = JSON.stringify(myObject); res.json(jsonString);  
})
```

Con el nodo `main.js` aún ejecutándose, navegue a nuestra nueva ruta y debería ver el mismo resultado de la figura 2.6.



Figura 2.6 Respuesta JSON de Node.js

Ahora debería tener una aplicación Express.js funcional que pueda manejar respuestas JSON y HTML, lo cual es un excelente comienzo para nuestras aplicaciones. Como con muchas cosas, podríamos dedicarle un libro entero a Express.js, pero lo dejaremos aquí por ahora.

Con ambas aplicaciones, debemos empezar a rastrear los cambios en el código y a enviarlos a un repositorio remoto. Cuanto antes se acostumbre a hacerlo, mejor.

2.3 Seguimiento de cambios de código con Git

Git es una herramienta de código abierto que tiene una serie de características que hacen que actualizar, compartir y rastrear código sea mucho más fácil. Git es un tipo de sistema de control de versiones al que a menudo se denomina control de código fuente porque rastrea el código fuente de muchas aplicaciones.

Git es sin duda el sistema de control de versiones más popular del mundo y es una herramienta fundamental que impulsa la tecnología de código abierto. Usé Git para ayudar a escribir muchos...

Iteraciones guardadas de este libro. Las principales características de Git en las que nos centraremos en este libro son

- Seguimiento de cambios en nuestro código
- Ignorar ciertos tipos de archivos
- Revertir a versiones anteriores de nuestro código
- Empujar (cargar) nuestro código a una ubicación de almacenamiento no local (llamada repositorio remoto)
- Extraiga (descargue) nuestro código cuando entremos en producción
- Activar flujos de trabajo para utilizar nuestro código (como CI/CD y otras automatizaciones)

Estas funciones se utilizarán en varios momentos a lo largo del libro para ayudarnos continuamente con la implementación en general y nuestro viaje hacia Kubernetes.

Asegúrese de instalar Git en su equipo local antes de continuar.

Visite <https://git-scm.com/downloads> Porque es bastante sencillo de instalar y empezar. Para verificar que Git esté instalado, ejecute el siguiente comando en su línea de comandos.

Listado 2.22 Verificación de que Git está instalado

```
git --versión
```

Esto debería responder con un número de versión (p. ej., Git versión 2.30.1) y no con un error. Si recibe un error, deberá instalar Git como se mencionó anteriormente.

¿QUÉ ES EL CONTROL DE VERSIONES?

Escribir en un cuaderno y hacer copias para un archivador es una forma tradicional de controlar versiones.

Tomemos un ejemplo moderno pero sencillo: Control + Z (o Comando + Z en macOS). Si eres como yo, te encantará el atajo de teclado "Deshacer" y probablemente lo uses a menudo. Es una forma de control de versiones, ya que revierte el texto a un estado histórico. Rehacer (Control + Mayús + Z o Comando + Mayús + Z en macOS) también es una forma de control de versiones, ya que avanza en el tiempo a un estado más reciente del texto.

¿Qué tan difícil sería la vida sin deshacer y rehacer? Ni me lo imagino. Si creciste a finales del siglo XX, quizá recuerdes lo difícil que era antes de que existieran, porque quizás usabas una máquina de escribir de verdad con corrector líquido en tus documentos.

El control de versiones de documentos lleva este concepto a un nivel completamente nuevo y realmente libera el potencial de cómo las personas y los equipos pueden escribir código de manera efectiva.

El control de versiones es muy similar a los atajos de teclado de deshacer y rehacer, pero se aplica a archivos y carpetas completos. Hay una trampa: para usar el control de versiones, necesitas lo siguiente:

- Una herramienta instalada que está diseñada para el control de versiones
- Para escribir los comandos correctos para utilizar la herramienta
- Para recordar cuándo y cómo utilizar la herramienta

A lo largo de los años, han surgido numerosas herramientas para facilitar el control de versiones. La herramienta que usaremos es también la más popular: Git.

Para gestionar nuestros proyectos basados en Git, utilizamos un repositorio remoto. En este libro, utilizaremos dos tipos de repositorios remotos: autogestionados y gestionados por terceros. En el siguiente capítulo, implementaremos un repositorio autogestionado para facilitar la implementación manual.

GitHub.com es una herramienta de repositorio remoto gestionado por terceros que permite almacenar código en la nube, compartirlo con otros y activar diversas canalizaciones de automatización, que analizaremos con más detalle a lo largo de este libro. GitLab es otro repositorio remoto popular que puede ser gestionado por terceros o por uno mismo, pero queda fuera del alcance de este libro. Ahora, actualicemos nuestros proyectos para empezar a usar Git.

2.3.1 Usando Git en nuestros proyectos

Comenzar a usar Git es un proceso sencillo. Comienza indicándole a Git que inicie el seguimiento de cambios en cada uno de los directorios del proyecto que creamos en este capítulo: ~/Dev/roadtok8s/py/ y ~/Dev/roadtok8s/js/. Este proceso se conoce como inicializar un repositorio Git. Comencemos con nuestra aplicación de Python en la siguiente lista.

Listado 2.23 Uso de git init para inicializar un repositorio Git

```
cd ~/Dev/roadtok8s/py/ git init
```

Nuestra aplicación Node.js se muestra en la siguiente lista.

Listado 2.24 Uso de git init para nuestra aplicación Node.js

```
CD ~/Dev/roadtok8s/js/git init
```

Ambos responderán con algo como Inicializado . ., nombrando la ruta a su repositorio. repositorio Git vacío en .

Fácil, ¿verdad? Vemos que el comando git init crea un repositorio (repo , para abreviar) dentro del directorio donde se ejecuta git init .

Ahora tenemos dos repositorios en nuestro directorio ~/Dev/roadtok8s/: py y js. Actualmente, estos repositorios solo se almacenan en nuestra máquina local. Pronto los configuraremos para que se almacenen en un repositorio remoto.

Si ejecutas git init en un directorio que ya tiene un repositorio, recibirás un mensaje como "Reinitialized existing Git". He visto este resultado en muchos . . repositorios. Seguro que tú también.

Si no sabe si su directorio tiene un repositorio, simplemente ejecute git status. Si ve fatal: not a git repository, tiene la respuesta. Si ve casi cualquier otra cosa, es muy probable que la carpeta ya esté en un repositorio de Git.

Amplíemos un poco el estado de Git para comprender mejor lo que hace.

ARCHIVO Y ESTADO DE GIT

Git es una herramienta pasiva que requiere la intervención del usuario, lo que significa que debes decirle exactamente cuándo ocurrió algo y qué quieres que se haga ante ese cambio.

Hasta ahora, solo le hemos indicado a Git que exista en las carpetas de nuestro proyecto (como repositorio). Antes de empezar a indicarle que rastree los archivos, usaremos el comando git status dentro del directorio de nuestro proyecto de Python. git status generará lo que vemos en la figura 2.7.

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    src/
    venv/

nothing added to commit but untracked files present (use "git add" to track)
```

Figura 2.7 Estado de Git para nuestro proyecto Python

La figura 2.7 nos muestra algunas cosas:

- Estamos trabajando en una rama de Git llamada "main" o "master", según tu configuración. Las ramas permiten gestionar diferentes versiones de tu código. La ramificación en Git queda fuera del alcance de este libro.
- "Aún no hay confirmaciones" significa que aún no le hemos dicho a Git que rastree ningún cambio en nuestro código.
- Los archivos no rastreados proporcionan una lista de carpetas (que incluyen archivos) junto con archivos que podríamos rastrear o ignorar (agregándolos a un archivo .gitignore)

Si repetimos este proceso para nuestra aplicación Node.js, deberíamos ver aproximadamente el mismo resultado del estado de git como se ve en la figura 2.8.

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    node_modules/
    package-lock.json
    package.json
    src/

nothing added to commit but untracked files present (use "git add" to track)
```

Figura 2.8. Estado de Git para nuestro proyecto Node.js

Algunos editores de texto (como VSCode) facilitan mucho el uso de Git gracias a sus funciones integradas. Si en algún momento Git te resulta demasiado complejo para tu nivel actual, te recomiendo descargar VSCode y revisar cómo usar Git con VSCode.

En ambos repositorios tenemos una lista de archivos sin seguimiento. Antes de comenzar a rastrear cualquiera de estos archivos, en realidad queremos ignorar la gran mayoría de ellos, gracias a los paquetes de terceros almacenados en `venv/` y `node_modules/`.

2.3.2 Ignorar archivos

Hay dos carpetas enormes en cada uno de nuestros repositorios (directorios de proyecto) que nunca queremos rastrear con Git. Estas carpetas son

- `venv` de Python /
- `node_modules` de Node.js /

Estos directorios pueden ser enormes porque son paquetes de terceros que pueden o no tener mucho peso (archivo

guardados, número de archivos, etc.) así como una serie de dependencias propias.

Ambos proyectos tienen referencias a los paquetes de terceros que se han almacenado en `venv/` y `node_modules/`.

Estas referencias son `requirements.txt` y `package.json`, respectivamente. Estas carpetas se pueden eliminar y recuperar en cualquier momento simplemente ejecutando `python -m pip install -r requirements.txt` y `npm install`, respectivamente.

Esperamos que los paquetes de terceros que usamos también tengan repositorios Git accesibles para todo el mundo; así, obtener copias de ellos es bastante sencillo. En otras palabras, dejaremos que los paquetes de terceros sean responsables de su propio código. Si bien este principio tiene sus limitaciones, estas quedan fuera del contexto de este libro.

Para ignorar estas carpetas y cualquier otro archivo, necesitamos crear un archivo `.gitignore` en la raíz de nuestro repositorio (junto a donde ejecutamos `git init`). Observa que el nombre del archivo empieza con un punto (.) y suele llamarse `punto-git-ignore`.

Primero, revisemos qué hay en nuestra carpeta/directorio de la aplicación web de Python:

```
cd ~/Dev/roadtok8s/py/ ls -aa #1
```

#1 Los usuarios de Windows pueden usar `dir /a` en lugar de `ls -aa`.

Esto debería generar

```
.git      origen  venv
```

Si no ves `.git`, cometiste algún error durante la fase de inicialización de Git. El directorio `.git` es donde se almacena toda la información de seguimiento de Git.

Si ejecutamos `git status` en este momento, veremos que `venv` aparece en los archivos sin seguimiento. Crearemos un archivo `.gitignore` específicamente para nuestro proyecto de Python para evitar que `venv` sea rastreado. Asegúrese de que `.gitignore` esté en la misma carpeta o directorio que el directorio `.git`, como se muestra en la siguiente lista.

Listado 2.25 Ejemplo de `gitignore` en Python

```
venv/
.env
.venv

__pycache__/
*.py[cod]
*$py.class

.DS_Store
```

Este es un ejemplo sencillo de un archivo `.gitignore` de Python.

Recomiendo usar el archivo `.gitignore` oficial de Python en GitHub ([https://](https://github.com/github/gitignore/blob/main/Python.gitignore)

github.com/github/gitignore/blob/main/Python.gitignore).

Ejecuta `git status` y verifica que el directorio `venv` ya no exista. Si sigues viendo `venv`, significa que `.gitignore` no está guardado o está en una ubicación incorrecta.

Creemos otro archivo `.gitignore` simple para nuestro proyecto local en la siguiente lista. Al igual que con Python, recomiendo usar el archivo `.gitignore` oficial de node en GitHub (<https://github.com/github/gitignore/blob/main/Node.gitignore>).

Listado 2.26 Ejemplo de .gitignore en Node.js

```
módulos_de_nodo/
.env
.venv

registros *.log npm-debug.log*

.DS_Store
```

Nuevamente, ejecute `git status` y verifique que la carpeta `node_modules` ya no aparezca en la lista. Si aún ve `node_modules`, significa que `.gitignore` no está guardado o está en una ubicación incorrecta. Ignorar archivos es fundamental para evitar que nuestro repositorio remoto sea demasiado grande o que carguemos información confidencial accidentalmente.

2.3.3 Archivos de seguimiento

El seguimiento de archivos es un proceso mayoritariamente manual y debe realizarse con frecuencia para ayudarnos a comprender los cambios que hemos realizado con el tiempo. Este proceso de seguimiento es fácil de perder de vista para un principiante, ya que podría pensar: "Ya guardé el archivo. ¿Por qué debería también rastrearlo?".

Guardar el archivo y rastrear los cambios son dos cosas muy diferentes. El rastreo indicará qué se añadió o eliminó de un archivo (o directorio) en distintos momentos, según cuándo decidas rastrear los cambios con Git. Para comprender mejor este proceso, comencemos a rastrear los archivos de nuestro proyecto de Python. Primero, navegaremos a nuestro repositorio local y luego agregaremos cada archivo que queramos rastrear con el comando `git add <path>`.

git add hace algunas cosas:

- Prepara Git para los archivos/carpetas que queremos rastrear correctamente ahora.
- Toma los archivos o archivos dentro de las carpetas en su estado guardado actual (como lo que realmente contienen cuando ejecuta `git add my/file/path.txt`).
- Le permite “desprepararlos” con `git reset <path>` como `git reset src/main.py`.
- Todavía no rastrea, guarda ni consolida los cambios. Esto se hará más adelante.

La siguiente lista muestra un ejemplo de seguimiento de los archivos de nuestro proyecto Python.

Listado 2.27 Uso de git add para rastrear archivos de Python

```
cd ~/Dev/roadtok8s/py/ git add  
src/main.py git add src/  
requirements.txt
```

Si consideramos `git add <path>` como `git prepare <path>`, vamos por buen camino. El objetivo de este paso es asegurarnos de que estamos rastreando los archivos que queremos rastrear, no los que no queremos rastrear.

Considero que Git es una forma de rastrear cambios incrementales interconectados de alguna manera. Git add es una forma sencilla de garantizar que solo se incluyan los archivos y carpetas que están interconectados de alguna manera. En otras palabras, si añado dos funciones que no están relacionadas, debería considerar añadir las y rastrear sus cambios por separado. Es una buena práctica, ya que te ayudará a ti y a otros a comprender los cambios que has realizado con el tiempo.

Para finalizar el proceso de seguimiento, necesitamos confirmar los cambios realizados. Confirmar es como decir: "Sí, esos archivos que queremos rastrear, rastreémoslos ahora". El comando es `git commit -m "mi mensaje sobre estos cambios"`.

`git commit` hace algunas cosas:

- Finaliza, solidifica o guarda los cambios desde la última confirmación en función de lo que se agregó con `git add`
- Toma una instantánea de los archivos o archivos dentro de las carpetas que agregamos con `git add`
- Requiere un mensaje para describir qué sucedió con estos archivos agregados desde la última confirmación

A continuación se muestra un ejemplo práctico de cómo confirmar los archivos de nuestro proyecto Python:

```
git commit -m "Confirmación inicial del proyecto Python"
```

De ahora en adelante, cada vez que realices cambios incrementales en tu aplicación, deberás agregarlos y confirmarlos en tu repositorio. Puedes recordar el proceso así:

1. Realizar cambios en archivos o carpetas de nuestro proyecto.
2. Pregunte: "¿Hay archivos que no necesitan seguimiento?" Si Entonces, actualiza `.gitignore`.
3. Prepare Git para los cambios con `git add`.
4. Solidifique los cambios junto con un mensaje sobre los cambios `git commit -m "mi mensaje sobre estos cambios"`.
5. En un próximo capítulo, ejecutaremos el proceso de carga. nuestro código en otro lugar con `git push`.

Omití a propósito algunas funciones de Git (como Git Stash o Git Checkout) para centrarme principalmente en los fundamentos del seguimiento de archivos. Recomiendo leer la documentación oficial de Git para obtener más información sobre estas funciones: <https://git-scm.com/docs>.

Pongamos nuestras habilidades en práctica y hagamos lo mismo con los archivos de nuestro proyecto Node.js en la siguiente lista.

Listado 2.28 Uso de git add para rastrear archivos Node.js

```
cd ~/Dev/roadtok8s/js/ git
add src/ git add
paquete.json git add
paquete-lock.json
```

Como vemos aquí, git add src nos muestra cómo rastrear un directorio completo, lo que agregará todos los archivos y subcarpetas, así como los archivos de estas, dentro de src/, a menos que estén en .gitignore. Podemos confirmar nuestros cambios confirmándolos con el comando git commit -m "Initial node.js project commit".

¡Excelente trabajo! Ahora Git rastrea nuestros archivos. Si alguna vez hacemos cambios en el futuro, el proceso es muy sencillo.

1. Agregamos el archivo que queremos rastrear con git add ruta/relativa/al/archivo.
2. Confirmar nuestros cambios con git commit -m "Commit mensaje".

O podemos hacerlo mediante

1. Agregamos la carpeta que queremos rastrear con git add relative/path/to/folder/.
2. Confirmar nuestros cambios con git commit -m "Commit mensaje".

Puedes agregar y confirmar archivos con la frecuencia que desees. Como regla general, recomiendo confirmar con la mayor frecuencia posible y un mensaje de confirmación que describa los cambios realizados.

Por ejemplo, supongamos que nuestro proyecto de Python necesita una nueva ruta URL. Podemos crearla en nuestro módulo de Python y luego notificar a Git sobre la nueva ruta con un mensaje como "Nueva ruta URL añadida para renderizar contenido HTML".

Otro buen ejemplo sería si quisiéramos gestionar un nuevo método HTTP para la ingesta de datos en Node.js. Podríamos crear esa nueva URL en nuestro módulo JavaScript y confirmarla con un mensaje como "soporte para la ingesta de datos del formulario de contacto mediante una solicitud POST".

Un mal ejemplo sería si agregáramos un montón de características a nuestro proyecto en un montón de archivos diferentes y simplemente agregáramos un mensaje con "cosas cambiadas".

Estos ejemplos no pretenden ser prescriptivos, sino más bien ilustrar la importancia de escribir buenas prácticas de confirmación acompañadas de mensajes de confirmación detallados.

2.3.4 Reinicio completo del repositorio Git

Veo una y otra vez que los principiantes abandonan el uso correcto de Git simplemente porque cometieron algunos errores con sus repositorios. Cuando esto sucede, recomiendo reiniciar todo el repositorio. Esta no es una buena práctica en general, pero es excelente para quienes aún estamos aprendiendo Git y muchas otras herramientas de implementación.

Lo que sugiero es que reiniciemos nuestros repositorios Git eliminando todo rastro de nuestros antiguos repositorios Git y comenzando desde cero. Hacer esto crea problemas innecesarios que pueden ser...

Se puede evitar aprendiendo más sobre Git, pero es una excelente manera de continuar aprendiendo muchas de las otras tecnologías de este libro. Git es, sin duda, una herramienta que recomiendo mejorar, y si usas las herramientas de este libro, probablemente lo harás. Git no debería ser la razón por la que tu progreso en el aprendizaje sobre cómo implementar software se detenga.

Veamos un ejemplo de cómo podemos reiniciar nuestro repositorio Git para nuestro proyecto Python. Este proceso es el mismo para nuestro proyecto Node.js:

1. Navegue hasta el directorio de nuestro proyecto Python (`cd ~/Dev/roadtok8s/py/`).
2. Destruya la carpeta `.git` : macOS/
 - Linux: `rm -rf .git`. El comando `rm -rf` eliminará carpetas completas y todos los archivos que contengan.
 - Windows: `rmdir /S .git`. El comando `rmdir /S` (S mayúscula) eliminará carpetas enteras y todos los archivos que contienen.
 - Nota: No utilices estos comandos a la ligera porque pueden volverse en tu contra si los ejecutas en la carpeta incorrecta.
3. Verifique que la carpeta `.git` haya desaparecido con `ls -aa` (macOS/Linux) o `dir /a` (Windows).

Con la carpeta Git destruida, podemos reiniciar nuestro repositorio Git. ¿Recuerdas cómo se hace? Aquí tienes una lista con viñetas para que te sea más fácil recordarlo:

- Navegue a la carpeta que desea rastrear (`cd ~/Dev/roadtok8s/py/`).
- Inicializar el repositorio Git (`git init`).
- Crea un archivo `.gitignore` y agrega los archivos o carpetas que necesitas ignorar.

- Agregue archivos o carpetas que desea rastrear (`git add relative/path/to/file`).
- Confirme sus cambios (`git commit -m "Mensaje de confirmación"`).
- Sube tu código cuando esté listo. Si borraste tu historial de Git, podrías tener que añadir `--force` a tus envíos, como `git push origin main --force`, para forzar que los cambios se envíen al repositorio remoto. Forzar el envío con `--force` sobrescribirá cualquier cambio que se haya realizado en el repositorio remoto y debe usarse con precaución.

Te animo a practicar este proceso de destruir y recrear tu repositorio de Git para familiarizarte con Git y el proceso de seguimiento de archivos. Antes de subir nuestro código a un repositorio remoto como GitHub, repasemos otra característica importante de Git: el registro de Git.

2.3.5 Registros y visualización de Git

Reiniciar un repositorio Git es una excelente manera de practicar Git y familiarizarse con el proceso de seguimiento de archivos. Sin embargo, tiene una desventaja: perdemos nuestro historial de confirmaciones. Esta sección nos ayudará a comprender mejor cómo podemos revertir a versiones anteriores de nuestro código, por lo que quizás tengamos que reiniciar nuestros repositorios cada vez con menos frecuencia.

Una de las razones para escribir buenos mensajes de confirmación es ayudarnos a comprender lo que hicimos en el pasado y, potencialmente, a revertirlo a un estado anterior. Para ello, podemos revisar nuestro historial de confirmaciones con Git Log. Consulta la siguiente lista.

Listado 2.29 Revisión de nuestro registro del historial de confirmaciones de Python

```
cd ~/Dev/roadtok8s/py/git registro
```

Si estamos trabajando en un repositorio que tiene un historial de confirmaciones, nuestra salida debería parecerse a la figura 2.9.

```
commit 278484deaddcaf231e099b5684734f87f0f7366b (HEAD -> main)
Author: Justin Mitchel <my-email>
Date:   Wed Nov 23 15:50:28 2022 -0600

    Initial Python commit
```

Figura 2.9 Registro de confirmación inicial de Git para nuestro proyecto Python

Dentro de esta única entrada de registro, vemos algunos datos importantes:

- `commit <commit-id> (HEAD → main)`: este es el ID de confirmación para la rama en la que estamos (por ejemplo, principal o posiblemente maestra).
- **Autor**: este es el autor configurado para esta confirmación. Esto es genial para los equipos.
- **Fecha**: esta es la fecha y hora de la confirmación misma. Esta es otra característica excelente para uso personal y en equipo, ya que nos permite verificar cuándo se realizó un cambio.
- **Mensaje de confirmación**: Finalmente, vemos nuestro mensaje de confirmación (p. ej., "Confirmación inicial de Python"). Incluso desde el primer mensaje, podemos ver cómo lo que escribimos aquí puede ayudarnos a comprender lo que sucedió en esta confirmación y a profundizar en el tema.

Como podemos ver, el mensaje de confirmación puede desempeñar un papel crucial al revisar cualquier punto de nuestro historial. Mejores mensajes de confirmación son mejores para todos, ¡incluso para una versión futura de ti! Nuestro ID de confirmación (p. ej., "278484deaddcaf231e099b5684734f87f0f7366b") puede ser

revisado ingresando el comando `git show <commit-id>` como se muestra en la siguiente lista.

Listado 2.30 Ejemplo de presentación de Git

```
git muestra 278484deaddcaf231e099b5684734f87f0f7366b
```

Esto me dará todos los archivos que agregué al commit con los cambios asociados, como se ve en la figura 2.10.

```
diff
new file mode 100644
index 0000000..0c51dbc
--- /dev/null
+++ b/src/main.py
@@ -0,0 +1,20 @@
+import pathlib
+from fastapi import FastAPI, Request
+from fastapi.responses import HTMLResponse
+from fastapi.templating import Jinja2Templates
```

Figura 2.10 Ejemplo de presentación de Git

Aunque esto sea un poco difícil de leer, analicémoslo un momento: vemos exactamente qué ha cambiado en cada archivo en un momento dado. El signo `+` indica una línea añadida y el signo `-` indica una línea eliminada (lo cual no se muestra en esta confirmación).

Esto es increíblemente poderoso y realmente puede ayudarnos a crear un código mejor y más fácil de mantener.

Si necesitamos volver a este punto en el tiempo, podemos hacerlo con el comando `git reset <commit-id>`. `git reset` (y `git reset <commit-id> --force`) deben usarse con moderación porque literalmente pueden borrar mucho de su arduo trabajo sin usar otras características de Git (por ejemplo, ramas).

Si necesita volver a versiones anteriores de su código, siempre recomiendo almacenar su código en otra computadora (llamado repositorio remoto) para que siempre pueda volver a

(y) yyg it. Lo abordaremos en la siguiente sección. Antes de enviar nuestro código a otro lugar, repasemos algunos comandos fundamentales de Git .

2.3.6 Descripción general de los comandos fundamentales de Git

Es absolutamente cierto que Git puede volverse muy complejo, por eso quiero mostrarte algunos comandos clave para tu referencia:

- `git init`: inicializa un repositorio Git. `git add <ruta/relativa/al/archivo>`: agrega un archivo para rastrear.
- `git add <ruta/relativa/a/carpeta>`—Agrega una carpeta para rastrear.
- `git commit -m "Mensaje de confirmación"`: confirma o finaliza los cambios con un mensaje personalizado. `git push`: envía los cambios a un repositorio remoto. `git pull`: extrae los cambios de un repositorio remoto. `git log`: muestra todas las confirmaciones recientes.
- `git show <commit-id>`: muestra todos los archivos y cambios asociados con una confirmación.
- `git reset <commit-id>`—Revierte a una confirmación anterior. Omita `<commit-id>` y volverá a la confirmación más reciente.
- `git reset <commit-id> --hard`: Revierte a una confirmación anterior y borra todos los cambios realizados desde entonces. (Use esto con precaución).

Aprender y comprender Git requiere tiempo y práctica, así que te animo a revisar estos comandos con la mayor frecuencia posible. Ahora que podemos rastrear nuestro código, vamos a subirlo a...

repositorio remoto para que pueda alojarse lejos de nuestra máquina y potencialmente compartirse con otros.

2.4 Enviar código a GitHub

GitHub.com ha sido un aliado y pionero de la comunidad de desarrollo de software desde hace mucho tiempo. GitHub aloja código gratuito, tanto en modo privado como público, para millones de desarrolladores de todo el mundo. En esta sección, configuraremos nuestra cuenta de GitHub para alojar los dos proyectos que creamos en este capítulo.

Mantendremos esta parte altamente enfocada en enviar código a GitHub porque continuaremos usando cada vez más funciones de GitHub a lo largo de este libro.

Estos son los pasos que seguiremos:

1. Crea una cuenta de GitHub.
2. Crear un nuevo repositorio para cada una de nuestras aplicaciones.
3. Sube nuestro código a GitHub.

Esta sección trata sobre la implementación del comando `git push` en nuestro flujo de trabajo. Si aún no tienes una, crea una cuenta de GitHub.

2.4.1 Creación de una cuenta de GitHub

Para este libro, deberá registrarse en GitHub. Existen alternativas a GitHub, pero este libro se ha diseñado pensando en el almacenamiento de código en GitHub y en el uso de canalizaciones de automatización con GitHub Actions en capítulos posteriores. Las alternativas a GitHub, como GitLab, son

es perfectamente capaz de realizar estas actividades, pero configurarlas está fuera del alcance de este libro.

En el próximo capítulo, veremos cómo implementar código utilizando nuestro repositorio auto hospedado sin GitHub, para que pueda ver desde una perspectiva de primeros principios cómo funciona el hospedaje relacionado con Git.

Por ahora, crea una cuenta en GitHub en <https://github.com/signup>. Una vez que tenga una cuenta de GitHub, creemos un par de repositorios para alojar nuestro código.

2.4.2 Crear un repositorio Git en GitHub

Vamos a crear dos nuevos repositorios en GitHub, uno para cada aplicación. Se recomienda encarecidamente tener un repositorio por aplicación; de lo contrario, sus proyectos se volverán demasiado complejos.

Crea tu primer repositorio yendo a <http://github.com/new>. Si es la primera vez que creas un repositorio en GitHub, este enlace podría convertirse en tu nuevo mejor amigo.

Comience con su aplicación Python y la siguiente configuración. Tenga en cuenta que deje muchas opciones en blanco o sin marcar. Esto es intencional, ya que ya configuramos un repositorio local (figura 2.11).

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository](#).

Owner * **Repository name ***

 jmitchel3 / roadtok8s-py 

Great repository names are short and memorable. Need inspiration? How about [improved-lamp?](#)

Description (optional)

☒  **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐  **Private**
You choose who can see and commit to this repository.

Initialize this repository with:
Skip this step if you're importing an existing repository.

☐ **Add a README file**
This is where you can write a long description for your project. [Learn more](#).

Add .gitignore
Choose which files not to track from a list of templates. [Learn more](#).

.gitignore template: **None** ▼

Choose a license
A license tells others what they can and can't do with your code. [Learn more](#).

License: **None** ▼

 You are creating a public repository in your personal account.

[Create repository](#)

Figura 2.11 Creación de un repositorio en GitHub

Después de seleccionar Crear repositorio, se le dirigirá a la Página del nuevo repositorio. Verás una sección llamada Acceso rápido. Configuración que le brindará algunas opciones sobre cómo comenzar Con tu nuevo repositorio. Usaremos la opción ". push" . . 0 para un repositorio existente desde la línea de comandos.

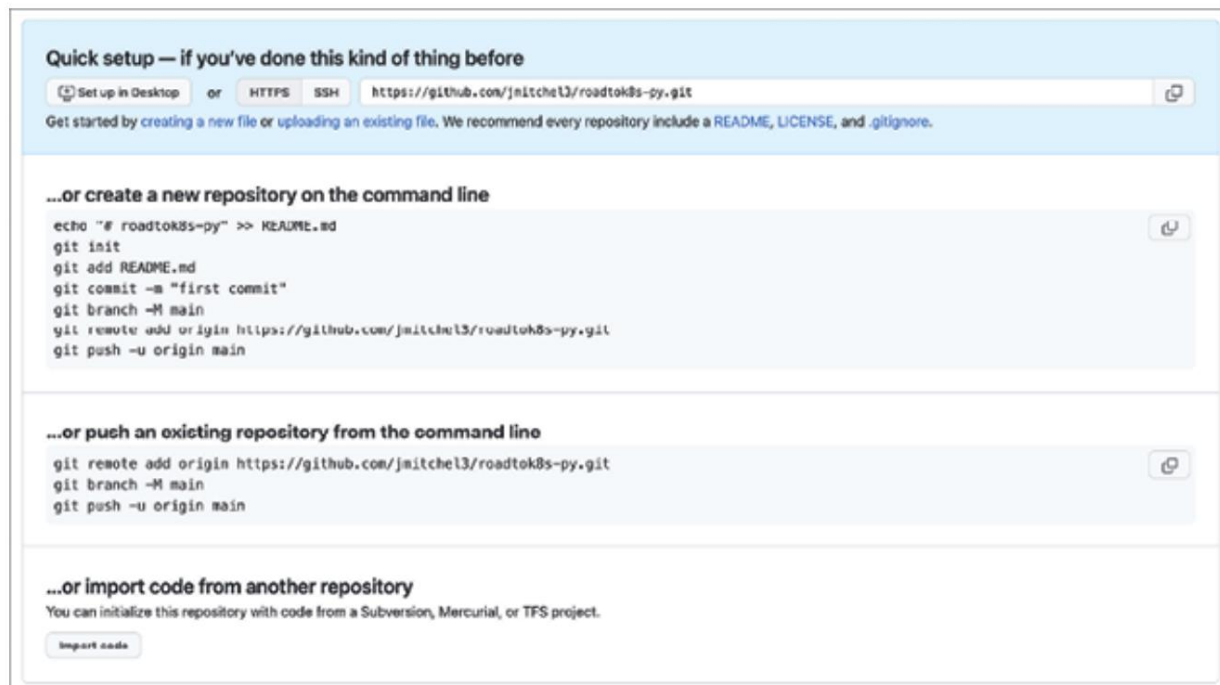


Figura 2.12 Un repositorio recién creado en GitHub para nuestra aplicación Python

Antes de entusiasmarse demasiado y comenzar a copiar y pegar código de GitHub, analicemos en profundidad lo que debe suceder para configurar nuestro repositorio Git local para enviarlo a este repositorio GitHub.

2.4.3 Configurando nuestro repositorio git local para GitHub

Antes de poder subir nuestro código a GitHub, debemos asegurarnos de que nuestro repositorio local de Git esté listo para subir código. Para ello, añadiremos un repositorio remoto a nuestro repositorio local de Git. Hay una línea clave en la sección "Configuración rápida" a la que debemos prestar atención (figura 2.13).

...or create a new repository on the command line

```
echo "# roadtok8s-py" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/jmitchel3/roadtok8s-py.git
git push -u origin main
```



Figura 2.13 adición remota a través de un nuevo repositorio de GitHub

¿Sabes por qué esta es la línea clave? ¡Es porque ya ejecutamos todos los demás comandos! Con la excepción de crear un archivo Readme.md, ejecutamos los siguientes comandos en la sección anterior:

- git init
- git add . git
- commit -m "Confirmación inicial"

Tendremos que seguir usando git add path/to/some/file/or/folder/ junto con git commit -m "Commit message", pero no tendremos que ejecutar git init o git remote add nuevamente.

Ejecutemos git remote add. Para ello, accedamos a la carpeta `cd ~/dev/roadtok8s/py/` de nuestro proyecto y ejecutemos `git remote add origin https://github.com/yourusername/roadtok8s-py.git` (reemplace `yourusername` con su nombre de usuario de GitHub), como recomienda GitHub. "origin" es el nombre que le damos al repositorio remoto, ya que "origin" es un nombre convencional para nuestro repositorio remoto principal. En el próximo capítulo, revisaremos este concepto con un nombre diferente para un repositorio remoto.

Ahora que hemos agregado el repositorio remoto, podemos enviar nuestro código a GitHub, como se muestra en la siguiente lista.

Listado 2.31 GitHub push

```
git push origen principal
```

¿Tuviste errores?

- Si encuentra un error que indica que main no es una rama válida, intente `git branch --show-current` para ver qué rama está usando.
- Si encuentra un error que indica que el origen no es un control remoto válido, intente `git remote -v` para ver qué controles remotos ha configurado.

¿Lo lograste? Si es así, ¡felicidades! ¡Acabas de subir el código de tu proyecto a GitHub! Estamos un paso más cerca de subir nuestro código a producción. Ahora es el momento de hacer lo mismo con nuestro otro proyecto.

2.4.4 Enjuagar y repetir

Repetir el proceso para nuestro proyecto Node.js debería ser sencillo en este momento, pero a modo de resumen, esto es lo que debes hacer:

1. Cree un nuevo repositorio en GitHub con solo el nombre `roadtok8s-js`.
2. Copie el comando `git remote add` desde GitHub.
3. Agregue el repositorio remoto a su repositorio local `cd git origin`
`~/dev/roadtok8s/js/ https:// && remoto agregar`
`github.com/yourusername/roadtok8s-js.git.`
4. Envía tu código a GitHub con `git push origin main`.

Usar Git es crucial para crear aplicaciones fáciles de mantener y que podamos implementar continuamente en producción. Este libro seguirá profundizando en los conceptos de Git y GitHub a medida que avancemos.

Resumen

- Tanto Python como JavaScript son excelentes opciones para aprender los conceptos básicos de la creación de aplicaciones web.
- Incluso las aplicaciones simples tienen muchas dependencias y partes móviles para ponerse en funcionamiento. El
- seguimiento de paquetes de terceros con el uso de `requirements.txt` para Python y `package.json` para JavaScript/Node.js ayuda a crear un entorno repetible y consistente para su aplicación.
- FastAPI se puede utilizar para crear una aplicación web simple en Python que pueda manejar el ciclo de solicitud/respuesta.
- Express.js se puede utilizar para crear una aplicación web simple en JavaScript/Node.js que pueda manejar el ciclo de solicitud-respuesta.
- Los motores de plantillas se pueden utilizar para representar contenido HTML en Python y JavaScript/Node.js, eliminando así el código repetitivo.
- Se pueden usar funciones para mapear URL mientras se manejan métodos HTTP en FastAPI y Express.js.
- Git es un sistema de control de versiones que se puede utilizar para rastrear cambios en su código y volver a versiones anteriores del mismo.
- El uso del control de versiones es un primer paso esencial para crear aplicaciones sólidas que se puedan implementar, compartir y mantener.

3 Implementación manual con máquinas virtuales

Este capítulo cubre

- Creación y conexión a servidores basados en la nube
- Comprender los fundamentos del servidor web con NGINX
- Instalación de repositorios Git autoalojados
- Configuración e instalación de aplicaciones web de producción
- Implementación de aplicaciones que se ejecutan en segundo plano con Supervisor
- Automatizar el proceso de implementación con ganchos de Git
- Protegemos nuestro servidor aprovechando firewalls y claves SSH

En esta parte de nuestro proceso, implementaremos manualmente nuestro código en un servidor. Esto implica subirlo desde nuestro ordenador local al servidor usando únicamente Git. Una vez subido el código, se ejecutarán scripts personalizados para garantizar que nuestro servidor se actualice con la última versión de nuestra aplicación. El proceso de compartir código de forma segura entre ordenadores es precisamente la razón por la que Git es una herramienta tan importante y que vale la pena dominar.

Este proceso es similar a conducir un vehículo con transmisión manual. Aunque requiere un poco más de práctica, es una excelente oportunidad para desarrollar tus habilidades antes de pasar al método automático. Si eres de Estados Unidos, puede que estés acostumbrado a conducir una transmisión automática y te parezca extraña. Sin embargo, si sabes conducir una transmisión manual, podrías tener...

Gran apreciación del valor de aprender el proceso manual antes del automático.

Aprender a implementar manualmente nos enseñará el proceso y nos proporcionará el resultado deseado, siendo solo un poco más desafiante que escribir el código de la aplicación, como hicimos en el capítulo 2. Los proveedores de plataforma como servicio (PaaS), como Heroku y AWS Elastic Beanstalk, son herramientas que utilizan código de aplicación y Git para automatizar muchos de los pasos que realizaremos manualmente en este capítulo. Los proveedores de PaaS pueden ser una excelente manera de implementar una aplicación sin preocuparse por la infraestructura subyacente. La desventaja de usar una PaaS suele ser el precio y la flexibilidad.

Para tener éxito en este capítulo, debe tener un conocimiento básico del ciclo de solicitud/respuesta HTTP, la funcionalidad de URL y algo de experiencia en la línea de comandos.

Además, deberías haber completado dos aplicaciones web en el capítulo 2.

Nuestro primer paso es crear un servidor remoto donde implementar nuestro código. Usaremos un proveedor de nube llamado Akamai Connected Cloud (ACC) para aprovisionar una máquina virtual (VM) que aloje nuestro código. Usaremos Ubuntu 22.04 LTS como sistema operativo.

3.1 Creación y conexión a un servidor remoto

La computación en la nube ha desbloqueado una enorme barrera de entrada a la hora de implementar una aplicación a prácticamente cualquier escala. Si no existiera, tendría que comprar usted mismo todos los ordenadores (por ejemplo, servidores físicos) y el equipo de red relacionado. También tendría que...

Negociar con los proveedores de servicios de internet (ISP) para obtener direcciones IP estáticas propias, entre muchos otros obstáculos desafiantes que escapan al alcance de cualquier libro. Las direcciones IP estáticas nos permiten ejecutar nuestros sitios web y aplicaciones en internet. Los proveedores de la nube gestionan la infraestructura informática física por nosotros: los servidores, la red, el cableado, la alimentación, la refrigeración, la seguridad física, las direcciones IP y muchos otros aspectos para garantizar el funcionamiento de nuestro software. Aquí es donde entra en juego ACC (anteriormente Linode).

ACC es un proveedor líder mundial de servicios en la nube que ofrece una amplia gama de servicios, incluyendo máquinas virtuales, redes de entrega de contenido, almacenamiento de objetos, bases de datos y Kubernetes administrado. Utilizaremos ACC como nuestro principal proveedor de nube para este libro, pero los conceptos y técnicas se pueden aplicar a prácticamente cualquier proveedor de nube gracias a la potencia de las tecnologías de código abierto que hemos utilizado y seguiremos utilizando.

Un tema clave de este libro es la portabilidad y la flexibilidad. Este capítulo no es la excepción. El objetivo es adquirir las habilidades, técnicas y la comprensión necesarias para implementar nuestras aplicaciones en prácticamente cualquier servidor, servicio de alojamiento o proveedor de nube.

3.1.1 Aprovisionamiento de una máquina virtual

El aprovisionamiento de una máquina virtual consiste en alquilar una parte de uno o varios servidores de un proveedor de nube. El servidor físico utiliza virtualización, lo que permite alquilar solo una parte específica de una computadora más grande, en lugar de la máquina completa.

Si lo considera como un centro comercial, solo tiene que alquilar una unidad en lugar de todo el complejo. Una ventaja importante de este proceso de alquiler es la posibilidad de seleccionar...

cantidad deseada de potencia de cómputo, incluida RAM, CPU y más, lo que resulta en mayor eficiencia y ahorro de costos.

Este paradigma constituye la base de la computación en la nube, que, como aprenderemos en este libro, es un método excelente para implementar todo tipo de aplicaciones.

El tipo específico de máquina virtual que vamos a alquilar lo proporciona el servicio Linode de ACC. Hagámoslo ahora:

1. Crea una cuenta en ACC en linode.com/rk8s (los lectores del libro también reciben un crédito promocional).
2. Inicie sesión en ACC (<https://linode.com>).
3. Haga clic en el menú desplegable Crear > y luego seleccione Linode.
4. Utilice la siguiente configuración:
 - Distribución: Ubuntu 22.04 LTS (o la última versión LTS)
 - Región: Dallas, TX (o la región más cercana a usted)
 - Plan—CPU compartida > Linode 2 GB (o más)
 - Etiqueta—rk8s-vm
 - Etiquetas—<opcional>
 - Contraseña raíz—Establezca una buena
 - Claves SSH— <recomendado> (revise el apéndice D para aprender a crear una y agregarla a Linode)
 - Todas las demás configuraciones—<opcional>
5. Haga clic en Crear Linode.
6. Después de unos minutos, verá aparecer una dirección IP. en la consola Linode (figura 3.1).

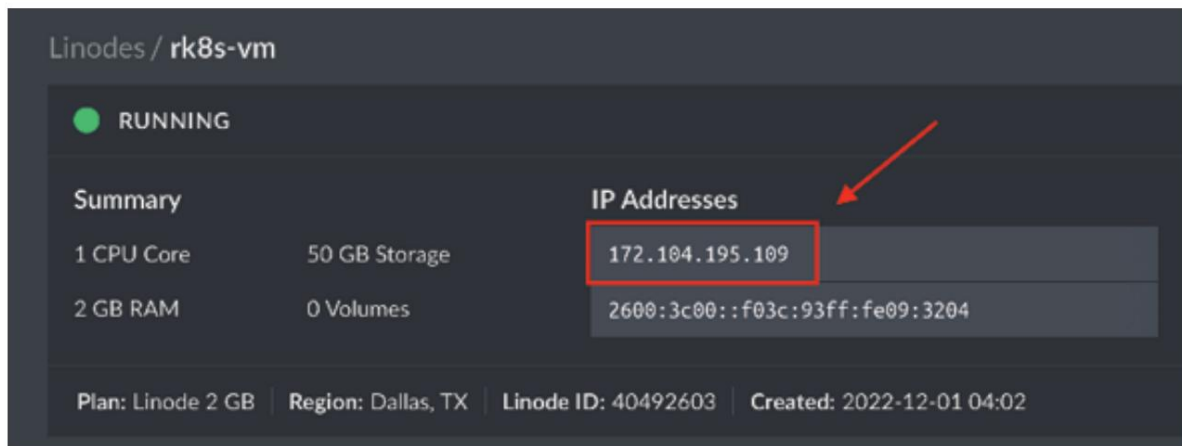


Figura 3.1 VM Linode aprovisionada con una dirección IP

Si esta es tu primera experiencia aprovisionando una máquina virtual o un servidor remoto, es un buen momento para que te hagas una idea de lo que acabas de lograr. Por menos de \$10 (o probablemente \$0), alquilaste una parte de una computadora ubicada en un centro de datos en Dallas, Texas, o en otra ubicación de tu elección. Ahora tienes control total sobre esta computadora y puedes ejecutar cualquier tipo de aplicación en ella. Además, tienes una dirección IP accesible desde cualquier lugar del mundo, lo que te permite influir positivamente en la vida de alguien al otro lado del mundo con tu aplicación. Lo mejor de todo es que solo tardaste unos 5 minutos en poner en funcionamiento este servidor.

Este mismo proceso es el que las grandes empresas tecnológicas utilizan a diario: aprovisionar máquinas virtuales para ejecutar sus aplicaciones. La mayor diferencia entre nuestras necesidades en este libro y las suyas suele deberse al gran volumen de máquinas virtuales y la potencia que podemos necesitar. Por ejemplo, podríamos necesitar solo una máquina virtual para ejecutar nuestra aplicación, mientras que ellos podrían necesitar miles. La buena noticia es que el proceso es el mismo y puede mejorarse mediante técnicas llamadas infraestructura como código (IaC). La IaC queda fuera del alcance de este libro, pero vale la pena estudiarla si busca una mejor alternativa al aprovisionamiento manual.

servidores a través de una consola web (este proceso a menudo se conoce como ClickOps).

Ahora que tenemos un servidor remoto listo, necesitamos una forma de empezar a controlarlo. Para ello, usaremos un protocolo conocido como Secure Shell (SSH), que permite iniciar sesión y conectarnos de forma segura a nuestra máquina virtual. Una vez conectado, podemos empezar a configurarlo y ejecutar todo tipo de comandos. SSH es la forma en que accedemos a la línea de comandos del servidor para, por ejemplo, instalar software basado en el servidor, trabajar con las dependencias de la aplicación, configurar un repositorio Git básico y, finalmente, desplegar nuestra aplicación. (IaC también puede ser muy útil en este proceso).

3.1.2 Conexión mediante SSH

Conectarse a un servidor remoto es tan fundamental hoy en día que la mayoría de las computadoras de escritorio y portátiles incorporan un cliente SSH. Los dispositivos móviles tienen compatibilidad nativa variable, pero casi todos tienen aplicaciones compatibles con SSH. SSH permite conectarse a computadoras de nuestra red local, así como a computadoras de todo el mundo a través de internet.

Para realizar SSH, necesitamos al menos cinco cosas:

- Un host remoto
- Una dirección IP o un nombre de dominio (figura 3.2)
- Un nombre de usuario
- Una contraseña o una clave SSH instalada Una
- conexión a la misma red que el host remoto (por ejemplo, Internet)

Cuatro de estas cinco cosas se completaron al aprovisionar nuestra máquina virtual. Supondré que está conectado a internet para acceder a nuestro host remoto en Akamai Linode.

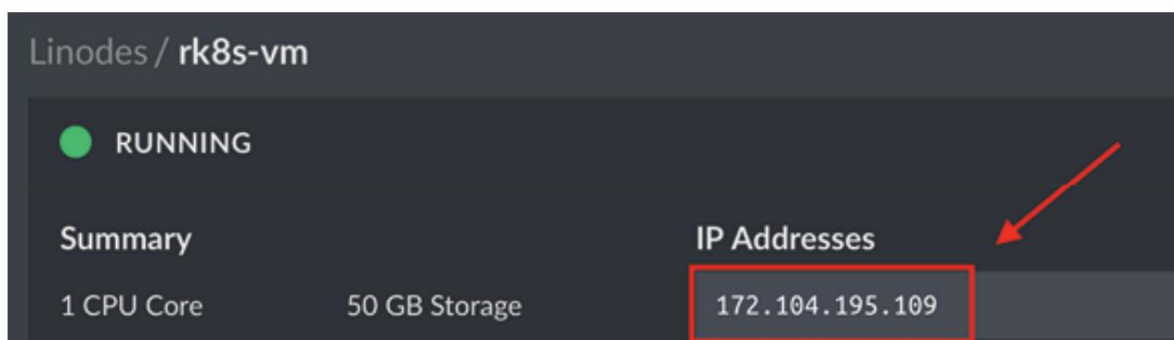


Figura 3.2 Nuestra dirección IP de host actual

Veamos cómo estos elementos se corresponden con lo que usaremos:

- Host remoto : máquina virtual Akamai Linode
- Dirección IP : 172.104.195.109 (ver figura 3.2)
- Contraseña: Contraseña de root (ver sección 3.1.1)
- Nombre de usuario—root

El nombre de usuario root es el predeterminado en muchas distribuciones de Linux y sistemas operativos Ubuntu. Algunos proveedores de nube cambian este nombre de usuario, y es posible que también cambie en Akamai Linode en el futuro. En nuestro caso, podemos encontrar esta información en la consola de Akamai Linode, como se muestra en la figura 3.3.

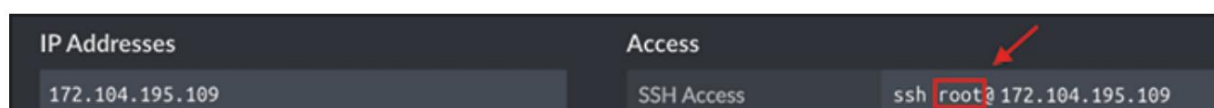


Figura 3.3 Ejemplo de conexión SSH de la consola Akamai

Ahora que tenemos toda la información necesaria, conectémonos a nuestro host remoto. Dependiendo de su configuración, es posible que se le solicite la contraseña del usuario root. Con esto en mente, abra la línea de comandos (Terminal o PowerShell) e introduzca el comando que se muestra a continuación.

Listado 3.1 Ejemplo de conexión SSH

```
ssh raíz@172.104.195.109
```

Dado que es la primera vez que nos conectamos a este host remoto y a esta dirección IP en particular, se ejecutarán algunos procesos estándar. El primer proceso es un mensaje, como en la figura 3.4, que pregunta si queremos permitir que nuestro equipo local confíe y se conecte al servidor remoto. Esta comprobación de autenticidad solo debería realizarse una vez, a menos que se haya producido algún cambio en el host remoto o en los registros de hosts remotos de nuestro equipo local.

```
The authenticity of host '172.104.195.109 (172.104.195.109)' can't be established.  
ED25519 key fingerprint is SHA256:QQ0Gq6tTS1N1CxJ/ytqX4vuLREopozQQ1bxBP4bVrIQ.  
This key is not known by any other names  
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Figura 3.4 Verificación de la autenticidad de la confirmación del host

Si ve esto, escriba "sí" para continuar y pasar a la siguiente sección. Tras confirmar con "sí", este mensaje no volverá a aparecer a menos que se conecte a un nuevo ordenador o elimine la dirección IP del host (p. ej., 172.104.195.109) de su archivo ~/.ssh/known_hosts. Si no ve esta respuesta, puede deberse a varias razones:

- Utilizaste un nombre de usuario o dirección IP incorrectos en tu comando ssh .
- Su clave SSH no es válida o está instalada incorrectamente (aprenda a usar claves SSH en el apéndice D).
- Usted o el servidor tienen un firewall que bloquea la conexión, lo cual es posible si no se trata de un servidor nuevo.
- Ha utilizado SSH para esta dirección IP particular, por lo que deberá eliminarlo de ~/.ssh/known_hosts.

Después de verificar la autenticidad del host remoto, se nos solicitará la contraseña del usuario root.

Se le solicitará esta contraseña cada vez que se conecte a este host remoto, a menos que tenga una clave SSH instalada en él. La instalación de una clave SSH en un host remoto para una conexión SSH sin contraseña se explica en el apéndice C. En este punto, su línea de comando local debería convertirse en la línea de comando remota que se parece a la figura 3.5.

```
The programs included with the Ubuntu system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
root@localhost:~#
```

Figura 3.5 Primera entrada de la línea de comandos del host remoto

Si ve este resultado, se ha conectado correctamente a su host remoto. Por segunda vez en este capítulo, ¡es hora de celebrar! Su computadora ahora está conectada directamente a otra computadora a través de internet, sin necesidad de un navegador web. ¡Felicidades!

Quizás no le sorprenda saber que el primer paso será actualizar su sistema. No estamos actualizando el sistema operativo, sino la lista de paquetes disponibles.

Antes de ejecutar el comando para actualizar, aprendamos qué son `sudo` y `apt`. `Sudo` es un comando que permite ejecutar un comando como superusuario con permisos especiales para ejecutar o instalar algo. `Sudo` debe usarse con moderación, pero suele ser necesario al realizar cambios en el software o la configuración del sistema. `Apt` es un gestor de paquetes para distribuciones de Linux basadas en Debian. Este comando es similar a `npm` o `pip`, ya que instala todo tipo de software de terceros en nuestro nombre, pero...

específicamente para sistemas Linux. apt también se puede utilizar para instalar varios entornos de ejecución de lenguajes de programación como Python y Node.js, como se muestra en la siguiente lista.

Listado 3.2 Uso de apt para actualizar la lista de paquetes disponibles del sistema

```
sudo apt actualización#1
```

#1 Este es un comando común que usaremos cuando comencemos a trabajar con contenedores en capítulos futuros.

Ahora tenemos la combinación perfecta para crear nuestro primer sitio web estático en nuestro host remoto. Será increíblemente sencillo y utilizará una herramienta increíble llamada NGINX (se pronuncia "engine-x").

3.2 Servir sitios web estáticos con NGINX

Si retrocediéramos unos 20 años, encontraríamos muchos sitios web estáticos y sencillos que ofrecen poco más que un folleto impreso en línea. Si bien los sitios web estáticos sencillos no son tan llamativos como los dinámicos, siguen cumpliendo una función valiosa y pueden usarse con gran eficacia.

En esta sección, implementaremos nuestro primer sitio web simplemente modificando código HTML y configurando un servidor web llamado NGINX. NGINX puede realizar muchas funciones que exploraremos en próximos capítulos, pero por ahora, solo lo usaremos para enviar un sitio web estático simple a la dirección IP pública de nuestro servidor remoto. Este es el proceso para usar NGINX:

1. Actualice la lista de paquetes disponibles del sistema.
2. Instalar NGINX.

3. Crea un archivo HTML simple.
4. Configure NGINX para servir nuestro archivo HTML.

Cada vez que instale software en un equipo Linux distribuido a través del Gestor de Paquetes APT (apt), debe actualizar la lista de paquetes disponibles. Esto se debe a que esta se actualiza con frecuencia y queremos asegurarnos de instalar la última versión del software. Después de actualizarla, instalaremos NGINX con el siguiente comando.

Listado 3.3 Instalación de NGINX en Ubuntu

```
sudo apt update  
sudo apt install nginx
```

Cuando se le solicite instalar NGINX, escriba y y presione Entrar. Esto instalará NGINX y todas sus dependencias. Una forma de automatizar el comando `apt install <paquete>` es incluir la opción `-y` para que podamos aceptar automáticamente cualquier solicitud que pueda aparecer, terminando con un comando como `sudo apt install nginx -y`.

Una vez que NGINX termine de instalarse, abra su dirección IP en su navegador web (p. ej., <http://172.104.195.109>). Debería ver algo como la figura 3.6.



Figura 3.6 NGINX instalado y mostrando la página predeterminada

¡Felicitaciones! Has instalado NGINX correctamente en tu host remoto y ahora estás sirviendo un sitio web estático al mundo. Aún necesitamos configurar esta página web para mostrar nuestro propio contenido, pero por ahora, esto es emocionante.

3.2.1 Adoptar máquinas virtuales efímeras

Cuando implementé por primera vez un software como este, me encariñé mucho con las máquinas virtuales y la dirección IP que usaba. Tenía miedo de hacer grandes cambios y arruinar gran parte del trabajo que hice en esa máquina.

Este temor era doble: estaba usando un método antiguo para administrar archivos con un servidor (FTP) y no trataba mis máquinas virtuales como si fueran efímeras.

El Protocolo de Transferencia de Archivos, o FTP, sigue siendo un método válido para transferir archivos en internet, pero no lo recomiendo para código simplemente porque existe Git. Git es mucho mejor porque permite rastrear los cambios en el código a lo largo del tiempo. FTP es más como el Viejo Oeste, y los archivos podrían mostrar...

surgió de la nada con muy poca explicación de por qué.

Personalmente, recomiendo evitar usar FTP a toda costa. Si necesitas usar algo como FTP, considera usar la extensión Remote-SSH de VSCode para una experiencia más fluida.

La segunda razón por la que tenía miedo de hacer cambios era que no trataba mis máquinas virtuales como si fueran efímeras. Como usaba FTP, adquirí la mala costumbre de tratarlas como un disco duro externo que, casualmente, también ejecutaba mi software. Esta lógica es errónea, ya que una máquina virtual es tan fácil de destruir y recrear que usarla para conservar datos es una mala idea.

Tratar las máquinas virtuales como efímeras también le ayudará a preparar su código y proyectos para las herramientas correctas para garantizar que, si necesita destruir una máquina virtual, pueda hacerlo sin perder ningún dato.

Con todo esto en mente, veamos algunas formas de eliminar NGINX y empezar de nuevo:

- Destruir la máquina virtual: esta es la forma más fácil de comenzar de nuevo y te recomiendo que practiques.
- Eliminar NGINX a través del administrador de paquetes APT: este es un enfoque más común, porque destruir y configurar máquinas virtuales puede resultar tedioso, especialmente sin herramientas de automatización e IaC:
 - `sudo apt remove nginx --purge` y `sudo apt autoremove` eliminarán NGINX y cualquier otro paquete que ya no sea necesario. `sudo apt purge nginx` y `sudo apt autoremove`
 - eliminarán NGINX y cualquier otro paquete que ya no sea necesario. `sudo apt purge --auto-remove nginx` eliminará NGINX y cualquier otro paquete que ya no
 - sea necesario.

Ahora que tenemos una forma de instalar y eliminar NGINX, es hora de que aprendamos a modificar la página HTML predeterminada que sirve NGINX.

3.2.2 Modificar la página HTML predeterminada de NGINX

Para mí, la página web predeterminada de NGINX es una alegría. Me recuerda que he comenzado un nuevo proyecto que necesita configuración. Quizás te parezca un sitio web de los 90, y tienes toda la razón. En cualquier caso, veamos cómo podemos modificar ligeramente esta página para personalizarla mientras aprendemos algunos conceptos básicos de NGINX.

Tras instalar NGINX, nuestra máquina virtual tiene una nueva ruta en el sistema: `/var/www/html`. Este es el directorio raíz predeterminado para servir archivos HTML estáticos con NGINX. Si abrimos este directorio, veremos un archivo llamado `index.nginx-debian.html`.

Esta es la página HTML predeterminada que NGINX proporciona. Modifiquemos esta página mediante nuestra conexión SSH y la línea de comandos. Comenzaremos eliminando la página predeterminada de NGINX con la siguiente lista.

Listado 3.4 Eliminación de la página NGINX predeterminada

```
sudo rm /var/www/html/index.nginx-debian.html
```

Vamos a crear un nuevo archivo HTML que servirá como nuestra nueva página web. Para ello, usaremos una técnica que utiliza `cat` (un comando para mostrar el contenido de un archivo en la terminal), el comando `EOF` (Fin de Archivo) y una ruta de archivo para crear un nuevo archivo multilínea. La siguiente lista muestra un ejemplo de la sintaxis de este comando.

Listado 3.5 Ejemplo de sintaxis del comando cat EOF

```
cat <<EOF > ruta/a/su/archivo
Esta es una nueva línea
Esta es otra nueva línea
    Esta es una línea con pestañas
Esta es la última línea
EOF
```

Siguiendo esta misma sintaxis, veamos un ejemplo práctico de su uso creando un nuevo archivo HTML en el siguiente listado.

Listado 3.6 Creación de un nuevo archivo index.html para NGINX

```
gato <<EOF > /var/www/html/index.html <!DOCTYPE html>
<html> <head> <title>Hola
mundo</
    title> </
        head> <body> <h1>¡Hola mundo!</h1>

        <p>¡Llegamos pronto!</p> </body> </html>
    EOF
```

Para verificar el contenido de este nuevo archivo podemos utilizar nuevamente cat con el comando cat /var/www/html/index.html.

En este punto, nuestro directorio /var/www/html/ debería contener únicamente index.html . Abramos nuestra dirección IP para revisar los cambios en internet, como se muestra en la figura 3.7.

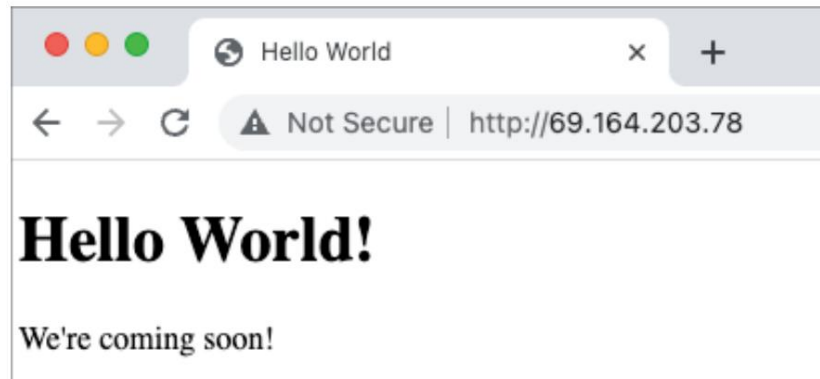


Figura 3.7 Verificando nuestra nueva página web

Como puede observar en la figura 3.7, la dirección IP ha cambiado con respecto a los ejemplos anteriores. Tómese un momento y pregúntese por qué cree que es así.

¿Fue una errata? ¿Un error de programación?

No, simplemente realicé el mismo ejercicio que les pedí hace unos momentos: destruir la máquina virtual y empezar de cero. A lo largo de este libro, veremos que las direcciones IP cambian constantemente porque reciclamos continuamente nuestras máquinas virtuales.

Es importante destacar que no modificamos nada relacionado con la configuración de NGINX, pero sí usamos las funciones integradas para mostrar una nueva página HTML. Si conoces bien las tecnologías frontend, podrías mejorar significativamente el HTML que añadimos y mejorar su aspecto. Por ahora, lo dejaremos así, ya que es hora de configurar un repositorio Git remoto para almacenar el código de nuestra aplicación.

3.3 Repositorios Git remotos autoalojados

El método manual de usar `cat` y EOF es adecuado para un ejemplo rápido y sencillo, pero muy inadecuado para gestionar un proyecto de software de envergadura. Es hora.

Usamos nuestro código del capítulo 2 con Git para enviar (y recibir) nuestro código directamente a una carpeta en nuestro servidor remoto.

Antes de sumergirnos de lleno en el uso de Git en este servidor, tenga en cuenta algunas ideas importantes cuando se trata de autohospedar un servidor.

Repositorio Git:

- Pérdida de datos: si tratamos una VM como efímera, cualquier código o dato que almacenemos en esa VM no estará seguro y probablemente se destruirá de forma permanente si nuestra VM falla o se destruye.
- Pequeñas ventajas: Los repositorios Git autoalojados suelen ofrecer muy pocas ventajas en comparación con servicios de terceros como GitHub o GitLab. Estas herramientas se han perfeccionado durante años para que la gestión de proyectos Git sea mucho más eficaz que nuestra propia solución autoalojada.
- Complejidad agregada e innecesaria: si bien puede ser simple comenzar y útil para ejemplos básicos, los repositorios Git autohospedados pueden volverse complicados rápidamente a medida que nuestro proyecto escala.
- Seguridad: los repositorios Git autohospedados pueden ser una pesadilla de seguridad porque colaborar en estos repositorios requiere una multitud de pasos para garantizar la seguridad de todo el sistema.
- Automatización: Git no tiene una forma nativa de ejecutar flujos de trabajo automatizados dentro de las actividades estándar de Git, mientras que herramientas de terceros como GitHub Actions y Gitlab CI/CD se pueden usar para automatizar todo el proceso de creación, prueba e implementación de código, todo a través de un único push de Git.

Con estos elementos en mente, aún queremos comprender la mecánica de usar un repositorio Git autoalojado como medio para transportar nuestro código local a producción. En próximos capítulos, adaptaremos este método al alojamiento de nuestro

código únicamente en GitHub e implementación a través de una canalización de automatización.

En el capítulo 2, configuramos dos aplicaciones web con un repositorio Git local y privado, así como un repositorio Git remoto de terceros alojado en GitHub (público o privado). En el resto de esta sección, configuraremos una ubicación adicional para nuestro código en cada proyecto: un repositorio Git remoto y privado en nuestra máquina virtual.

Un repositorio Git simple es un repositorio sin directorio de trabajo, lo que significa que solo tenemos el historial de cambios, pero no el código real. Esto también significa que no editaremos el código que llega a nuestro servidor, sino que solo enviaremos y recibiremos código del repositorio simple. Una vez que nuestro código esté en un repositorio simple, usaremos un gancho basado en Git para descomprimirlo en un directorio de trabajo que será la base de nuestra aplicación implementada.

El proceso de principio a fin para crear un repositorio privado y remoto será el siguiente:

1. Inicie sesión en el servidor remoto a través de SSH e instale Git.
2. Cree un repositorio Git básico para nuestras aplicaciones Python y Node.js.
3. Actualice nuestro repositorio Git local para que apunte al nuevo repositorio remoto, incluidas las claves SSH según sea necesario.
4. Envíe nuestro código al repositorio remoto.
5. Crea un gancho de Git que nos permitirá
6. Extraiga nuestro código en un directorio de trabajo.
7. Instalar o actualizar las dependencias de nuestra aplicación.
8. Reinicie todos los procesos del servidor web.

3.3.1 Iniciar sesión en el servidor con SSH

Como primer paso, debemos acceder a la línea de comandos de nuestra máquina virtual. Asegúrese de usar SSH para iniciar sesión ahora mismo, como se muestra en la siguiente lista.

Listado 3.7 SSH en su host

```
ssh root@<tu-ip>
```

Ahora que hemos iniciado sesión, podemos instalar Git en nuestro servidor. En el siguiente listado, usaremos el mismo comando que usamos en el listado 3.8 para instalar Git en nuestro servidor.

Listado 3.8 Instalación de Git en el servidor

```
apt-get update apt-  
get install git -y
```

Ahora que tenemos Git instalado, podemos crear nuestros repositorios Git básicos.

3.3.2 Creación de los repositorios Git básicos

Crear un repositorio básico consiste en crear un directorio y luego ejecutar el sencillo comando `Git git init --bare`. Es recomendable nombrar estos directorios con la extensión `.git` para distinguirlos de otros directorios y dejar claro que son repositorios Git básicos .

Después de crear el repositorio Git básico, necesitamos crear una referencia simbólica con HEAD a la rama principal . En este caso, al igual que en GitHub, usaremos la rama principal como rama predeterminada. HEAD es simplemente una forma de identificar la confirmación más reciente en un repositorio. A medida que realizamos cambios en nuestro código localmente, queremos que esos cambios se reflejen en nuestro repositorio remoto, y HEAD nos permite...

Haz esto, como veremos cuando actualicemos los ganchos de Git en la siguiente sección.

Creemos los repositorios Git básicos para la aplicación web Python y nuestra aplicación web Node.js, como se muestra en la siguiente lista.

Listado 3.9 Inicialización de los repositorios de aplicaciones web

```
mkdir -p /var/repos/roadtok8s/py.git cd /var/
repos/roadtok8s/py.git git init --bare git
symbolic-ref HEAD
refs/heads/main

mkdir -p /var/repos/roadtok8s/js.git cd /var/repos/
roadtok8s/js.git git init --bare git symbolic-
ref HEAD refs/
heads/main
```

El Listado 3.9 destaca el mismo proceso que seguimos en GitHub (capítulo 2), excepto que usamos la línea de comandos en lugar de un sitio web para crear estos nuevos repositorios remotos. El formato de la URL para estos repositorios remotos será el siguiente:

```
ssh://root@$DIRECCIÓN_IP_VM/var/repos/roadtok8s/py.git ssh://
root@$DIRECCIÓN_IP_VM/var/repos/roadtok8s/js.git
```

Reemplace `$VM_IP_ADDRESS` con la dirección IP de nuestra máquina virtual actual. Para ello, podemos usar `curl ifconfig.me` o buscarlo manualmente en la consola de Akamai Linode, como hicimos antes. Prefiero usar `curl`, ya que está integrado en la mayoría de las distribuciones de Linux y es mucho más automatizado.

Dado que usamos el formato `$VM_IP_ADDRESS`, podemos usar la sustitución de cadenas integrada en bash para reemplazar este valor con la dirección IP de nuestra máquina virtual. Para ello, ejecutamos el comando de la siguiente lista.

Listado 3.10 Reemplazo de la dirección IP de la máquina virtual

```
export VM_IP_ADDRESS=$(curl ifconfig.me) echo "esta es tu IP:
$VM_IP_ADDRESS"
```

Para utilizar la variable `VM_IP_ADDRESS` , podemos utilizar los siguientes comandos:

```
echo "git remoto agregar vm ssh://root@$VM_IP_ADDRESS/var/repos/roadtok8s/py.gi
"t"
echo "git remoto agregar vm ssh://root@$VM_IP_ADDRESS/var/repos/roadtok8s/js.gi
"t"
```

El resultado exacto de estos comandos variará según la dirección IP de su máquina virtual y, por lo tanto, se omite en este libro. Asegúrese de usar el resultado real antes de continuar.

Las alarmas deberían estar sonando en tu cabeza en base a lo que hicimos en el capítulo 2 porque ahora podemos actualizar nuestro código local para apuntar a estos repositorios remotos, e incluso podemos comenzar a enviar código aquí también.

3.3.3 Enviar código local al repositorio remoto

Ahora que hemos configurado nuestros hosts Git remotos en nuestras máquinas virtuales, es hora de actualizar los repositorios locales de nuestro proyecto para que apunten a estos nuevos repositorios remotos. Una vez actualizados, podemos enviar nuestro código a los repositorios remotos.

El proceso es así:

1. Navegue a la raíz de nuestro repositorio de proyecto local (`cd ~/Dev/roadtok8s/py/` o `cd ~/Dev/roadtok8s/js/`).

2. Agregue el repositorio remoto a nuestro repositorio local (git agregar vm ..).
3. Envía nuestro código al repositorio remoto (git push).

Un ejemplo de este proceso para nuestra aplicación web Python es el siguiente: `. cd ~/Dev/roadtok8s/py/ . git remote add vm ssh://root@45.79.47.168/var/repos/roadtok8s/py.git` (Reemplace 45.79.47.168 con su IP) `. git push vm main`.

En este ejemplo, hemos utilizado el control remoto llamado vm en lugar de origin por dos razones:

- Origen: Este nombre ya lo utiliza nuestro repositorio remoto de GitHub como nombre predeterminado . vm: Usar este nombre significa
- que no es habitual en Git, y en próximos capítulos, usaremos solo GitHub. Esto también indica que vm es un caso especial relacionado con las lecciones que está aprendiendo en este libro.

Repita este mismo proceso para su Node.js. Si la transferencia a su máquina virtual se realiza correctamente, el resultado será similar al de la figura 3.8.

```
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 4 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (10/10), 1.02 KiB | 1.02 MiB/s, done.
Total 10 (delta 0), reused 0 (delta 0)
To ssh://
    4b825dc642cb6eb9a060e54bf8d69288fbee4904 -> main
```

Figura 3.8 Envío exitoso al repositorio remoto

Si estas inserciones no se realizaron correctamente, verá un mensaje de error. A continuación, se muestran algunos mensajes de error que puede haber encontrado y su significado:

- Fatal: 'vm' no parece ser un repositorio Git. ¿ Usaste "git remote add vm ssh://... "? Si no es así, probablemente verás este error. Fatal: No se pudo leer desde el
- repositorio remoto. ¿ Tienes permiso para acceder al repositorio remoto? Si no es así, probablemente verás este error. Error: src refsPEC main no coincide con
- ninguno. Esto significa que tu rama local no es la principal , como la que creamos en el capítulo 2. Una solución rápida puede ser tan simple como "git branch -m master main", suponiendo que "master" es la rama que has estado usando.

Si la carga local se realizó correctamente, nuestro código debería estar en nuestra máquina virtual. Sin embargo, si accede a los repositorios vacíos, se encontrará con cierta confusión, ya que el código podría parecer inexistente. Para aclarar esto, revisemos el contenido de los repositorios vacíos.

3.3.4 Git hook para extraer código

Los repositorios básicos son un tipo especial de repositorio que solo contiene los metadatos del proyecto. Esto significa que el código en sí no se almacena en el repositorio básico, sino en un directorio separado que es un clon del repositorio básico. En nuestro caso, los metadatos del proyecto ya están almacenados en el repositorio básico, pero necesitamos clonar el código desde el repositorio básico al directorio donde lo ejecutaremos.

Ahora, enviaremos el código a otro directorio que es un clon del repositorio básico. Esto se hace mediante un archivo post-

Gancho de Git de recepción . Este gancho es un script de shell bash que se ejecuta tras enviar un paquete al repositorio remoto. Lo usaremos para clonar el código del repositorio vacío al directorio donde lo ejecutaremos.

Primero, creemos una nueva carpeta en nuestra VM para cada proyecto, como se muestra en la siguiente lista.

Listado 3.11 Creación de una nueva carpeta para cada proyecto

```
mkdir -p /opt/proyectos/roadtok8s/py mkdir -p /  
opt/proyectos/roadtok8s/js
```

La ruta /opt/ es una ubicación común para almacenar código en sistemas Linux. La ruta /opt/projects/roadtok8s/ es donde almacenaremos el código de este libro. La ruta /opt/projects/roadtok8s/py/ es donde almacenaremos el código de Python, y la ruta /opt/projects/roadtok8s/js/ es donde almacenaremos el código de Node.js.

Ahora que tenemos estas dos carpetas creadas, es hora de configurar nuestro gancho Git para hacer lo siguiente:

- Clona el código del repositorio vacío al directorio donde ejecutaremos nuestro código.
- Instalar las dependencias para nuestro proyecto.
- Ejecute o reinicie cualquier proceso en ejecución que permita que nuestros proyectos se ejecuten.

Primero, crearemos el gancho de Git para nuestro proyecto de Python. Para ello, crearemos un archivo llamado post-receive en el directorio de ganchos de nuestro repositorio básico. Este archivo se ejecutará tras una subida al repositorio remoto. Lo usaremos para clonar el código del repositorio básico al directorio donde lo ejecutaremos. Después de crear el archivo,...

hazlo ejecutable para que Git tenga permiso para ejecutar el archivo, como se muestra en la siguiente lista.

Listado 3.12 Creación del gancho de Git para nuestro proyecto Python

```
cd /var/repos/roadtok8s/py.git/hooks touch post-  
recepción chmod +x  
post-recepción#1
```

#1 Esto hará que el archivo sea ejecutable.

El nombre de archivo post-receive es un nombre especial para un gancho de Git, así que asegúrese de usarlo; de lo contrario, el gancho no se ejecutará.

Este archivo contendrá lo siguiente:

- git checkout: este es el comando raíz para extraer código con Git.
- HEAD: esta es una referencia que especifica la última confirmación en la rama del código que estamos extrayendo. --work-
- tree=<path/to/working/dir/>: este es un indicador específico de Git que especifica el directorio en el que se extraerá el código.
- --git-dir=<path/to/bare/repo/dir/>—Esta es una bandera específica de Git que especifica el directorio donde se encuentra el repositorio vacío. Usar esta bandera permite que nuestro gancho post-recepción sea un poco más flexible.
- -f—Esta es una bandera específica de Git que fuerza la verificación a sobrescribir cualquier archivo existente en el directorio de trabajo. Esto es útil si queremos actualizar nuestro código sin tener que eliminar manualmente el código existente.

Con estos conceptos en mente, creemos el archivo posterior a la recepción con el comando de la siguiente lista.

Listado 3.13 Creación del archivo posterior a la recepción

```
exportar ÁRBOL_DE_TRABAJO=/opt/proyectos/roadtok8s/py
exportar DIR_GIT=/var/repos/roadtok8s/py.git
```

```
gato <<EOF > "$GIT_DIR/hooks/post-recepción" #!/bin/bash git --work-
tree=$WORK_TREE
--git-dir=$GIT_DIR checkout HEAD - f EOF
```

```
chmod +x "$GIT_DIR/hooks/post-recepción"
```

Repita este proceso para el proyecto Node.js, reemplazando py con js donde sea necesario.

Dado que el gancho post-recepción es simplemente un script de bash, podemos llamarlo en cualquier momento para asegurar su correcto funcionamiento. Cada vez que se realicen cambios en este gancho, se recomienda llamarlo inmediatamente para asegurar su correcto funcionamiento y la aplicación de los cambios en el código, como se muestra en la siguiente lista.

Listado 3.14 Verificación del gancho posterior a la recepción

```
exportar PY_GIT_DIR=/var/repos/roadtok8s/py.git exportar
JS_GIT_DIR=/var/repos/roadtok8s/js.git
```

```
bash $PY_GIT_DIR/hooks/post-recepción bash
$JS_GIT_DIR/hooks/post-recepción
```

Si ve el error error: pathspec 'HEAD' no coincide con ningún archivo conocido por git, es muy probable que no haya enviado su código desde su computadora local a su VM.

Si ve un error o no, verifiquemos que podemos enviar nuestro código a nuestro servidor ahora mismo con el código en la siguiente lista.

Listado 3.15 Envío de código al servidor desde la computadora local

```
cd ~/dev/roadtok8s/py git  
push vm principal
```

```
cd ~/dev/roadtok8s/js git  
push vm principal
```

Una vez enviado esto, puedes verificar el código en tu servidor ejecutando los siguientes comandos SSH:

```
ssh root@<tu-ip> ls -la /opt/projects/roadtok8s/py ssh root@<tu-ip>  
ls -la /opt/projects/roadtok8s/js
```

En este punto, tenemos todo el código en nuestra máquina y un método para instalarlo y actualizarlo continuamente. El siguiente paso es instalar las distintas dependencias de software para que nuestros proyectos puedan ejecutarse.

3.4 Instalación de las dependencias de las aplicaciones

Anteriormente en este capítulo, vimos lo fácil que es instalar NGINX para ejecutar un sitio web sencillo. Para tener un sitio web más potente y completo, necesitaremos instalar muchas más dependencias de software. Más dependencias de software implican mayor complejidad, y mayor complejidad implica más posibilidades de fallos. Comenzaremos a mitigar estos posibles problemas configurando entornos específicos para cada proyecto.

Dado que tenemos dos aplicaciones diferentes con dos entornos de ejecución distintos, necesitaremos abordar la instalación de dependencias de forma distinta. Para nuestro proyecto de Python, usaremos un entorno virtual y la herramienta de paquetes de Python (pip).

con una versión específica de Python 3. Para nuestro proyecto Node.js, utilizaremos el Node Version Manager (nvm) para instalar una versión específica de Node.js y el Node Package Manager (npm) correspondiente.

Los requisitos de ejecución para nuestros dos proyectos son los siguientes. Verifique cada versión con los comandos que aparecen junto a ella:

- Python 3 versión 3.8 o superior: verifique esto en su sesión SSH con `python3 --version`.
- Versión 16.14 o superior de Node.js: verifique esto con `node --version`.

Dado que usamos Ubuntu 20.04, Python 3.8 ya debería estar instalado, mientras que Node.js no. Independientemente de lo que tenga instalado, explicaremos el proceso de instalación de diferentes versiones de Python y Node.js para que esté mejor preparado para el uso a largo plazo de estas herramientas.

3.4.1 Instalación de Python 3 y nuestro proyecto FastAPI

La biblioteca estándar de Python cuenta con numerosas funciones integradas que la hacen ideal para el desarrollo web. Sin embargo, no es suficiente para crear una aplicación web completa, y algunos equipos Linux no la tienen instalada por defecto (a diferencia de sus equivalentes en macOS y Windows). Para ampliar las capacidades de Python y prepararnos para un entorno de producción, necesitaremos instalar bibliotecas adicionales en nuestro sistema Linux, ya que algunas de las bibliotecas estándar no están disponibles en equipos Linux sin instalaciones adicionales a nivel de sistema operativo; por ejemplo, venv (<https://docs.python.org/3/library/venv.html>).

Python 3 se instala por defecto en los sistemas Ubuntu; cuanto más reciente sea la versión de Ubuntu, más reciente será probablemente la instalación predeterminada de Python 3. Como siempre, verificamos la versión de Python en nuestro sistema con `python3 --version` (o `python --version` para entornos virtuales) para garantizar que nuestro código pueda ejecutarse con esta versión. Independientemente de la versión de Python 3 que tengamos instalada, instalaremos la última versión junto con algunos otros paquetes mediante el gestor de paquetes `apt`:

- `python3`: este paquete proporciona el comando `python3`, que se utiliza para ejecutar Python 3. `python3-venv`: este
- paquete proporciona el módulo `venv`, que se utiliza para crear entornos virtuales.
- `python3-pip`: este paquete proporciona la herramienta `pip`, que se utiliza para instalar paquetes de Python. `build-essential`:
- este paquete proporciona la herramienta `gcc`, que se utiliza para compilar paquetes de Python.

Cada uno de estos paquetes es necesario para configurar el entorno y las versiones de nuestra aplicación Python de forma que sean adecuados para su uso en producción, como se muestra en la siguiente lista. Estos paquetes se instalarán de nuevo cuando empecemos a usar contenedores con nuestra aplicación Python.

Listado 3.16 Paquetes de desarrollo de Python para Linux estándar

```
sudo apt update #1 sudo  
apt install python3 python3-pip python3-venv build-essential -y #2
```

#1 Siempre ejecute esto antes de instalar nuevos paquetes.

#2 El uso de `-y` responderá automáticamente sí a cualquier solicitud.

Durante la instalación, es posible que vea una advertencia sobre daemons que usan bibliotecas obsoletas. Si es así, presione Intro o Retorno para continuar. Los daemons son procesos en segundo plano que se ejecutan en

su máquina que se actualiza de vez en cuando, y esta advertencia potencial es un ejemplo de ello.

Una vez completada la instalación de Python, debe ejecutar "sudo system reboot" para reiniciar el equipo. Esto garantizará que se carguen todos los paquetes nuevos y que los daemons obsoletos se actualicen. Ejecutar "sudo system reboot" también cerrará la sesión SSH , por lo que deberá volver a conectarse para continuar.

Reiniciar después de instalaciones como esta no siempre es necesario, pero dado que cambiamos la configuración global de Python 3, conviene reiniciar. Una vez reiniciado el sistema, verifique que Python 3 esté instalado con `python3 --version` y que la versión sea 3.10 o posterior.

En este punto, es posible que sus alarmas mentales estén sonando debido a la línea Python . . . version is 3.10 or greater (La versión de Python es 3.10 o mayor). Este proceso de instalación nos ayudará a implementar nuestra aplicación básica de Python, pero nos deja con la pregunta: ¿Por qué no especificamos la versión exacta de Python que necesitamos?

La instalación de versiones específicas de lenguajes de programación en máquinas virtuales, incluido Python, está fuera del alcance de este libro. En realidad, resolvemos este problema de instalación de versiones directamente cuando comenzamos a utilizar contenedores más adelante en este libro. La razón por la que no instalamos una versión específica de Python 3 es que hay demasiadas formas posibles de instalar Python 3, y nuestro código solo necesita Python 3.8 o superior para ejecutarse en producción.

Ahora es el momento de crear un entorno virtual de Python para aislar los requisitos de software de nuestro proyecto Python del sistema en general (ver listado 3.17). Para ello, crearemos un espacio dedicado en el directorio `/opt/venv` para nuestro entorno virtual. Eliminar el entorno virtual del código del proyecto nos ayudará a mantener el directorio del proyecto limpio y organizado, pero es opcional, al igual que en su equipo local. Usaremos la ubicación `/opt/venv`.

una y otra vez a lo largo de este libro, ya que es mi ubicación preferida para entornos virtuales en sistemas basados en Linux, así como en contenedores Docker.

Listado 3.17 Creación del entorno virtual del lado del servidor

```
python3 -m venv /opt/venv/
```

De ahora en adelante, podremos usar rutas absolutas para referenciar los ejecutables disponibles en nuestro entorno virtual. Aquí hay algunos ejemplos de comandos que probablemente usaremos en el futuro:

- `/opt/venv/bin/python`—Esta es la ruta al ejecutable de Python 3.
- `/opt/venv/bin/pip`: esta es la ruta al ejecutable pip. `/opt/venv/bin/gunicorn`:
- esta es la ruta al ejecutable gunicorn.

En producción, el uso de rutas absolutas ayudará a mitigar los problemas que pueden surgir debido a instalaciones de Python en todo el sistema, así como a conflictos de versiones de paquetes de Python.

Es más, si utiliza rutas y ubicaciones consistentes para sus proyectos, terminará escribiendo menos código de configuración y logrará que sus proyectos se ejecuten de manera más rápida y confiable.

Las rutas absolutas son clave para un entorno de producción repetible y exitoso.

Ahora, tenemos las condiciones para instalar las dependencias de nuestro proyecto Python usando pip desde el ejecutable de Python 3 del entorno virtual. Usaremos el archivo `requirements.txt` que subimos y extrajimos con Git anteriormente en este capítulo, ubicado en `/opt/venv/roadtok8s/py/src/requirements.txt`. Consulte la siguiente lista.

Listado 3.18 Instalación de paquetes de Python con rutas absolutas

```
/opt/venv/bin/python -m pip install -r \
/opt/venv/roadtok8s/py/src/requisitos.txt
```

Estamos casi listos para empezar a ejecutar esta aplicación en el servidor, pero antes, quiero actualizar mi gancho de post-recepción para Git para que el comando del listado 3.18 se ejecute cada vez que enviemos nuestro código a nuestro repositorio remoto. Esto garantizará que nuestro entorno de producción esté siempre actualizado con el código y las dependencias más recientes.

ACTUALIZAR EL GANCHO GIT PARA LA APLICACIÓN PYTHON

Veamos otro caso práctico de uso de los ganchos de Git: la instalación de software de aplicación. Para ello, podemos usar el comando del listado 3.18 en nuestro gancho posterior a la recepción, ya que usamos rutas absolutas al ejecutable de Python y al archivo requirements.txt, como se muestra en el siguiente listado.

Listado 3.19 Actualización del gancho de Git para la aplicación Python

```
exportar ÁRBOL_DE_TRABAJO=/opt/proyectos/roadtok8s/py
exportar DIR_GIT=/var/repos/roadtok8s/py.git

gato <<EOF > "$GIT_DIR/hooks/post-recepción" #!/bin/bash
git --work-
tree=$WORK_TREE --git-dir=$GIT_DIR checkout HEAD -
F

# Instalar los requisitos de Python /opt/venv/bin/
python -m pip install -r $WORK_TREE/src/require
ments.txt
EOF
```

En este caso, sobrescribimos el gancho post-recepción original con el contenido del listado 3.19. Si desea verificar esto

El gancho funciona. Simplemente ejecuta `bash /var/repos/roadtok8s/py.git/hooks/post-receive` y verás el resultado del comando `pip`. Con todas las dependencias instaladas para nuestra aplicación Python, ¡estamos listos para ejecutar el código!

EJECUTAR LA APLICACIÓN PYTHON

Hemos llegado a un punto clave en nuestro viaje: tenemos una aplicación Python que podemos ejecutar en nuestro servidor. Es hora de ejecutarla, comprobar si funciona y verificar que podemos acceder a ella desde nuestro navegador. Ambas cosas deberían ser ciertas, pero solo hay una manera de averiguarlo.

Primero, analicemos los paquetes de Python que necesitamos para ejecutar una aplicación de Python lista para producción:

- **gunicorn:** Gunicorn es un servidor HTTP de interfaz de puerta de enlace de servidor web (WSGI) basado en Python. En otras palabras, gunicorn convierte las solicitudes HTTP en código Python.
Gunicorn es una excelente opción para producción, ya que es un servidor HTTP maduro, estable y rápido. Es flexible, por lo que se puede usar con casi cualquier framework web de Python: FastAPI, Flask, Django, Bottle, Tornado, Sanic y muchos otros. Con gunicorn, incluso puedes crear tu propio framework web de Python desde cero. **uvicorn:** Uvicorn es un servidor HTTP de Interfaz de Puerta de Enlace Asíncrona (ASGI) basado en Python.
- **ASGI** es una especificación de Python para servidores HTTP asíncronos. Para el desarrollo local, podemos usar uvicorn directamente. En producción, simplemente usamos el servidor gunicorn con la clase de trabajador uvicorn.

La configuración base que necesitaremos para gunicorn es la siguiente:

- `--worker-class uvicorn.workers.UvicornWorker_`: Indica a gunicorn que use la clase de trabajador uvicorn , necesaria para las aplicaciones FastAPI (y potencialmente para otras aplicaciones ASGI). `--chdir /opt/projects/roadtok8s/py/src`: Indica a gunicorn
 - que cambie al directorio src antes de ejecutar la aplicación. Podría cambiar el directorio antes de ejecutar la aplicación, pero prefiero usar `--chdir . main:app`: Indica a gunicorn que busque en main.py la variable app , ya que app es una instancia de la clase de aplicación FastAPI() . Gracias al indicador chdir , no es necesario especificar la ruta
 - completa al archivo main.py. `--bind "0.0.0.0:8888"`: Indica a gunicorn dos cosas principales: usar el puerto 8888 en nuestro servidor local y escuchar en todas las direcciones IP. Vincular gunicorn de esta manera significa que deberíamos poder acceder a la aplicación desde nuestro navegador usando la
 - dirección IP pública de nuestro servidor y el puerto 8888.
-
- `--pid /var/run/roadtok8s-py.pid`: Esto le indica a gunicorn que escriba el ID del proceso (PID) en el archivo /var/run/roadtok8s-py.pid. Esto es útil para detener la aplicación posteriormente.

Ahora ejecutemos esta configuración de referencia y veamos si funciona, como se muestra en la siguiente lista.

Listado 3.20 Comando para ejecutar la aplicación Python

```
/opt/venv/bin/gunicorn \  
  --worker-class uvicorn.workers.UvicornWorker \  
  --chdir /opt/projects/roadtok8s/py/src/ \  
  main:app \  
  --bind "0.0.0.0:8888" \  
  --pid /var/run/roadtok8s-py.pid
```

Una vez ejecutado este comando, su aplicación debería iniciarse y devolver el dominio `http://0.0.0.0:8888`, que coincide con el que usamos para enlazar gunicorn. Si tiene otra aplicación ejecutándose en el puerto especificado, este dominio no aparecerá, sino que recibirá un error. Si todo salió bien, debería ver el mismo resultado que en la figura 3.9.

```
[2023-03-08 21:35:58 +0000] [63003] [INFO] Starting gunicorn 20.1.0
[2023-03-08 21:35:58 +0000] [63003] [INFO] Listening at: http://0.0.0.0:8888 (63003)
```

Figura 3.9 Ejecución de la salida de la aplicación Python

Podrías tener la tentación de abrir tu navegador local en `http://0.0.0.0:8888`, pero esto no funcionará porque se ejecuta en un servidor en la nube. Gracias a la asignación de `0.0.0.0`, podemos abrir nuestro navegador en `http://<tu-ip>:8888` y ver la aplicación ejecutándose de forma idéntica a tu versión local; consulta la figura 3.10 para ver un ejemplo de servidor en vivo.

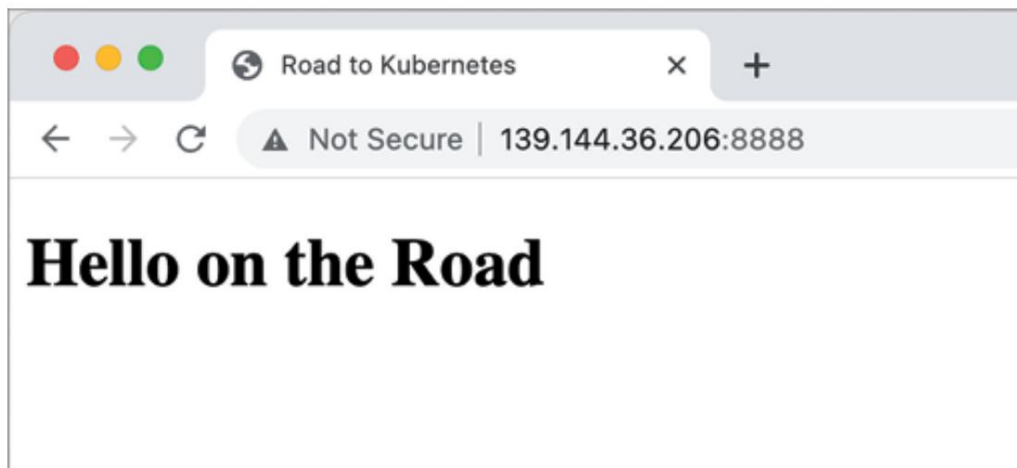


Figura 3.10 Servidor Python ejecutándose en el puerto 8888

¡Felicitaciones! Ha ejecutado correctamente una aplicación Python en un servidor en la nube. Aún tenemos que mejorarla, pero tenemos una base sólida sobre la que construir. Antes de continuar, debemos detener esto.

servidor para que podamos realizar otros cambios en el servidor, incluida la instalación de Node.js y nuestra aplicación Express.js.

Para detener esta aplicación, tenemos tres opciones diferentes:

- Reinicie la máquina virtual (sudo system reboot).
- Utilice Ctrl+C en nuestro teclado como lo haríamos durante el desarrollo local.
- Utilice el ID de proceso almacenado en la ruta /var/run/roadtok8s-py.pid mencionada anteriormente. Podemos ejecutar `kill -9 $(cat /var/run/roadtok8s-py.pid)` para detener la aplicación con el contenido de este archivo, si existe. No elimine el archivo PID, ya que esto no detendrá la aplicación.

Ahora es el momento de pasar a nuestra aplicación Node.js. Instalaremos Node.js y nuestra aplicación Express.js en el mismo servidor que nuestra aplicación Python. Esto facilita considerablemente la implementación, pero dificulta considerablemente el escalado. Escalar estas aplicaciones con fluidez es una de las principales razones para adoptar Kubernetes.

3.4.2 Instalación de Node.js y nuestra aplicación Express.js

Ya hemos ejecutado múltiples entornos de ejecución de aplicaciones en nuestra máquina dos veces con NGINX y Python 3. En esta sección, configuraremos nuestro entorno para nuestra aplicación Node.js y Express.js. Configurar entornos puede ser tedioso, ya que cada herramienta requiere un conjunto diferente de dependencias para su correcto funcionamiento, y cada aplicación también tiene requisitos diferentes. Node.js no es la excepción.

Al igual que nuestro entorno de desarrollo local, necesitaremos instalar la última versión LTS de Node.js para garantizar nuestro

La aplicación funciona correctamente. Para ello, utilizaremos el Administrador de versiones de Node (NVM). NVM facilita enormemente la instalación de varias versiones de Node.js y el Administrador de paquetes de Node (NPM) asociado.

Antes de instalar nvm, podrías pensar que deberías usar el método basado en Linux usando el gestor de paquetes apt y el comando apt-get install nodejs. Si bien este método puede ser adecuado en muchas ocasiones, como vimos con Python, apt-get puede instalar una versión demasiado desactualizada.

Al momento de escribir este libro, apt-get install nodejs instala una versión de Node.js mucho más antigua (versión 12.22.9) que la que necesitamos para ejecutar nuestra aplicación de manera segura.

Dicho esto, instalemos nvm y la última versión LTS de Node.js usando el paquete curl de Linux junto con un script de instalación oficial de nvm .

INSTALACIÓN DEL ADMINISTRADOR DE VERSIONES DE NODO

El script de instalación de Node Version Manager y la documentación relacionada se pueden encontrar en <http://nvm.sh>. Aquí está el proceso que seguiremos para instalarlo:

- Declaramos una variable bash NVM_VERSION y la establecemos en la versión de nvm que queremos instalar, y en este caso, Utilice v0.39.3.
- curl -o- <your-url>: Usar curl -o- abrirá una URL y mostrará la respuesta en la terminal. Es similar a "echo 'my statement'" , pero para URL. Esto nos permitirá encadenar comandos con una tubería |. bash: Enviaremos la respuesta desde la URL e indicaremos a la línea de comandos (en este caso, la
- shell bash) que ejecute el script.

Veamos cómo se ve este comando en la práctica, como se muestra en la siguiente lista.

Listado 3.21 Instalación del administrador de versiones de nodo

```
exportar NVM_VERSION="v0.39.3" #1 curl
-o- \ https://
    raw.githubusercontent.com/nvm-sh/nvm/$NVM_VERSION/install.sh \

| bash
```

#1 Reemplace v0.39.3 con la versión de nvm que desea instalar.

Una vez finalizada la instalación, los resultados deberían corresponder a la figura 3.11.

```
root@localhost:~# export NVM_VERSION="v0.39.3"
root@localhost:~# curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/
$NVM_VERSION/install.sh | bash
  % Total    % Received % Xferd  Average Speed   Time    Time     Time
Current                                  Dload  Upload   Total   Spent    Left
Speed
100 15916  100 15916    0     0  110k      0 --:--:-- --:--:-- --:--:--
 110k
=> nvm is already installed in /root/.nvm, trying to update using git
=> => Compressing and cleaning up git repository

=> nvm source string already in /root/.bashrc
=> bash_completion source string already in /root/.bashrc
=> Close and reopen your terminal to start using nvm or run the following
to use it now:

export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" # This loads nvm
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion" # Th
is loads nvm bash_completion
```

Figura 3.11 Instalación de la salida del Node Version Manager

Para utilizar nvm de inmediato, necesita:

- Cierre y vuelva a abrir su conexión SSH.

- Recargue su perfil bash con la fuente ~/.bashrc.

Ambos métodos son válidos y funcionarán, pero suelo recargar mi perfil de bash porque es un poco más rápido. El perfil de bash es básicamente un archivo con una serie de opciones de configuración del usuario para comandos y atajos.

Con nvm instalado y listo, verifiquemos que funciona e instalemos la última versión LTS de Node.js (al momento de escribir este libro, v18.15.0) usando nvm install.

Listado 3.22 Verificación de que nvm está instalado

```
nvm --versión #1 nvm
install --lts #2 nvm install
18.15.0 #3
```

#1 Si esto falla, intente salir de su sesión SSH y volver a conectarse.
 #2 Opción 1 para instalar la última versión LTS de Node.js #3 Opción 2 para instalar una versión específica de Node.js

A medida que se completa nvm , debería ver el siguiente resultado (figura 3.12).

```
root@localhost:~# nvm install --lts
Installing latest LTS version.
Downloading and installing node v18.15.0...
Downloading https://nodejs.org/dist/v18.15.0/node-v18.15.0-linux-x64.tar.xz...
#####
#####
##### 100.0%
Computing checksum with sha256sum
Checksums matched!
Now using node v18.15.0 (npm v9.5.0)
```

Figura 3.12 Instalación de Node.js a través de la salida de nvm

Suponiendo que la instalación de Node.js a través de nvm fue exitosa, su sistema ahora debería tener los siguientes comandos:

- nodo: el entorno de ejecución de Node.js; verifique la versión instalada con node -version.

- npm—El administrador de paquetes de Node; verifique la versión instalada con `npm -version`.
- npx—Una utilidad para ejecutar paquetes de Node.js; verifique la versión instalada con `npx -version`.

Ahora tienes una forma repetible de instalar Node.js y las herramientas asociadas en tu equipo. Esta es una excelente manera de garantizar que puedas instalar la misma versión de Node.js en cualquier equipo Linux que necesites. nvm también es compatible con otros sistemas operativos; simplemente no es tan fácil de instalar como en Linux y Ubuntu. Ahora por fin podemos instalar nuestras dependencias de Express.js y ejecutar nuestra aplicación.

INSTALE LAS DEPENDENCIAS DE NUESTRAS APLICACIONES

Como discutimos en el capítulo 2, usaremos npm para instalar las dependencias de nuestra aplicación para nuestra aplicación Express.js. Comenzaremos navegando a nuestro directorio de aplicación en `/opt/projects/roadtok8s/js/`, actualizaremos la versión global de npm, instalaremos nuestras dependencias declaradas en `package.json` y luego verificaremos que las dependencias se instalaron correctamente con `npm list`, como se muestra en la siguiente lista.

Listado 3.23 Instalación de las dependencias de la aplicación Express.js

```
cd /opt/projects/roadtok8s/js npm install -g
npm@latest npm install lista npm
```

Al igual que antes, esto creará un directorio `node_modules` en el directorio de nuestra aplicación. Con las aplicaciones Node.js, no cambiamos la ubicación de `node_modules` como hicimos con la ubicación de `venv` para nuestro proyecto Python, ya que las aplicaciones Node.js no requieren este paso, para bien o para mal. Ahora por fin podemos obtener nuestras dependencias de Express.js.

Actualicemos nuestro gancho posterior a la recepción para instalarlos cada vez que se envíe una nueva confirmación a nuestro repositorio.

ACTUALIZAR EL GANCHO GIT PARA LA APLICACIÓN JAVASCRIPT

Nuevamente, debemos actualizar nuestro gancho post-recepción para instalar las dependencias de nuestra aplicación. En la siguiente lista, usaremos el comando npm para instalarlas.

Listado 3.24 Actualización del gancho de Git para la aplicación Node

```
exportar ÁRBOL_DE_TRABAJO=/opt/proyectos/roadtok8s/js exportar
DIR_GIT=/var/repos/roadtok8s/js.git

gato <<EOF > "$GIT_DIR/hooks/post-recepción" #!/bin/bash git --
work-
tree=$WORK_TREE --git-dir=$GIT_DIR checkout HEAD -
F

# Instalar los requisitos de Node.js cd $WORK_TREE
npm install

EOF
```

En este caso, sobrescribimos el gancho post-recepción original con el contenido del listado 3.24. Para verificar el funcionamiento de este gancho, ejecute `bash /var/repos/roadtok8s/js.git/hooks/post-receive` y debería ver la salida del comando `npm install`. Con todas las dependencias instaladas para nuestra aplicación Express.js, ¡estamos listos para ejecutar el código!

3.4.3 Ejecución de la aplicación Express.js

En entornos de producción, prefiero usar rutas absolutas a los módulos que queremos ejecutar. En nuestro caso, ejecutaremos el módulo `main.js` en el directorio `src` de nuestra aplicación JavaScript. También declararemos un puerto aquí para garantizar que nuestra aplicación sea flexible ante cambios en el puerto, ya que esto será útil cuando comencemos a desactivar puertos en secciones posteriores. El comando es tan simple como el siguiente.

Listado 3.25 Ejecución de la aplicación Node.js

```
PUERTO=5000 nodo /opt/proyectos/roadtok8s/js/src/main.js
```

Ahora debería poder abrir su navegador en `http://<su-dirección-ip>:5000` y ver la figura 3.13.



Figura 3.13 Express.js Hola Mundo

¡Felicitaciones! Has instalado Node.js y lo has conectado correctamente.

Para detener esta aplicación, tenemos tres opciones diferentes:

- Reinicie la máquina virtual (`sudo system reboot`).
- Utilice `Control+C` en nuestro teclado como lo haríamos durante el desarrollo local.
- Utilice el ID de proceso que se almacena en el mismo directorio del módulo `main.py` como nuestro `en /opt/projects/roadtok8s/js/src/main.pid`. Este es un

Ubicación ligeramente diferente a la de Python, pero el concepto es el mismo.
Ejecuta `kill -9 $(cat /opt/projects/roadtok8s/js/src/main.pid)`.

En este punto, deberíamos poder ejecutar dos entornos de ejecución diferentes para dos aplicaciones distintas. El problema es garantizar que estos entornos de ejecución se ejecuten sin intervención manual. Para ello, usaremos una herramienta llamada Supervisor.

3.5 Ejecutar múltiples aplicaciones en segundo plano con Supervisor

Supervisor es una forma sencilla de convertir nuestros comandos de ejecución de aplicaciones en procesos en segundo plano que se iniciarán, detendrán o reiniciarán cuando nuestro sistema lo haga o cuando lo necesitemos. El supervisor ciertamente no es la única manera de hacerlo, pero es bastante accesible.

El proceso de configuración para que cualquier aplicación sea administrada por Supervisor es el siguiente:

- Cree un archivo de configuración de Supervisor para la aplicación en `/etc/supervisor/conf.d/`.
- Actualice el archivo de configuración del Supervisor con la información correcta para la aplicación (por ejemplo, directorio de trabajo, comando a ejecutar, ubicaciones de registro, etc.).
- Actualice el gancho posterior a la recepción de Git para varios comandos basados en Supervisor para garantizar que se esté ejecutando la versión correcta de la aplicación.

Comencemos instalando Supervisor y creando un archivo de configuración para nuestra aplicación Python.

3.5.1 Instalación de Supervisor

Instalar Supervisor es tan fácil como instalar NGINX utilizando el administrador de paquetes apt , como se muestra en la siguiente lista.

Listado 3.26 Instalación de Supervisor en Ubuntu

```
sudo apt update  
sudo apt install supervisor -y
```

Al instalar Supervisor, se creará una nueva carpeta en `/etc/supervisor/conf.d/`. Aquí almacenaremos los archivos de configuración de cada aplicación que queramos administrar con Supervisor.

A continuación se muestran algunos comandos útiles para recordar cuando se trabaja con Supervisor:

- `sudo supervisorctl status`: enumera todas las aplicaciones administradas actualmente por Supervisor. `sudo supervisorctl update`: actualiza los archivos de configuración de Supervisor. Es lo mismo que `sudo supervisorctl reread && sudo supervisorctl update`. `sudo supervisorctl reread`: lee los archivos de configuración de Supervisor.
- `sudo supervisorctl start <app-name>`: inicia la aplicación con el nombre `<app-name>`. `sudo supervisorctl stop <app-name>`: detiene la aplicación con el nombre `<app-name>`. `sudo supervisorctl restart <app-name>`: reinicia la aplicación con el nombre `<app-name>`.
-

Usamos algunos de estos comandos mientras configuramos nuestras aplicaciones.

3.5.2 Configurar Supervisor para aplicaciones

Para configurar Supervisor para cualquier aplicación determinada, necesitaremos hacer lo siguiente dentro de un archivo de configuración:

- Nombra el proceso Supervisor ([programa:tuombre]).
- Define el directorio de trabajo para la aplicación (directorio=/ruta/a/tu/aplicación).
- Define el comando para ejecutar la aplicación (comando=/ruta/a/tu/comando).
- Decide si la aplicación debe iniciarse automáticamente, reiniciarse automáticamente y cuántas veces debe volver a intentar iniciarse. (autostart=true, autorestart=true, startretries=3).
- Define la ubicación de registro para la aplicación en relación con stdout y stderr. (stderr_logfile=/ruta/a/su/registro, stdout_logfile=/ruta/a/su/registro).

Comencemos con la aplicación Python creando un archivo de configuración. La configuración del supervisor se muestra en la siguiente lista.

Listado 3.27 Configuración del supervisor para Python

```
exportar APP_CMD="/opt/venv/bin/gunicorn \
    --worker-class uvicorn.workers.UvicornWorker \ main:app --bind "0.0.0.0:8888" \ --pid /
    var/run/roadtok8s-py.pid" cat << EOF > /etc/supervisor/
    conf.d/roadtok8s-py.conf [programa:roadtok8s-py] directorio=/
    opt/projects/roadtok8s/py/src comando=$APP_CMD autostart=true autorestart=true startretries=3
    archivo_registro_stderr=/var/log/
    supervisor/roadtok8s/py/stderr.log archivo_registro_stdout=/var/log/
    supervisor/roadtok8s/py/
    stdout.log EOF
```

En este punto, ver esta configuración debería ser bastante intuitivo. Creemos nuestro archivo de configuración de Node.js, como se muestra en la siguiente lista.

Listado 3.28 Configuración del supervisor para Node.js

```
export NODE_PATH=$(cual nodo)#1 cat <<
EOF > /etc/supervisor/conf.d/roadtok8s-js.conf [programa:roadtok8s-js]
directorio=/opt/projects/
roadtok8s/js/src comando=$NODE_PATH main.js

inicio automático=verdadero
inicio automático=verdadero
inicioreinicios=3

archivo_registro_stderr=/var/log/supervisor/roadtok8s/js/stderr.log archivo_registro_stdout=/var/log/
supervisor/roadtok8s/js/stdout.log EOF
```

#1 Obtiene la ruta al ejecutable Node.js según la configuración de su sistema

Con ambas configuraciones en su lugar, necesitamos crear los directorios para nuestros archivos de registro de salida, como se muestra en la siguiente lista.

Listado 3.29 Creación de directorios de registro

```
sudo mkdir -p /var/log/supervisor/roadtok8s/py sudo mkdir -p /
var/log/supervisor/roadtok8s/js
```

Ahora tenemos todas las condiciones necesarias para ejecutar estas aplicaciones mediante Supervisor. Ejecutemos los siguientes comandos para actualizar la configuración de Supervisor e iniciar las aplicaciones.

Listado 3.30 Actualización de la configuración del supervisor

```
sudo supervisorctl update
```

La figura 3.14 muestra la salida de este comando.

```
roadtok8s-js: added process group
roadtok8s-py: added process group
```

Figura 3.14 Procesos de supervisor agregados

Dado que declaramos `autostart=true` en nuestros archivos de configuración, las aplicaciones deberían iniciarse automáticamente. Verifiquemos que se estén ejecutando en la siguiente lista.

Listado 3.31 Comprobación del estado de Supervisor

```
sudo supervisorctl estado
```

Si configuramos todo correctamente deberíamos ver lo siguiente (figura 3.15).

```
root@localhost:~# sudo supervisorctl status
roadtok8s-js                RUNNING    pid 89743, uptime 0:00:04
roadtok8s-py                RUNNING    pid 89738, uptime 0:00:09
root@localhost:~# █
```

Figura 3.15 Salida del estado del supervisor

También deberíamos poder verificar que nuestras aplicaciones se estén ejecutando en

- `http://<your-ip>:8888` (Python vía Gunicorn) `http://<your-ip>:3000` (Express vía Node.js)

Para iniciar, detener, reiniciar u obtener el estado de cualquier aplicación administrada por Supervisor, es tan sencillo como: `sudo supervisorctl <verb> <app-name>` donde `<app-name>` es `roadtok8s-py` o `roadtok8s-js`. Aquí hay algunos ejemplos:

- `sudo supervisorctl iniciar roadtok8s-py`

- `sudo supervisorctl detener roadtok8s-py`
- `sudo supervisorctl reiniciar roadtok8s-js`
- `sudo supervisorctl estado`

El paso final de este proceso es actualizar nuestros ganchos posteriores a la recepción para reiniciar cualquiera de estas aplicaciones después de que se envíe la confirmación y se completen las instalaciones, como se muestra en las siguientes listas.

Listado 3.32 Gancho de posrecepción del supervisor

```
exportar ÁRBOL_DE_TRABAJO=/opt/proyectos/roadtok8s/py exportar
DIR_GIT=/var/repos/roadtok8s/py.git

gato <<EOF > "$GIT_DIR/hooks/post-recepción" #!/bin/bash git --work-
tree=$WORK_TREE
--git-dir=$GIT_DIR checkout HEAD - f

# Instalar los requisitos de Python /opt/venv/bin/python -m pip
install -r $WORK_TREE/src/requirements.txt

# Reiniciar la aplicación Python en Supervisor sudo supervisorctl restart roadtok8s-py

EOF
```

Listado 3.33 Actualización del gancho posterior a la recepción del supervisor para Node.js

```
exportar ÁRBOL_DE_TRABAJO=/opt/proyectos/roadtok8s/js exportar DIR_GIT=/
var/repos/roadtok8s/js.git

gato <<EOF > "$GIT_DIR/hooks/post-recepción" #!/bin/bash git --work-
tree=$WORK_TREE
--git-dir=$GIT_DIR checkout HEAD - f

# Instalar los requisitos de Node.js cd $WORK_TREE npm
install

# Reiniciar la aplicación Node.js en Supervisor sudo supervisorctl restart roadtok8s-js
EOF
```

Ahora que estas aplicaciones están siempre en ejecución, debemos empezar a pensar en cómo acceder a ellas. Para ello, implementaremos NGINX como proxy inverso para dirigir el tráfico a la aplicación correcta según la URL, en lugar de usar un número de puerto. Después, implementaremos un firewall para limitar el acceso a esta máquina virtual solo a los puertos necesarios.

3.6 Sirva múltiples aplicaciones con NGINX y un firewall

NGINX es mucho más que una herramienta para servir sitios web estáticos; también puede utilizarse para redireccionar tráfico hacia arriba a otra aplicación. En nuestro caso, esta otra aplicación estará en el mismo servidor que NGINX, pero, en teoría, podría existir prácticamente en cualquier lugar siempre que NGINX pueda acceder a ella (por ejemplo, a través de una dirección IP pública o privada).

Este proceso se denomina configuración de un proxy inverso. El objetivo de usar proxies inversos es redirigir el tráfico, ocultar aplicaciones que se estén ejecutando en un servidor (p. ej., Python o Node.js), implementar un balanceador de carga y mucho más.

El balanceo de carga es simplemente el proceso de reenviar el tráfico a un servidor que pueda gestionarlo. Si el servidor no puede gestionarlo, el balanceador de carga lo reenviará a otro servidor si puede. El balanceo de carga puede ser más complejo, pero esa es la idea general. NGINX también está diseñado para gestionar el balanceo de carga, pero es un concepto que no profundizaremos en este libro.

3.6.1 Configuración de NGINX como proxy inverso

Queremos configurar NGINX para que reenvíe el tráfico a nuestro Aplicaciones basadas en supervisor que se ejecutan en las siguientes ubicaciones:

- `http://localhost:8888` (Python a través de gunicorn) `http://localhost:3000` (Express a través de Node.js)

Cabe señalar que estos son nombres de dominio local (DNS), pero fácilmente podrían ser direcciones IP u otros nombres de dominio público.

Configuramos ambas aplicaciones para que se ejecuten en este servidor (localhost) y en los puertos predeterminados de cada aplicación. Podemos configurar NGINX para que reenvíe tráfico a estas aplicaciones añadiendo lo siguiente a nuestro archivo de configuración de NGINX, como se muestra en la siguiente lista.

Listado 3.34 Configuración del proxy inverso de NGINX

```

gato <<EOF > /etc/nginx/sites-available/roadtok8s servidor { escuchar 80;

nombre_del_servidor
    localhost;

    ubicación / {#1
        proxy_pass http://localhost:8888;
    }

    ubicación /js/ {#2 proxy_pass
        http://localhost:3000/;
    }
}
EOF

```

#1 Ubicación / será el índice raíz de este servidor que se reenvía a la aplicación Python.

#2 La ubicación /js/ con la barra diagonal final se reenviará a la aplicación Express.js. Si omite la barra diagonal final, es posible que la aplicación Express.js no funcione correctamente.

La ubicación /etc/nginx/sites-available/roadtok8s permite a NGINX saber que se trata de un posible archivo de configuración. Podemos habilitar esta configuración creando un enlace simbólico al archivo en /etc/nginx/sites-enabled/. Para ello, ejecutamos el código de la siguiente lista.

Listado 3.35 Creación de un enlace simbólico al archivo de configuración de NGINX

```

sudo ln -s /etc/nginx/sites-available/roadtok8s \ /etc/nginx/sites-enabled/
roadtok8s

```

Después de crear este enlace simbólico, debemos eliminar el archivo de configuración predeterminado que NGINX crea cuando se instala usando `sudo rm /etc/nginx/sites-enabled/default`, y luego reiniciaremos NGINX con `sudo systemctl restart nginx`.

systemctl es un administrador de procesos integrado, muy parecido a Supervisor.

que NGINX usa por defecto (systemctl es un poco más complejo de configurar que Supervisor , por eso no lo usamos antes en el capítulo).

Ahora abra su navegador en las siguientes ubicaciones:

- `http://<your-ip>`; se servirá a través de Python y gunicorn. `http://<your-ip>/js` (sin barra final) se servirá a través de Express mediante Node.js.

Ahora contamos con una forma robusta de enrutar el tráfico a nuestras aplicaciones, pero hay una falla evidente: ¡nuestros puertos también siguen siendo accesibles! Para solucionarlo, instalemos un firewall.

3.6.2 Instalación de un firewall sencillo

Se recomienda encarecidamente agregar un firewall a la máquina virtual porque queremos denegar todo el tráfico que no sea uno de los siguientes:

- Tráfico HTTP a través del puerto 80
- Tráfico HTTPS a través del puerto 443
- Tráfico SSH a través del puerto 22

El Firewall Simple (UFW) es una forma sencilla y eficaz de permitir que solo los puertos anteriores estén disponibles para el internet externo. En otras palabras, UFW bloqueará automáticamente el acceso a cualquier puerto que no necesite mantener abierto. Bloquearemos todos los puertos excepto los que usa NGINX (80 y 443) y para nuestras conexiones SSH (22).

Instalemos UFW:

```
sudo apt update#1 sudo apt  
install ufw -y#2
```

#1 Siempre que realice la instalación, actualice los paquetes apt .

#2 Instale UFW si aún no está instalado.

De forma predeterminada, UFW está deshabilitado. Antes de habilitarlo, debemos configurarlo para permitir el tráfico SSH y NGINX.

UFW cuenta con muchas funciones excelentes que se pueden aprovechar, pero para nuestros fines, solo permitiremos el tráfico SSH y NGINX. Queremos que se permita el tráfico SSH para poder acceder a nuestra máquina virtual por SSH y realizar envíos mediante Git. También queremos permitir todo el tráfico basado en NGINX para garantizar que nuestras aplicaciones web sean accesibles. Para ello, ejecutaremos los siguientes comandos:

```
sudo ufw permitir ssh sudo  
ufw permitir 'Nginx completo'
```

Si olvida permitir el tráfico SSH, su sesión SSH actual finalizará y podría perder el acceso para siempre. En ese caso, deberá eliminar su máquina virtual y empezar de nuevo.

Nginx Full admite tráfico HTTP y HTTPS, que se asignan a los puertos 80 y 443, respectivamente. No ejecutaremos HTTPS por el momento, pero está listo y disponible si lo necesitamos.

Después de ejecutar cada uno de estos comandos, verá el resultado Reglas actualizadas o Reglas actualizadas (v6) como respuesta.

Antes de habilitar estos cambios, verifiquemos la configuración actual.
Configuración de UFW:

```
Se agregó el programa ufw
```

Esto debería responder con

```
Se agregaron reglas de usuario (consulte 'ufw status' para ver el firewall en  
ejecución): ufw allow  
22/tcp ufw allow 'Nginx Full'
```

Ahora podemos habilitar nuestro firewall:

```
sudo ufw habilitar
```

Esto casi seguramente responderá con

```
El comando puede interrumpir las conexiones SSH existentes. ¿Continuar con la  
operación (y|n)?
```

Debe responder con y y presionar Enter con el resultado El firewall está activo y habilitado al iniciar el sistema.

Si todo salió bien, su firewall ya está activo y su seguridad ha mejorado. Sin duda, existen otras medidas que podría considerar para proteger aún más su máquina virtual, pero estas quedan fuera del contexto de este libro.

Ahora que tenemos un firewall instalado, podemos probar nuestra configuración de NGINX abriendo una nueva ventana del navegador y accediendo a `http://<your-ip>` y `http://<your-ip>/js`. Deberías ver los mismos resultados que antes.

La primera vez que implementé una aplicación, usé una variante de lo que hicimos en este capítulo. Me fue útil durante muchos años y también a otros durante ese mismo tiempo. Incluso tenía mi base de datos ejecutándose en la misma máquina.

En este punto, debería sentirse un poco incómodo con la fragilidad de esta configuración. Técnicamente funciona, pero tenemos un problema importante que debe abordarse: si esta máquina falla, toda la pila de producción también lo hará. En otras palabras,

En otras palabras, esta máquina es un punto único de fallo, y aquí está el porqué.
La máquina es

- Alojamiento de los repositorios Git de producción
- Creación y actualización de dependencias y entornos de aplicaciones
- Ejecutar dos aplicaciones principales en segundo plano
- Alojamiento y ejecución del servidor web NGINX

Aquí hay algunas cosas que podría considerar para ayudar a mitigar este problema:

- Mejora esta máquina y hazla más potente.
- Realice una copia de seguridad de esta máquina y su configuración para que podamos aprovisionar rápidamente otra si esta deja de funcionar.
- Cree una réplica de esta máquina y utilice un balanceador de carga para distribuir el tráfico entre las dos.

Si bien estas ideas pueden ser útiles, no resuelven el problema subyacente. La mejor solución es dividir cada componente de nuestra pila de producción en máquinas o servicios dedicados que puedan ejecutarse de forma independiente.

Lo interesante es que, en esencia, configuramos un entorno de desarrollo remoto con características similares a las de producción, con el que el público puede interactuar. Si elimináramos el acceso público (mediante UFW) y solo permitiéramos el acceso SSH, tendríamos una forma eficaz de escribir nuestro código desde prácticamente cualquier dispositivo usando solo SSH, un nombre de usuario y una contraseña.

Resumen

- SSH es una forma poderosa de acceder y administrar máquinas remotas prácticamente en cualquier parte del mundo.
- NGINX es una herramienta que permite el alojamiento de sitios web estáticos, así como funciones más robustas como servidores proxy inversos y balanceadores de carga.
- Instalar y configurar el software manualmente en una máquina remota nos brinda una visión más profunda de cómo usar el software y cuáles son los resultados esperados.
- Las aplicaciones web Python y Node.js normalmente no se ejecutan directamente, sino a través de un servidor web como NGINX.
- Las aplicaciones web deben ejecutarse como procesos en segundo plano para garantizar que siempre se estén ejecutando con herramientas como Supervisor.
- Configurar un host privado para nuestro código es una excelente manera de tener copias de seguridad redundantes, así como una forma de implementar código en producción.
- Los firewalls son una forma sencilla de bloquear puertos específicos para que no tengan acceso no deseado a nuestra máquina.

4 Implementación con GitHub

Comportamiento

Este capítulo cubre

- Uso de una canalización de CI/CD
- Configuración y uso de GitHub Actions
- Enriquecer los flujos de trabajo de GitHub Actions con secretos
- Automatización basada en documentos con Ansible

Cuando aprendes a conducir, todo es nuevo, así que cada paso debe considerarse cuidadosamente. Después de unos meses o años, empiezas a conducir sin pensar en cada detalle. Creo que este es el estado natural de todos los humanos: aprendemos y nos concentramos intensamente hasta que se vuelve automático y sin esfuerzo. Con el software ocurre algo similar. Nuestra necesidad innata de desviar la atención de cualquier tarea se refleja en cómo implementamos nuestro software en producción.

En el capítulo 3, implementamos elementos de automatización mediante ganchos de Git y Supervisor. Estos elementos siguen siendo importantes, pero les falta algo: la repetibilidad automatizada. En otras palabras, si queremos poner en marcha un nuevo servidor con la misma configuración, tenemos que repetir manualmente todos los pasos. Para lograr la repetibilidad automatizada más adelante en este capítulo, usaremos Ansible, una popular herramienta de línea de comandos basada en Python. Ansible utiliza un enfoque de automatización basado en documentos, lo que significa que declaramos nuestro estado deseado y Ansible se asegurará de que se cumpla. Este estado deseado incluiría elementos como que NGINX esté instalado, configurado y en ejecución; que Supervisor esté instalado, configurado y en ejecución;

Y así sucesivamente. Ansible es una herramienta muy potente que, en este capítulo, solo exploraremos superficialmente para explicar su aplicación a las tareas que vamos a automatizar, continuando así el trabajo que iniciamos en el capítulo 3.

Además de la automatización de la configuración mediante Ansible, también eliminaremos los ganchos de Git de nuestro flujo de trabajo para sustituirlos por pipelines de CI/CD. Estos pipelines pueden gestionar algunos aspectos críticos, como la ejecución de la automatización de la configuración y las pruebas del código para garantizar que esté listo para producción. Además, podemos usar pipelines de CI/CD para almacenar las variables y secretos del entorno de ejecución (p. ej., contraseñas específicas de la aplicación, claves API, claves SSH, etc.) y así convertirlos en una única fuente de información para nuestros proyectos. Esto es posible independientemente de cómo se utilicen (u obtengan) dichas variables y secretos en nuestro código.

El valor que se libera con este tipo de automatización se reduce a la portabilidad y la escalabilidad. Crear software adaptable es esencial para mantener un proyecto adaptable y controlar los costos, a la vez que se presta servicio a los usuarios, clientes o cualquier persona que necesite el software.

Comenzaremos profundizando en nuestra primera canalización de CI/CD, a modo de "Hola Mundo", para mostrar lo sencillo que puede ser gestionar software mediante flujos de trabajo de CI/CD basados en Git.

4.1 Introducción a las canalizaciones de CI/CD con GitHub Actions

Llevar código a producción es increíblemente emocionante. Es cuando se pone en práctica y cuando puedes tener un gran impacto en la vida de los usuarios a los que quieres servir. O podrías estar lanzando un código fallido. En cualquier caso, la producción es donde...

La magia ocurre porque incluso los programas fallidos pueden enseñarnos lecciones valiosas que están más allá del alcance del desarrollo de software.

La integración continua, o CI, es el proceso de combinar código de diversas fuentes (es decir, varios desarrolladores, varias máquinas) en un solo proyecto. Incluso si trabajas solo, es muy recomendable combinar el código nuevo con el antiguo periódicamente, incluso si eliminas por completo el código antiguo. El control de versiones con Git simplifica el proceso de integración continua y nos da la confianza para realizar grandes cambios en nuestro código base sin temor a perder nuestro trabajo.

Al fusionar o integrar código, es muy recomendable probarlo automáticamente para garantizar su estabilidad y funcionamiento. Existen numerosos libros sobre la escritura de pruebas automatizadas, también llamadas pruebas unitarias, para cualquier lenguaje de programación. Por lo tanto, aunque este libro no aborda este tema, es importante comprender que las pruebas automatizadas son un componente fundamental de la integración continua y, por lo tanto, de la entrega o implementación continua.

La entrega continua, o CD, es el proceso de implementación automática de código en un entorno de preproducción, pero similar a uno de producción. Este entorno se conoce a menudo como entorno de pruebas. Su propósito principal es realizar una revisión manual del código, así como el resultado de cualquier prueba automatizada. Una vez aprobado el entorno de pruebas, el código pasa a la fase final de producción, también conocida como "en vivo". La implementación continua es similar a la entrega continua, excepto que el código se implementa automáticamente en producción sin aprobación manual; es una automatización integral que incluye la compilación, las pruebas y la implementación en producción.

En este libro, realizaremos CI/CD con una herramienta popular llamada GitHub Actions. GitHub Actions es un servicio gratuito de GitHub.com que nos permite ejecutar código en sus servidores al realizar cambios en nuestros repositorios.

GitHub Actions puede usarse para ejecutar todo tipo de código y automatizaciones, pero lo usaremos principalmente para implementar nuestras aplicaciones y configurar nuestros entornos. Existen muchas herramientas de pipeline de CI/CD que realizan las mismas acciones que realizaremos en GitHub Actions, incluyendo opciones de código abierto como GitLab y nektos/act (consulta el apéndice D para saber cómo usar act).

4.1.1 Su primer flujo de trabajo de GitHub Actions

GitHub Actions es un sistema que ejecuta código en tu nombre, basado en tus archivos de definición de flujo de trabajo a nivel de repositorio. El proceso es el siguiente:

1. Cree un repositorio en GitHub.com para su proyecto.
2. Cree un archivo de definición de flujo de trabajo (o varios archivos) en su repositorio siguiendo una ruta y un formato de archivo específicos (más sobre esto en breve).
3. Confirme los archivos de flujo de trabajo y envíe su código a GitHub.
4. Ejecute flujos de trabajo a pedido o automáticamente según sus archivos de definición de flujo de trabajo.

Este proceso es el mismo para muchas herramientas de CI/CD que existen; GitHub Actions es solo una de muchas.

Los flujos de trabajo que defina dependen de usted y pueden ser tan simples o complejos como necesite. Se ejecutarán en los servidores de GitHub y podrán ejecutar cualquier código que les indique, siempre que sea posible. La duración y la frecuencia de ejecución del código dependen de usted. Es importante tener en cuenta que...

Aunque GitHub Actions tiene un nivel gratuito generoso, cuanto más lo uses, más te costará.

Para este libro y muchos otros proyectos, he usado GitHub Actions miles de veces y aún no he pagado un céntimo; a ti también te debería pasar. Si usar GitHub Actions te preocupa, como podría ser para algunos, también usaremos nektos/act, la alternativa autoalojada y de código abierto a GitHub Actions, que es prácticamente un sustituto directo de GitHub Actions.

Ahora, construyamos un flujo de trabajo "Hola Mundo" para comprender cómo definir y ejecutar flujos de trabajo. En el capítulo 2, creamos dos repositorios de GitHub; para esta sección, usaré mi repositorio roadtok8s-py (<https://github.com/jmitchel3/roadtok8s-py>).

En la raíz de tu proyecto de Python, crea la ruta `.github/workflows/`. Esta ruta es necesaria para que GitHub Actions reconozca los archivos de definición de tus flujos de trabajo. Para usuarios de Git más avanzados, los flujos de trabajo solo funcionarán en tu rama predeterminada (por ejemplo, principal o maestra), así que asegúrate de crear estas carpetas en tu rama predeterminada.

Ahora que hemos creado la carpeta base del flujo de trabajo de GitHub Actions, agregaremos un archivo llamado `hello-world.yaml` como nuestro primer flujo de trabajo con la ruta resultante `.github/workflows/hello-world.yaml`. El flujo de trabajo definirá los siguientes elementos:

- El nombre del flujo de trabajo
- Cuándo debe ejecutarse el flujo de trabajo
- Un solo trabajo para ejecutar (los flujos de trabajo pueden tener muchos trabajos)
- ¿Qué sistema operativo utilizar (esto se hace a través de imágenes de contenedores Docker)?
- Los pasos, o código, a ejecutar en el trabajo

Necesitamos convertir estos elementos al formato YAML, ya que es lo que requieren los flujos de trabajo de GitHub Actions. YAML es un formato de serialización de datos legible para humanos, comúnmente utilizado para configuración y programación declarativa. YAML suele ser ideal para revisiones rápidas, ya que es muy fácil de leer, como se puede ver en la siguiente lista.

Listado 4.1 Conceptos básicos de YAML

```
nombre: Hola mundo
elementos:
  subelemento:
    nombre: Hola
    descripción: mundo sublista:
  - elemento a

  - elemento b otra-
lista: [1, 2, 3]
```

Si bien el listado 4.1 es solo un ejemplo simple, puedes ver el formato en acción.

YAML se utiliza comúnmente para programación declarativa, que se preocupa por el resultado y no por los pasos para llegar a él.

La programación imperativa se centra en los pasos para obtener el resultado.

Python y JavaScript son ejemplos de programación imperativa, ya que diseñamos cómo fluyen los datos y qué debe hacer la aplicación. YAML se puede utilizar para ambos tipos de paradigmas de programación.

El Listado 4.2 es un archivo con formato YAML que los flujos de trabajo de GitHub Actions pueden comprender. GitHub Actions es un ejemplo de programación declarativa porque declaramos lo que queremos que se haga, independientemente de cómo se haga exactamente.

Listado 4.2 Hola mundo con acciones de GitHub

```
nombre: Hola mundo
en:
  flujo de trabajo_despacho:
  push:

trabajos:
  construir:
    Ejecuta en: ubuntu-latest Pasos: -
    Usos:
      acciones/checkout@v3 - Nombre: Hola Mundo
      Ejecuta: echo "Hola Mundo"
```

Después de crear este archivo localmente, asegúrese de confirmarlo y enviarlo a GitHub con lo siguiente:

```
git add .github/workflows/hello-world.yaml git commit
-m "Crear flujo de trabajo hello-world.yaml" git push origin main
```

Para comprender mejor qué hace este flujo de trabajo, analicémoslo:

- `on:push`: Esto indica a GitHub Actions que ejecute este flujo de trabajo cada vez que subimos código a nuestro repositorio. Podemos limitar el alcance de esta configuración, pero por ahora, la simplificaremos. `on:workflow_dispatch`: Esto indica a
- GitHub Actions que este flujo de trabajo se puede activar manualmente desde la interfaz de usuario de GitHub.com. Recomendando usar este activador manual con frecuencia al aprender GitHub Actions. `runs-on: ubuntu-latest`: Ubuntu es una de las opciones más flexibles para GitHub Actions, por lo que nos
- quedaremos con esta opción. Existen otras opciones para otros sistemas operativos, pero su uso queda fuera del alcance.

de este libro. Considere leer más sobre GitHub

Action Runners en los documentos oficiales [el](#)

<https://docs.github.com/es/actions/using-github-hosted-runners/about-github-hosted-runners>) y [el](#)

Revisa las imágenes disponibles en GitHub

<https://github.com/actions/runner-images#imagenes-disponibles>).

- Pasos: aquí es donde definimos los pasos, o el código, que Quiero correr. En nuestro caso, solo tenemos un paso, que es ejecutar el comando `echo "Hola Mundo"`. Este comando Imprimirá el texto "Hola mundo" en la consola. Pasos Tiene varias opciones, incluidas
 - usos—Este es el nombre de la acción que queremos utilizar (si no está integrado en GitHub Actions). En nuestro caso, estamos utilizando la acción incorporada `actions/checkout@v3`, que es Se utiliza para consultar nuestro código en GitHub. Este es un paso necesario para casi todos los flujos de trabajo.
 - nombre— Podemos nombrar opcionalmente cada paso; en este caso, Acabamos de usar "Hola mundo".
 - ejecutar—Este es el comando básico que nos permite ejecutar cualquier otro comando, como `echo "mi texto"` o incluso `python3 my_script.py`.

Incluso en este flujo de trabajo simple, podemos ver que GitHub Los flujos de trabajo de acciones pueden comenzar a volverse bastante complejos. rápidamente porque son muy flexibles. La flexibilidad es una gran cosa, pero también puede ser un poco abrumador al principio. El La mejor manera de aprender GitHub Actions es comenzar con algo simple. flujo de trabajo y luego desarrolle según sea necesario.

Revisemos este flujo de trabajo en [GitHub.com](https://github.com) visitando el sitio Pestaña Acciones de cualquier repositorio de GitHub, como se ve en la figura 4.1.

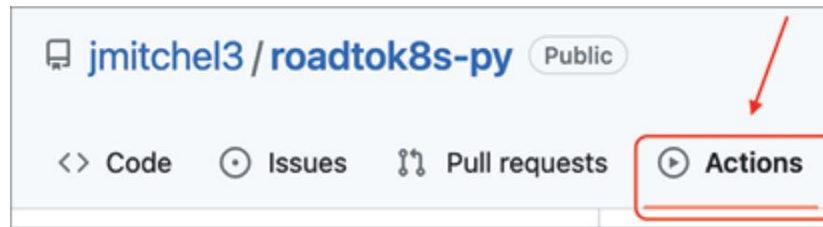


Figura 4.1 Pestaña Acciones de GitHub

Una vez en esta pestaña, debería ver nuestra Acción de GitHub en la barra lateral, como se muestra en la figura 4.2. Si no ve nuestro flujo de trabajo, significa que no envió su código al repositorio de GitHub correctamente, por lo que deberá revisar los pasos anteriores.

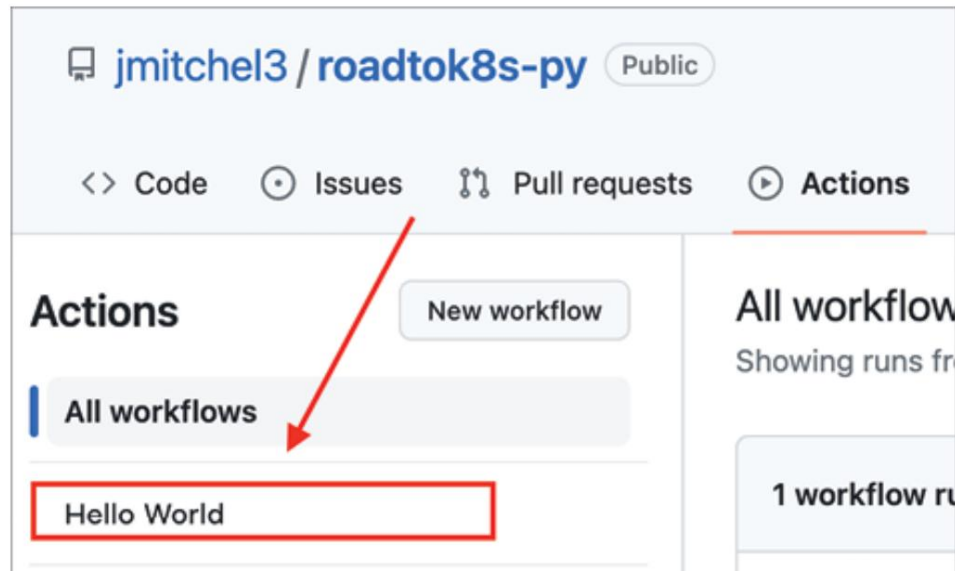


Figura 4.2 Barra lateral de Acciones de GitHub

El nombre que aparece aquí coincide directamente con la declaración de nombre: Hola Mundo en el archivo de flujo de trabajo, en la lista << #github-actions-hello-world >>. Para mantener el orden de estos nombres, suele ser recomendable colocar un número delante del nombre (p. ej., nombre: 01 Hola Mundo). Ahora haga clic en Hola Mundo y luego en Ejecutar flujo de trabajo, como se muestra en la figura 4.3.

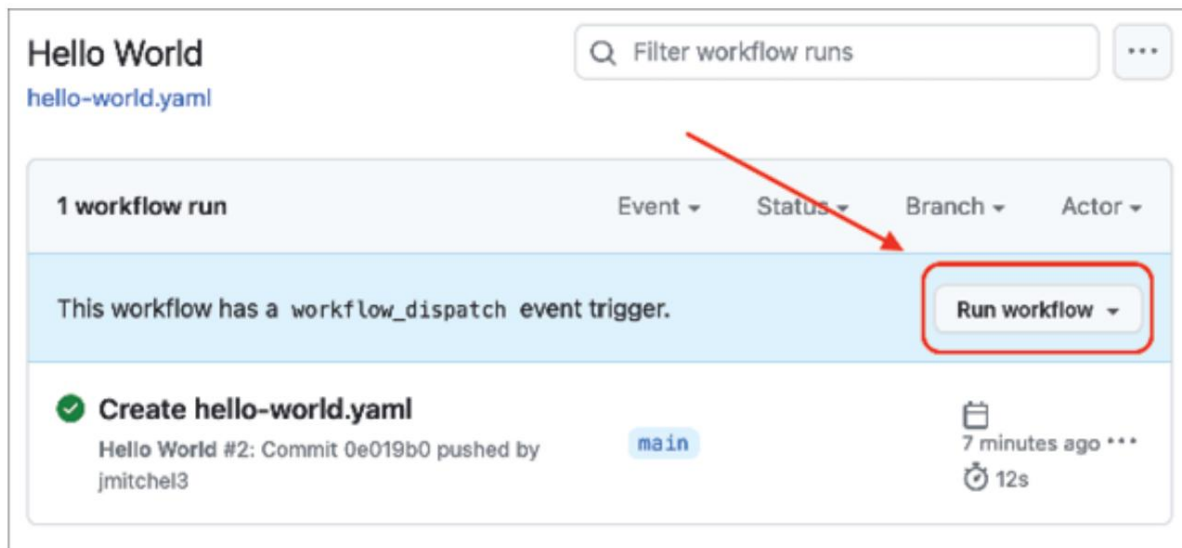


Figura 4.3 Flujo de trabajo de ejecución de acciones de GitHub

El motivo por el que podemos ver Ejecutar flujo de trabajo está directamente relacionado con la definición de `on:workflow_dispatch` en el archivo de flujo de trabajo.

Dado que incluimos `on:push`, es muy probable que este flujo de trabajo ya se haya ejecutado, como se indica en "Crear hello-world.yaml" en el historial, como se muestra en la figura 4.3. Haga clic en cualquiera de los elementos del historial para revisar el resultado del flujo de trabajo, como se muestra en la figura 4.4.

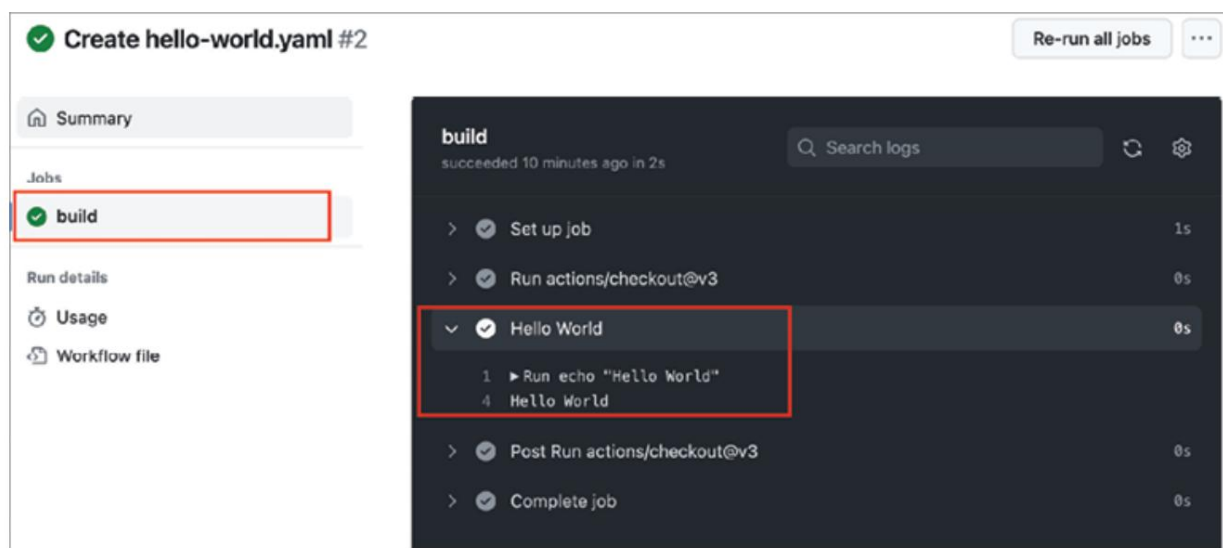


Figura 4.4 Resultado de la ejecución del flujo de trabajo de Acciones de GitHub

La figura 4.4 muestra algunos elementos importantes:

- El nombre del flujo de trabajo que se ejecutó
- Los trabajos con el nombre build (nuevamente vinculados directamente al archivo de flujo de trabajo)
- Los pasos que se ejecutaron en el trabajo (incluidos los que no definimos)
- El resultado de cada paso El
- resultado de nuestro paso específico llamado “Hola mundo”
- Se aprobaron todas las comprobaciones (por ejemplo, cada ícono de cada paso, así como el trabajo, tienen íconos de verificación en lugar de íconos de advertencia)

¡Felicitaciones! Acabas de ejecutar tu primer flujo de trabajo de GitHub Actions.

Este es un excelente primer paso en el mundo de las canalizaciones de CI/CD y GitHub Actions. El punto no es si tu flujo de trabajo fue exitoso o no. La cuestión es que acabas de usar una canalización de CI/CD sin servidor para ejecutar código en tu nombre. Este es un concepto muy importante que usaremos a lo largo de este libro.

Los flujos de trabajo de GitHub Actions son realmente potentes, pero se vuelven aún más potentes cuando les damos permiso para realizar una mayor variedad de tareas en nuestro nombre. En la siguiente sección, veremos cómo dar permiso a los flujos de trabajo de GitHub Actions para realizar tareas como instalar software en un servidor remoto almacenando contraseñas, claves API, claves SSH y otros secretos.

4.1.2 Creando tu primer secreto de GitHub Actions

Ahora que hemos completado nuestro primer flujo de trabajo, comencemos a trabajar en un ejemplo más práctico con el objetivo de simplemente instalar NGINX en un servidor remoto. Este ejemplo

requiere que usemos claves SSH privadas y públicas para trabajar con GitHub Actions y nuestras máquinas virtuales al mismo tiempo.

Comenzamos creando claves SSH específicas para usarlas específicamente en GitHub Actions y Akamai Linode. La clave privada se almacenará como un secreto de GitHub Actions y la clave pública se instalará automáticamente en las nuevas instancias aprovisionadas en Akamai Linode.

La creación de nuevas claves SSH garantizará que nuestras claves SSH personales nunca queden expuestas a GitHub Actions y, por lo tanto, nunca queden expuestas en ningún lugar además de nuestro entorno local. Esto también nos permitirá una automatización aún mejor cuando usemos Ansible con GitHub Actions más adelante en este capítulo. Si no tienes experiencia con claves SSH, considera revisar el apéndice C. La siguiente lista nos da el comando para crear una nueva clave SSH sin necesidad de introducir ninguna entrada; debería funcionar en diferentes plataformas.

Listado 4.3 Creación de una nueva clave SSH

```
# windows/mac/linux ssh-  
keygen -t rsa -b 4096 -f github_actions_key -N ""
```

Este comando genera los siguientes archivos en el directorio en el que ejecutó el comando

- `github_actions_key`: esta es la clave privada que instalaremos en un secreto de GitHub Actions.
- `github_actions_key.pub`: esta es la clave pública que instalaremos en Akamai Linode.

Una vez más, nunca debe exponer o compartir sus claves SSH personales y privadas (por ejemplo, `~/.ssh/id_rsa`) porque representa un importante problema de seguridad, que es exactamente el motivo por el que creamos un nuevo par de claves SSH para este propósito.

En este punto, agregaremos la clave privada (por ejemplo, `github_actions_key` sin `.pub`) a los repositorios de GitHub que creamos en el capítulo 2. Estos son los pasos:

1. Navega a tu repositorio en GitHub.
2. Haga clic en Configuración.
3. Navegue hasta Secretos y variables en la barra lateral.
4. Haga clic en Acciones, como se ve en la figura 4.5.

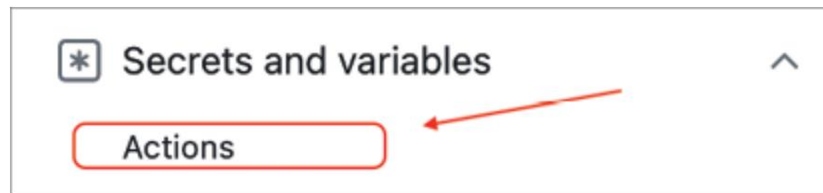


Figura 4.5 Enlace de secretos de GitHub Actions

En este punto, haz clic en "Nuevo Secreto de Repositorio" e introduce "SSH_PRIVATE_KEY". Luego, pega el contenido de tu clave privada recién creada (p. ej., `github_actions_key`) en el campo de valor. Haz clic en "Añadir Secreto" para guardar el nuevo secreto. La Figura 4.6 muestra el aspecto que debería tener.

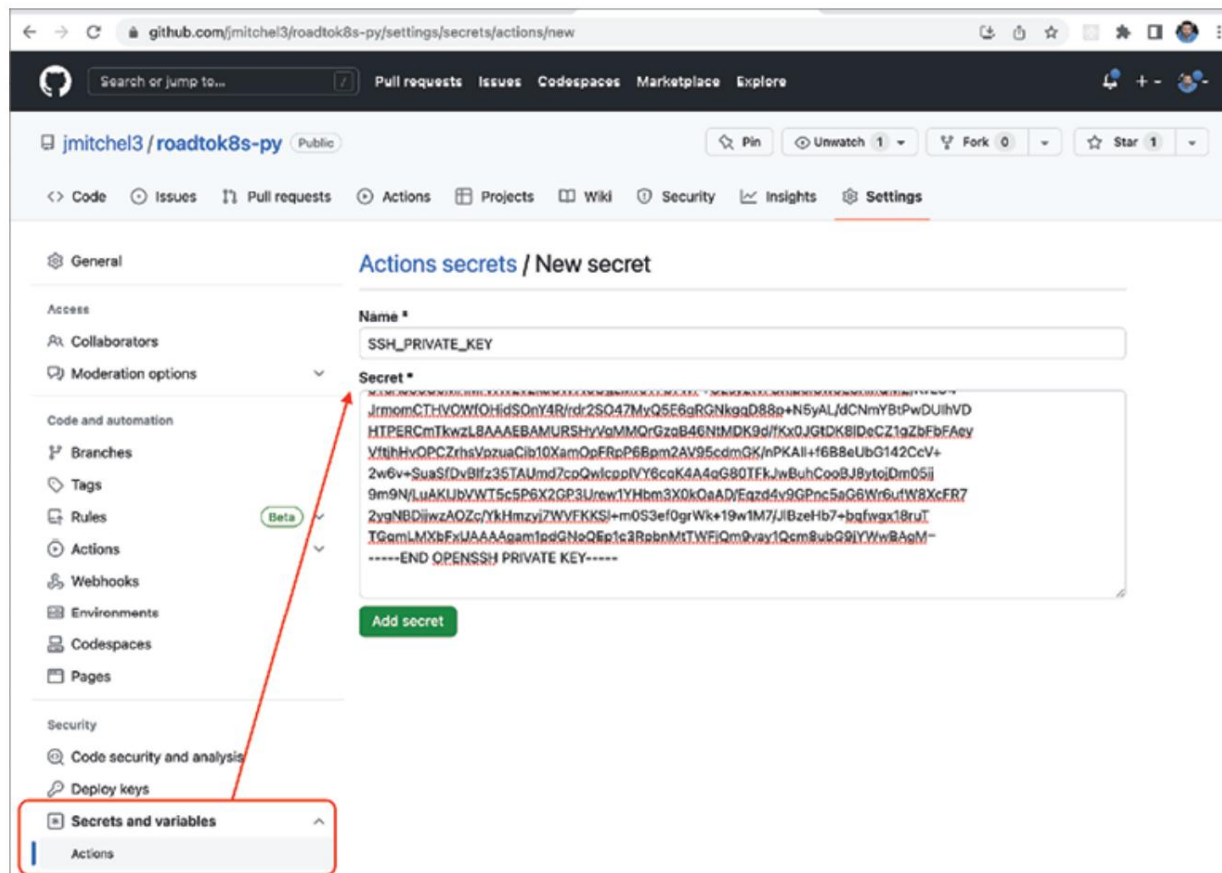


Figura 4.6 Nuevo secreto de GitHub Actions

Repita este mismo proceso para cualquier secreto nuevo que desee agregar. La Figura 4.6 solo muestra mi aplicación de Python (<https://github.com/jmitchel3/roadtok8s-py>), pero le recomiendo que repita esta misma clave también en el repositorio de su aplicación Node.js (<https://github.com/jmitchel3/roadtok8s-js>). Con la clave privada instalada en los secretos de GitHub Actions, es momento de llevar la clave pública a Akamai Connected Cloud Console.

4.1.3 Instalación de la clave SSH pública en Akamai Connected Cloud

La instalación de una clave SSH pública en nuestras nuevas instancias aprovisionadas por Akamai Connected Cloud nos permitirá conectarnos a la máquina virtual sin necesidad de una contraseña.

O bien, la entrada del usuario y habilitar GitHub Actions para realizar las tareas necesarias. Estos son los pasos para instalar una clave SSH pública en Akamai Connected Cloud:

1. Navegue hasta su consola Akamai Connected Cloud.
2. Haga clic en Cuenta en la esquina superior derecha.
3. Haga clic en Claves SSH en la barra lateral, como se ve en la figura 4.7.
4. Haga clic en Agregar una clave SSH.
5. Agregue una etiqueta para su clave (por ejemplo, Clave pública de GitHub Actions) y pegue el contenido de su clave pública recién creada (por ejemplo, github_actions_key.pub) en el campo Clave pública SSH, como se ve en la figura 4.8.

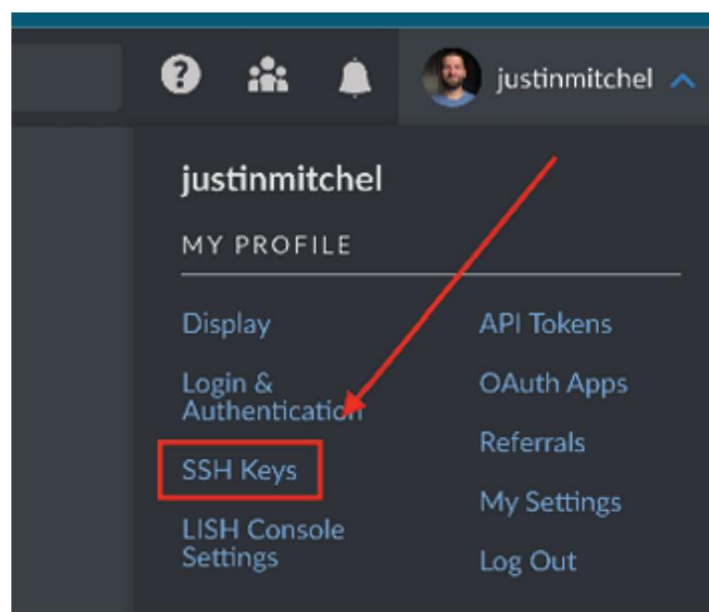


Figura 4.7 Barra de navegación de Akamai Linode: haga clic en Claves SSH.

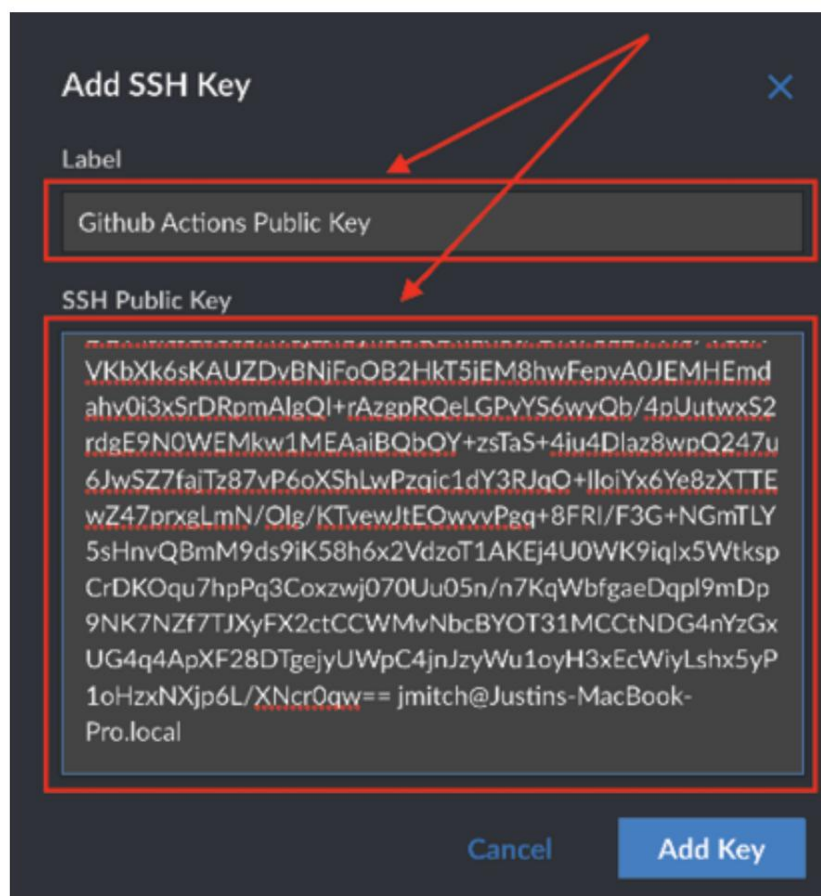


Figura 4.8 Akamai Linode: pegar nueva clave SSH.

Puede encontrar una guía más detallada sobre las claves SSH en el apéndice D. Con nuestras claves SSH instaladas, aprovisionemos una nueva máquina virtual en Akamai Connected Cloud.

4.1.4 Creación de una nueva máquina virtual en Akamai Linode

Ahora que tenemos nuestras claves SSH configuradas en Akamai Connected Cloud y los secretos de GitHub Actions, aprovisionaremos una nueva instancia de máquina virtual. Como recordarán, realizamos este mismo paso en la sección 3.1.1, así que seremos breves.

Vaya a Akamai Connected Cloud Linodes (<https://cloud.linode.com/linodes>) y haga clic en Crear Linode.

Utilice la siguiente configuración:

- Distribución: Ubuntu 22.04 LTS (o la última LTS).
- Región: Dallas, TX (o la región más cercana a usted).
- Plan: CPU compartida > Nanode 1 GB (o más grande).
- Etiqueta: rk8s-github-actions.
- Etiquetas—<opcional>.
- Contraseña de root: establezca una buena.
- Claves SSH: seleccione la clave SSH recién creada (por ejemplo, clave pública de GitHub Actions) y cualquier otra clave de usuario que pueda necesitar, como se ve en la figura 4.9.
- Todos los demás ajustes—<opcional>.

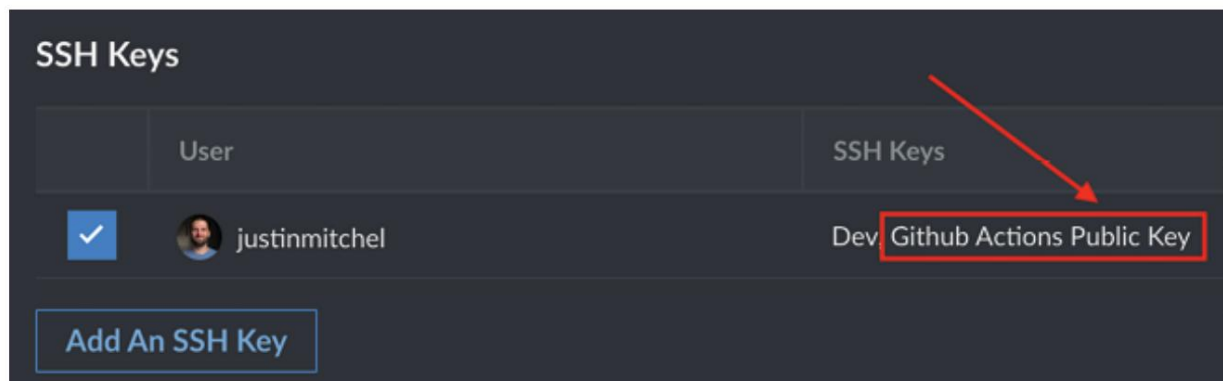


Figura 4.9 Akamai Linode: crear acciones de GitHub de VM con claves SSH.

Después de configurar estos ajustes, haga clic en "Crear Linode" y espere a que se asigne la dirección IP, como se muestra en la figura 4.10. Una vez asignada la dirección IP, pasaremos a la siguiente sección, donde usaremos GitHub Actions para instalar NGINX en esta máquina virtual.

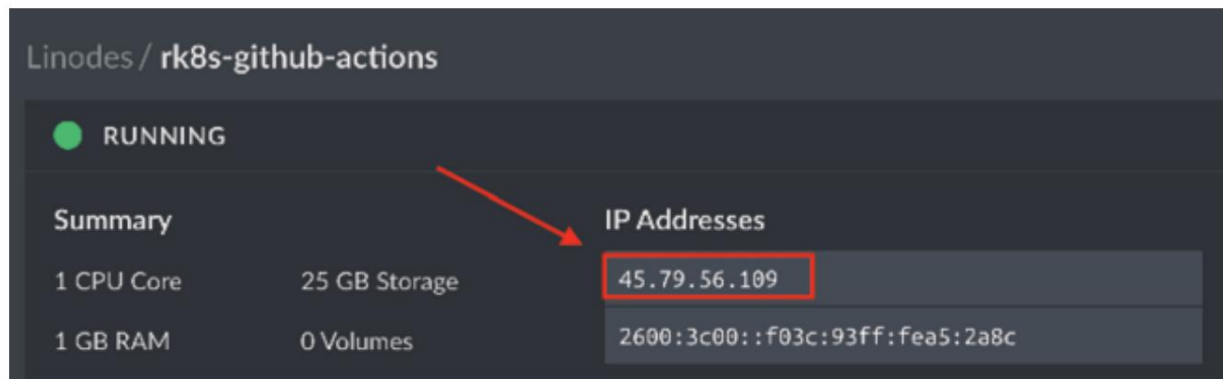


Figura 4.10 Akamai Linode: Dirección IP asignada a GitHub Actions

La Figura 4.10 muestra la dirección IP de la máquina virtual recién provisionada como 45.79.56.109. Esta dirección IP será diferente para usted, así que asegúrese de usar la dirección IP asignada a su máquina virtual.

En los secretos de acciones de tu repositorio de GitHub, debes agregar esta dirección IP, tal como hicimos con la clave privada SSH en la sección 4.1.3. Usa el nombre del secreto

AKAMAI_INSTANCE_IP_ADDRESS y el valor del secreto con tu dirección IP, como se muestra en la figura 4.11.

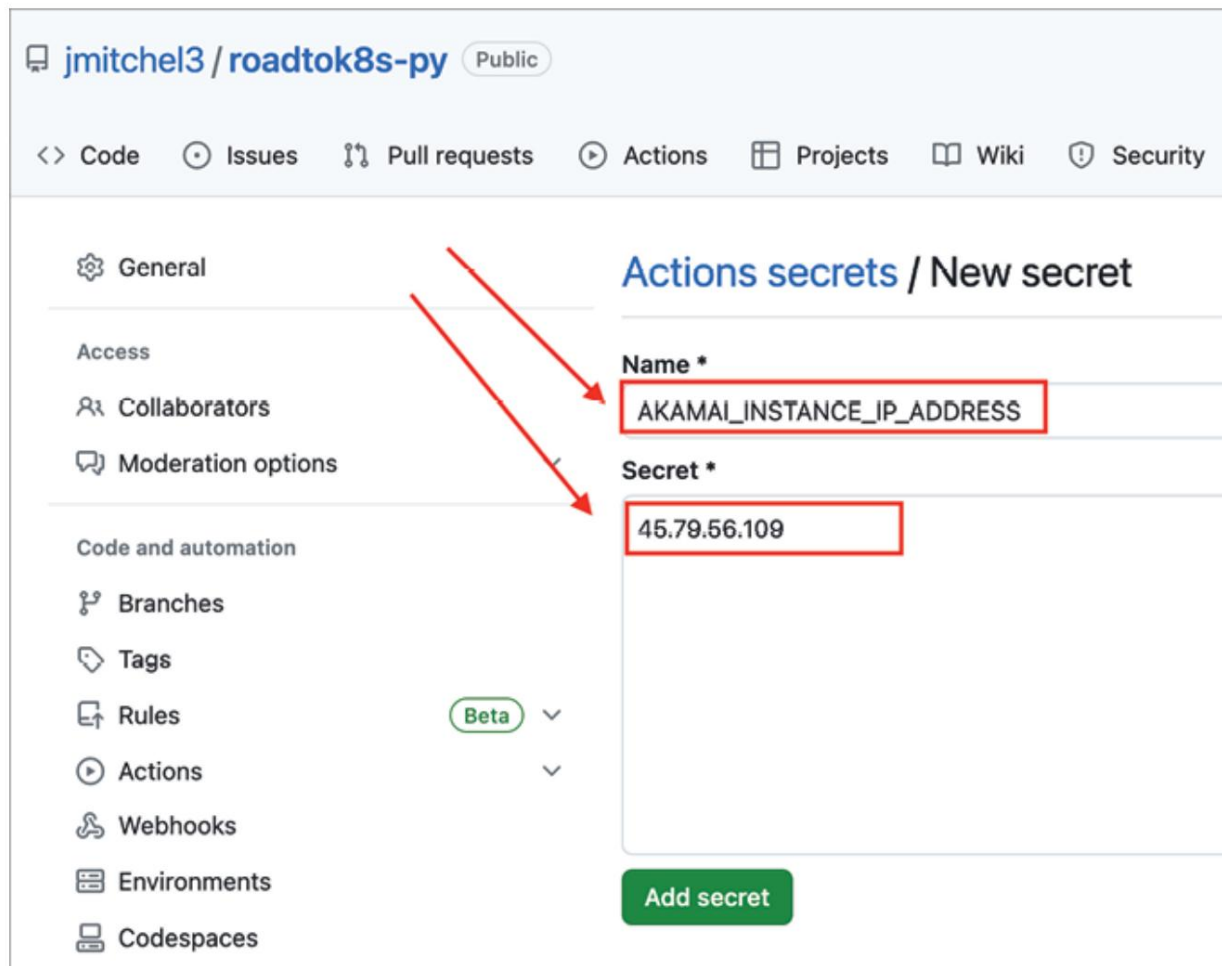


Figura 4.11 Acciones de GitHub: agregar secreto de dirección IP.

Con todas las condiciones establecidas correctamente, crearemos un flujo de trabajo de GitHub Actions que instalará NGINX en nuestra máquina virtual recién creada. El flujo de trabajo y el proceso son sumamente sencillos, pero es un paso importante para usar pipelines de automatización.

4.1.5 Instalación de NGINX en un servidor remoto con un flujo de trabajo de GitHub Actions

La forma estándar de instalar NGINX en sistemas Ubuntu es muy sencilla: primero usamos `sudo apt update` y luego `sudo apt install nginx` (o `apt-get install nginx`). Estos comandos te resultarán muy familiares, ya que...

Los usé en la sección 3.2. Esta simplicidad es una excelente manera de evaluar nuestro progreso en la adopción de nuevos métodos de implementación, por ejemplo, usando GitHub Actions.

Sabiendo que estos son los dos comandos, podemos usarlos con el comando ssh integrado en el formato `ssh <usuario>@<dirección-ip> <comando>`. Por ejemplo, si quisiéramos ejecutar `sudo apt update` y `sudo apt install nginx -y` en nuestra máquina virtual recién aprovisionada, ejecutaríamos los siguientes comandos:

```
ssh root@mi-dirección-IP sudo apt get actualización:
ssh root@mi-dirección-IP sudo apt get install nginx -y
```

Recuerde, el `-y` aprobará automáticamente cualquier solicitud durante esta instalación, lo que la hace completamente automatizada, asumiendo que el paquete nginx existe.

Con estos dos comandos en mente, nuestro nuevo flujo de trabajo utilizará dos de nuestros secretos de GitHub Actions:

- `SSH_PRIVATE_KEY`: esta es la clave privada que creamos en la sección 4.1.2.
- `AKAMAI_INSTANCE_IP_ADDRESS`: esta es la dirección IP de la máquina virtual que creamos en la sección 4.1.3.

Nunca debemos exponer secretos en GitHub Actions ni en los repositorios de Git. Los secretos deben serlo y tratarse como tal. Almacenarlos en GitHub Actions es una excelente manera de mantenerlos seguros, fuera de tu código base y del alcance de terceros.

El uso de secretos almacenados dentro de un flujo de trabajo de GitHub Actions requiere una sintaxis especial que se parece a esto: `$ {{ secrets.SECRET_NAME }}` así que usaremos lo siguiente:

- `${{ secrets.SSH_PRIVATE_KEY }}`—para nuestra clave privada SSH

- `{{ secrets.AKAMAI_INSTANCE_IP_ADDRESS }}`—para nuestra dirección IP de instancia de Akamai

Las sesiones SSH sin contraseña requieren al menos las siguientes condiciones:

- La clave privada : disponible localmente (por ejemplo, `~/.ssh/id_rsa`) y ejecutable con `chmod ~/.ssh/id_rsa` La clave pública: instalada remotamente
- (por ejemplo, `~/.ssh/authorized_keys`)
- Una dirección IP (o nombre de host): accesible a través de Internet (o simplemente para la máquina que intenta conectarse)

Hemos establecido las condiciones para sesiones SSH sin contraseña, pero aún necesitamos instalar la clave privada durante la ejecución de un flujo de trabajo de GitHub Actions. Esto significa que inyectaremos esta clave durante el flujo de trabajo para que los pasos futuros puedan usar esta configuración. Afortunadamente, este es un proceso sencillo, como se puede ver en el listado 4.4. Cree un nuevo archivo de flujo de trabajo en la ruta `.github/workflows/install-nginx.yaml` con el contenido del siguiente listado.

Listado 4.4 Instalación de NGINX con GitHub Actions

```

nombre: Instalar NGINX
en:
  flujo de trabajo_despacho:
    push:

trabajos:
  construir:
    runs-on: ubuntu-latest pasos: - usos:
      actions/
      checkout@v3 - nombre: Configurar el secreto
      de la clave privada SSH
      ejecutar:
        | mkdir -p ~/.ssh/ echo "$
        {{ secrets.SSH_PRIVATE_KEY }}" > ~/.ssh/id_rsa chmod 600 ~/.ssh/id_rsa - nombre: Establecer
        comprobación estricta de la clave de
      host ejecutar: echo "StrictHostKeyChecking=no" > ~/.ssh/config
      - nombre: Instalar NGINX ejecutar: | export MY_HOST="$
      {{ secrets.AKAMAI_INSTANCE_IP_ADDRES
      S }}"
      ssh root@$MI_HOST sudo apt update ssh
      root@$MI_HOST sudo apt install nginx -y

```

Como podemos ver, este flujo de trabajo introdujo dos pasos nuevos que no teníamos en nuestro flujo de trabajo "Hola Mundo", mientras que todo lo demás permaneció prácticamente intacto. Repasemos los nuevos pasos:

1. Configurar la clave privada SSH secreta: Nuevamente, vemos que usamos `run:` para ejecutar un comando. En este caso, usamos un comando multilínea utilizando la barra vertical (`|`) para habilitar varios comandos en este único paso. Podríamos dividir estos comandos en pasos separados para una mayor claridad en cada paso (por ejemplo, si uno...

Si falla alguno de los comandos, a menudo puede ser de ayuda tener varios pasos).

2. Configurar una verificación estricta de la clave de host : Normalmente, al conectarse por SSH en su equipo local, se le solicita que verifique la autenticidad de la clave de host. Este paso omite el proceso de verificación, ya que los flujos de trabajo de GitHub Actions no permiten la entrada del usuario. También puede considerar usar

```
echo "${secretos.DIRECCIÓN_IP_DE_INSTANCIA_AKAMAI}" $(ssh-keyscan -H ${secretos.A  
DIRECCIÓN_IP_DE_INSTANCIA_AKAMAI})" > ~/.ssh/hosts_conocidos
```

para agregar la clave de host al archivo de host conocido en lugar de

```
echo "StrictHostKeyChecking=no" > ~/.ssh/config
```

3. Instalar NGINX: Este paso es muy similar al anterior, excepto que no es necesario volver a configurar la clave privada SSH, ya que todo el trabajo de compilación tiene acceso a los resultados de los pasos anteriores. Simplemente ejecutamos los dos comandos para instalar NGINX en nuestro servidor remoto.

servidor.

Ahora puedes abrir tu navegador web en tu dirección IP y ver la página Hola Mundo de NGINX, como hemos visto antes.

Ver esta página puede no ser muy emocionante, ya que ya la hemos visto antes y no hay mucha actividad aquí, pero el hecho de que hayamos hecho esto con GitHub Actions es fundamental. Ahora podemos usar GitHub Actions para instalar software en servidores remotos, y podemos hacerlo de forma repetible y automatizada. Este es un gran paso hacia una canalización de CI/CD totalmente automatizada.

En este punto, podríamos sumergirnos en el maravilloso mundo de ssh y scp (un comando para copiar archivos) para instalar y configurar nuestro host remoto, pero en su lugar, vamos a implementar un

Una solución más robusta, Ansible, nos permitirá instalar y configurar nuestro host remoto. Desde la perspectiva de un desarrollador, Ansible y GitHub Actions se parecen mucho porque ambos usan archivos con formato YAML, pero Ansible está diseñado para automatizar la configuración de hosts remotos.

4.2 Automatización de máquinas virtuales con Ansible

Ansible es una forma divertida de instalar y configurar máquinas, y en esta sección te mostraré exactamente por qué.

Cuando consideramos el comando `sudo apt install nginx -y`, nos vienen a la mente algunas preguntas:

- ¿Está instalado NGINX?
- ¿Está funcionando NGINX?
- ¿Necesitamos agregar o actualizar nuestra configuración NGINX?
- ¿Necesitaremos reiniciar o volver a cargar NGINX después de configurarlo, si es necesario configurarlo?
- ¿Necesitamos realizar este mismo proceso en otra máquina? ¿Qué tal en otras 10? ¿En otras 1000?

Dado que este comando, `sudo apt install nginx -y`, plantea varias preguntas muy interesantes, debemos considerar su significado: el estado deseado. El estado deseado consiste en garantizar que cualquier máquina esté configurada exactamente como se necesita para que nuestras aplicaciones se ejecuten correctamente. Elegí NGINX porque es una aplicación increíblemente versátil que requiere muy poca sobrecarga para funcionar correctamente. NGINX también es una excelente alternativa cuando necesitamos instalar y configurar aplicaciones más complejas, como las de Python y Node.js.

NGINX también nos brinda funciones que podemos usar en ambos

aplicaciones, como equilibrio de carga y enrutamiento basado en rutas.

Ansible, al igual que otros programas de infraestructura como código (IaC), es una herramienta que nos ayuda a alcanzar el estado deseado en nuestras máquinas, independientemente de su estado actual. De esta manera, podemos alcanzar un resultado específico mediante archivos YAML llamados Playbooks de Ansible.

Ansible es un ejemplo de programación declarativa . Usaremos Ansible para lo siguiente:

- Instale dependencias específicas del proyecto y del nivel del sistema operativo para Python, Node.js, NGINX y Supervisor.
- Configurar Supervisor y NGINX.
- Inicie, reinicie y vuelva a cargar Supervisor y NGINX.

Ansible funciona a través de una conexión de shell segura (SSH), por lo que sigue siendo fundamental que tengamos las claves SSH adecuadas en la máquina para ejecutar Ansible y en la máquina o máquinas que queremos configurar.

Ansible es una herramienta potente con muchas opciones, por lo que recomiendo tener a mano la Documentación Oficial (<https://docs.ansible.com/ansible/latest/>) para revisar todas las opciones de configuración. Creemos nuestro primer Playbook de Ansible junto con un flujo de trabajo de GitHub Actions para ejecutarlo.

4.2.1 Flujo de trabajo de GitHub Actions para Ansible

Ansible es una herramienta basada en Python que podemos instalar en prácticamente cualquier máquina. En nuestro caso, seguiremos usando los flujos de trabajo de GitHub Actions como nuestra máquina de configuración y canalización de CI/CD. Aprovechar todas las capacidades de Ansible queda fuera del alcance de este libro, pero usaremos lo mínimo para lograr los resultados deseados.

Antes de crear nuestro primer Playbook de Ansible, crearemos un flujo de trabajo de GitHub Actions que establecerá todos los elementos necesarios para ejecutarlo . Este flujo de trabajo hará lo siguiente:

1. Configurar Python 3.
2. Instalar Ansible.
3. Implemente la clave SSH privada.
4. Cree un archivo de inventario de Ansible para nuestro host remoto.
5. Cree un archivo de configuración predeterminado de Ansible.
6. Ejecute cualquier Ansible Playbook.

Antes de analizar el flujo de trabajo de acciones de GitHub, revisemos los nuevos términos:

- Archivo de inventario : Ansible permite configurar una gran cantidad de hosts remotos mediante una dirección IP o incluso un nombre de dominio. Este archivo nos permite definir y agrupar los hosts remotos según nuestras necesidades. En nuestro caso, usaremos un host remoto dentro de un grupo.
- Archivo de configuración predeterminado : Para empezar, solo necesitamos configurar Ansible para usar el archivo de inventario personalizado que creamos, junto con algunas opciones relacionadas con SSH. Usaremos el archivo de configuración predeterminado para ello.
- Playbook de Ansible : Los playbooks son la función principal de Ansible que usaremos. Son archivos con formato YAML que definen el estado deseado de un host remoto o un grupo de hosts remotos. En proyectos más grandes, los playbooks pueden ser bastante complejos.

En el listado 4.5, vemos el archivo de flujo de trabajo que configurará Ansible y nuestro host remoto. Cree un nuevo archivo de flujo de trabajo en la ruta `.github/workflows/run-ansible.yaml` con el contenido del siguiente listado.

Listado 4.5 Ejecución de Ansible con GitHub Actions

nombre: Ejecutar Ansible

en:

flujo de trabajo_despacho:

trabajos: ejecutar playbooks:

runs-on: ubuntu-latest pasos: - usos:

actions/

checkout@v3 - nombre: Configurar Python 3

usos: actions/setup-python@v4 con:

python-version: "3.8" - nombre: Actualizar Pip e
instalar

Ansible

ejecutar:

| python -m pip install --upgrade pip python -m pip install

ansible - nombre: Implementar la clave SSH

privada

ejecutar:

| mkdir -p ~/.ssh/ echo "\$

{{ secretos.CLAVE_PRIVADA_SSH }}" > ~/.ssh/id_rsa chmod 600 ~/.ssh/id_rsa

- nombre: Archivo de inventario de Ansible para el host remoto

ejecutar:

| mkdir -p ./devops/ansible/ export

INVENTORY_FILE=./devops/ansible/inventory.ini echo "[mi_grupo_de_hosts]" >

\$INVENTORY_FILE echo "\${{ secrets.AKAMAI_INSTANCE_IP_ADDRESS }}"

>> \$INVENTORY_FILE - nombre: Archivo de configuración predeterminado de Ansible

ejecutar:

| mkdir -p ./devops/ansible/ cat <<EOF > ./

devops/ansible/ansible.cfg [valores predeterminados]

ansible_python_interpreter = '/usr/bin/python3' ansible_ssh_private_key_file =

~/.ssh/id_rsa

```
usuario_remoto = root
inventario = ./inventory.ini
comprobación_de_clave_de_host = False
EOF
```

```
- nombre: Ping a hosts de Ansible directorio-
de-trabajo: ./devops/ansible/ ejecutar: | ansible all -m ping -
nombre:
```

```
Ejecutar Playbooks de Ansible
```

```
directorio-de-trabajo: ./devops/ansible/ ejecutar: |
ansible-playbook install-nginx.yaml
```

```
- nombre: Implementar Python a través de Ansible
directorio de trabajo: ./devops/ansible/ ejecutar: | ansible-
playbook
deploy-python.yaml
```

Los dos archivos clave creados en este flujo de trabajo son `inventory.ini` y `ansible.cfg`. Este flujo de trabajo es un ejemplo completo de lo que se debe hacer para ejecutar un flujo de trabajo de Ansible. El comando `ansible all -m ping` simplemente garantiza que nuestra configuración previa se realizó correctamente y en la ubicación correcta. Vemos la declaración "working-directory" para cada uno de los comandos de Ansible (`ansible` y `ansible-playbook`) simplemente porque `ansible.cfg` se encuentra en el archivo `inventory.ini` y hace referencia a él en dicho directorio. Después de confirmar este flujo de trabajo en Git, subirlo a GitHub y ejecutarlo, verá el resultado en la figura 4.12.



```

1  ▶ Run ansible all -m ping
11 *** | SUCCESS => {
12   "ansible_facts": {
13     "discovered_interpreter_python": "/usr/bin/python3"
14   },
15   "changed": false,
16   "ping": "pong"
17 }

1  ▶ Run ansible-playbook install-nginx.yaml
11 ERROR! the playbook: install-nginx.yaml could not be found
12 Error: Process completed with exit code 1.

```

Figura 4.12 Resultado de la configuración de Ansible de GitHub Actions

Ansible puede verificar que se está conectando a nuestro host remoto y que no puede ejecutar el flujo de trabajo `install-nginx.yaml` (porque aún no se ha creado). Si ve el mismo resultado, está listo para crear su primer Playbook de Ansible. Si ve algún error relacionado con la conexión a localhost, es muy probable que su archivo `inventory.ini` se haya creado incorrectamente o que la dirección IP ya no esté configurada correctamente en sus secretos de GitHub Actions. Si ve algún error relacionado con permisos denegados o errores de clave SSH, es muy probable que su clave privada SSH no esté configurada correctamente.

Suponiendo que obtuviste el resultado que se muestra en la figura 4.12, ya estamos listos para crear nuestro primer Playbook de Ansible. Prepárate, porque aquí es donde ocurre la magia.

4.2.2 Creación de su primer Playbook de Ansible

En mi experiencia, los Playbooks básicos de Ansible son mucho más fáciles de entender que los flujos de trabajo de GitHub Actions, pero los Playbooks avanzados son mucho más difíciles. Por suerte, nos centraremos en los Playbooks básicos de Ansible en lo que resta de este libro. En cada Playbook, declararemos lo siguiente:

- Nombre: el nombre del playbook. Hosts: los
 - hosts y el grupo de hosts donde se ejecutará este playbook. Puede usar "all" (todos los hosts) o un grupo específico como [my_host_group], que definimos en inventory.ini en el listado 4.5. Para nuestro primer playbook, usar "all" es lo que podemos usar. Become: una palabra clave especial que nos permite ejecutar comandos como usuario root . No siempre se
 - recomienda usar un usuario root en entornos de producción, pero las complejidades de los permisos de usuario quedan fuera del alcance de este libro.
-
- Tareas: una lista de configuraciones que queremos que Ansible ejecute. Cada tarea se ejecuta después de la tarea anterior.

El listado 4.6 muestra el contenido de install-nginx.yaml, que crearemos en la ruta devops/ansible/install-nginx.yaml.

Listado 4.6 Instalación de NGINX con Ansible

```
- nombre: Instalar NGINX
  hosts: todos se
  vuelven: verdaderos
  Tareas:
    - Nombre: Instalar NGINX
      apt:
        nombre: nginx
        estado: presente
        update_cache: sí
```

¿Te sorprende lo sencillo que es? Probablemente no. Al fin y al cabo, son muchas más líneas que simplemente `sudo apt update && sudo apt install nginx -y`. La principal diferencia es que podemos ejecutar este mismo flujo de trabajo en miles de hosts si es necesario, con la misma facilidad que si lo hiciéramos en un solo host (suponiendo que se instalen las claves SSH correctas, por supuesto). Podemos hacerlo aún más robusto asegurándonos de que el servicio NGINX esté realmente...

ejecutándose en este host agregando una nueva tarea a este libro de jugadas, como se muestra en la siguiente lista.

Listado 4.7 Cómo garantizar que NGINX se esté ejecutando con Ansible

```
- nombre: Instalar NGINX
  hosts: todos se
  vuelven: verdaderos
  Tareas:
  - Nombre: Instalar NGINX
    apt:
      nombre: nginx
      estado: presente
      update_cache: sí - nombre:
  Asegurarse de que NGINX se esté ejecutando
  servicio:
    nombre: nginx
    estado: iniciado
    habilitado: sí
```

Una vez creado este archivo, confirmémoslo en Git, lo subamos y luego ejecutamos nuestro flujo de trabajo de Ansible de GitHub Actions. Debería ver el resultado que se muestra en la figura 4.13.



```

16 ▶ Run ansible-playbook install-nginx.yaml
11
12 PLAY [Install NGINX]
13
14 TASK [Gathering Facts]
15 ok: [***]
16
17 TASK [Install NGINX]
18 ok: [***]
19
20 TASK [Ensure NGINX is running]
21 ok: [***]
22
23 PLAY RECAP
24 *** : ok=3  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
25
  
```

Figura 4.13 Acciones de GitHub Ansible instala NGINX

Con Ansible, podemos ver el estado de cada tarea en ejecución en el orden en que se declaró. Si tuviera más de un host en su archivo de inventario, vería la misma salida para cada host. Esta es una excelente manera de ver el estado de cada tarea y cada host. De nuevo, vemos el estado deseado (por ejemplo, la tarea declarada), junto con el estado actual y el estado actualizado si ha cambiado. Esta es una excelente manera de ver qué está haciendo Ansible y por qué. Por supuesto, agregar más hosts remotos es tan fácil como seguir estos pasos:

1. Agregue cada host como un secreto en GitHub Actions Secrets

(por `AKAMAI_INSTANCE_IP_ADDRESS_1`,
ejemplo, `AKAMAI_INSTANCE_IP_ADDRESS_2`, etc.).

2. Agregue cada host al archivo `inventory.ini` en el flujo de trabajo (por ejemplo, echo

```
"${{ secrets.AKAMAI_INSTANCE_IP_ADDRESS_1 }}" "$  
>>          inventario.ini          y          eco  
{{ secrets.AKAMAI_INSTANCE_IP_ADDRESS_2 }}" >> inventory.ini).
```

3. Ejecute el flujo de trabajo de Acciones de GitHub.

Con este nuevo Ansible Playbook, ahora podemos instalar NGINX en cualquier cantidad de hosts remotos con un solo comando. Este es un gran paso hacia una canalización de CI/CD totalmente automatizada. Ahora necesitamos dar el siguiente paso hacia la automatización de la implementación de nuestras aplicaciones Python y Node.js.

4.2.3 Ansible para nuestra aplicación Python

En el capítulo 3, implementamos una aplicación Python y usamos varios ganchos de Git para realizar la configuración. Si bien este método funciona y es válido, es propenso a errores y requiere que alojemos nuestro repositorio de Git, lo cual no es ideal (por diversas razones).

Antes de comenzar a crear el libro de estrategias, recapitulemos lo que debemos hacer para que nuestra aplicación Python se ejecute en casi cualquier host remoto:

- Instale Python 3 y las dependencias relacionadas a nivel de sistema (como python3-dev, python3-venv, build-essential, etc.).
- Cree un entorno virtual Python 3.8+.
- Copiar o actualizar el código fuente.
- Instalar o actualizar los requisitos (a través del requirements.txt de Python).
- Instalar y configurar Supervisor para la aplicación Python.
- Instalar y configurar NGINX para la aplicación Python.

El código fuente y el archivo requirements.txt son los elementos que probablemente cambiarán más, mientras que las configuraciones de Supervisor y NGINX probablemente cambiarán muy poco. Comenzaremos añadiendo una configuración específica para cada herramienta. Para ello, añadiremos los siguientes archivos a nuestro repositorio local:

- conf/nginx.conf
- conf/supervisor.conf

La configuración de Supervisor en conf/supervisor.conf proviene directamente del listado 3.27. La idea es usar un comando específico para ejecutar nuestra aplicación Python (por ejemplo, gunicorn), junto con un directorio de trabajo y una ubicación para la salida del registro, todo lo cual se resume en el siguiente listado.

Listado 4.8 Configuración del supervisor para Python y Ansible

```
[programa:roadtok8s-py]
directorio=/opt/proyectos/roadtok8s/py/src comando=/
opt/venv/bin/gunicorn --worker-class
    uvicorn.workers.UvicornWorker principal: aplicación --bind "0.0.0.0:8888"
    --pid /var/run/roadtok8s-py.pid
inicio automático=verdadero
inicio automático=verdadero
inicioreinicios=3
archivo_registro_stderr=/var/log/supervisor/roadtok8s/py/stderr.log archivo_registro_stdout=/var/log/
supervisor/roadtok8s/py/stdout.log
```

NOTA: Si está familiarizado con la escritura de comandos bash multilínea, notará que `command=` no tiene el patrón de escape de nueva línea (`\`) en cada línea. Esto se hace a propósito debido a la forma en que Supervisor analiza este archivo.

La siguiente pieza es implementar la configuración de NGINX en `conf/nginx.conf` con algunas modificaciones leves a lo que se ve en el listado 4.9.

Listado 4.9 Configuración de NGINX para Python y Ansible

```
servidor
{
    escuchar 80;
    nombre_del_servidor _;
    ubicación /
    {
        contraseña_del_proxy http://127.0.0.1:8888;
        encabezado_del_conjunto_del_proxy Host $host;
        encabezado_del_conjunto_del_proxy X-Real-IP $dirección_remota;
        encabezado_del_conjunto_del_proxy X-Reenviado-Para $dirección_del_proxy_x_reenviado_f
    }
    proxy_set_header Proto reenviado X $esquema;
}
}
```

Vamos a crear nuestro archivo de Ansible pieza por pieza para comprender qué sucede. Primero, crearemos un nuevo Playbook de Ansible en la ruta `devops/ansible/deploy-python.yaml` con el contenido del listado 4.10.

Listado 4.10 Instalación de Python con Ansible Playbook, parte 1

```
- nombre: Implementar aplicación Python
  hosts: todos se
  convierten en: sí
```

Ahora pasemos a la primera tarea: actualizar la caché de paquetes del sistema. Esto se hace primero para que las tareas futuras puedan usarla si es necesario. El Listado 4.11 muestra la primera tarea. Defina la caché de paquetes y explique brevemente en qué condiciones las tareas futuras necesitarán usarla.

Listado 4.11 Instalación de Python con Ansible Playbook, parte 2

```
tareas:
  - nombre: Sistema de actualización y mejora
    apt:
      upgrade: sí
      update_cache: sí
      cache_valid_time: 3600
```

Esto actualizará la caché de paquetes del sistema durante aproximadamente una hora. En la siguiente lista, instalaremos los paquetes apt específicos que nuestro sistema necesita para ejecutar Python, Supervisor y NGINX.

Listado 4.12 Instalación de Python con Ansible Playbook, parte 3

```
- nombre: Instalar dependencias
  apt:
    pkg: -
      python3-pip - python3-
      dev - python3-venv -
      rsync - nginx -
      supervisor
    estado:
      presente
```

La declaración apt está integrada en Ansible y nos permite tener la línea "state: present" como opción en esta tarea para cualquier paquete que enumeremos. Disponemos de otras opciones para el estado (p. ej., "último", "ausente"). Lo potente de esta línea "state: present" reside en su repetibilidad, suponiendo que "apt install" es el paquete de instalación a nivel de sistema operativo. Esto es muy similar a lo que requirements.txt nos permite en nuestro proyecto de Python, pero para todo el sistema.

Dado que python3-venv es un componente esencial de las instalaciones realizadas en el listado 4.12, ahora podemos crear nuestro entorno virtual de Python 3.8+. El listado 4.13 muestra la tarea para crear el entorno virtual.

Listado 4.13 Instalación de Python con Ansible Playbook, parte 4

```
- nombre: Crear entorno virtual ansible.builtin.command:
  cmd: python3 -m venv /opt/venv crea: /
      opt/venv
```

El módulo de comandos integrado de Ansible (consulte el módulo de comandos en la documentación de Ansible para obtener más detalles) ejecuta un comando sin variables de shell (como las variables de bash \$HOME, \$PWD y \$USER). Si está familiarizado con estas variables de bash

Si necesita variables, puede usar `ansible.builtin.shell`. En esta tarea, se ejecutará el comando `python3 -m venv /opt/venv` solo si el directorio `/opt/venv` no existe debido a la declaración `"creates: "` con la carpeta que crea el entorno virtual. Esta es una excelente manera de garantizar que esta tarea se ejecute solo una vez para evitar que se ejecute varias veces, lo que puede causar problemas con otros comandos.

Ahora queremos crear los directorios para el código fuente (p. ej., el proyecto Python) y las carpetas de salida del registro, según lo especificado en Supervisor. Si necesita ejecutar una tarea varias veces para varios directorios, puede usar la declaración `with_items`, como se muestra en la siguiente lista.

Listado 4.14 Instalación de Python con Ansible Playbook, parte 5

```
- nombre: Configurar el directorio de la aplicación
  archivo:
    ruta: "{{ item }}" estado:
    directorio propietario:
    "{{ ansible_user }}" grupo: "{{ ansible_user }}"
    modo: '0755' con_elementos: - /opt/projects/
    roadtok8s/py/src/ - /
    var/log/supervisor/
    roadtok8s/py/
```

Como vemos aquí, `with_items` toma una lista de elementos e itera sobre el resto de la tarea. En este caso, la tarea consiste en crear un archivo o directorio. Esta tarea nos introduce a una sintaxis similar a la de Jinja2 con la variable `{{item}}`, que corresponde directamente a la lista de elementos en `with_items`, y la variable `{{ansible_user}}`, que representa al usuario raíz, tal como lo proporciona Ansible directamente gracias a la declaración `"becoming: yes"` al principio del playbook. El resto de la declaración del archivo sirve simplemente para garantizar que cada directorio se cree con la configuración correcta.

permisos y propiedad para que nuestro usuario (y otras aplicaciones) puedan utilizar correctamente estos directorios.

Ahora tenemos que tomar una decisión: ¿Cómo enviamos nuestro código a nuestras máquinas remotas? Por un lado, podríamos usar Git en las máquinas remotas para clonar o extraer el código, o podríamos sincronizarlo desde la máquina local (por ejemplo, el flujo de trabajo de GitHub Actions). Optaremos por sincronizar el código por las siguientes razones:

- Nuestro flujo de trabajo de GitHub Actions ya tiene una copia de pago de Git del código más actual.
- Ansible tiene una forma limpia e integrada de sincronizar archivos y directorios (sin necesidad de copiar cada uno de ellos cada vez).
- Dentro de los flujos de trabajo de GitHub Actions, nuestro comando de sincronización de Ansible es menos propenso a sincronizar accidentalmente archivos y carpetas que no deberían sincronizarse (por ejemplo, cualquier archivo que aún no esté administrado por Git y que aún no esté en `.gitignore`).
- Los flujos de trabajo de GitHub Actions se pueden ejecutar en repositorios de código privados. Intentar clonar o extraer un repositorio privado en una máquina remota requiere configuración adicional y no es tan sencillo como usar el comando de sincronización de Ansible.

Siempre que necesito copiar varios archivos a un host remoto con Ansible, suelo usar el módulo de sincronización integrado (consulta el módulo de sincronización en la documentación de Ansible para más detalles), ya que gestiona muy bien la copia recursiva de directorios. La siguiente lista muestra la tarea para sincronizar el código del equipo local al remoto.

Listado 4.15 Instalación de Python con Ansible Playbook, parte 6

```
- nombre: Copiar la aplicación FastAPI al servidor remoto
  sincronizar: src:
    '{{ playbook_dir }}/../../src/' dest: /opt/projects/
    roadtok8s/py/src/ recursivo: sí eliminar: sí
```

Se nos presenta otra variable integrada de Ansible llamada `{{ playbook_dir }}`, que representa la ubicación donde se ejecuta el playbook. En este caso, la variable `playbook_dir` hace referencia a la ubicación de este playbook, que se encuentra en `devops/ansible/`. Esto significa que debemos sincronizar la carpeta `src` desde dos niveles superiores, donde entra en juego `../../src/`. `dest` es la ubicación en la máquina remota donde queremos sincronizar los archivos. En este caso, queremos sincronizar los archivos con `/opt/projects/roadtok8s/py/src/`, la misma ubicación que creamos en el listado 4.16.

Usamos `recursive: yes` y `delete: yes` para garantizar que todos los archivos en `src/` sean los mismos desde GitHub a nuestros hosts remotos, manteniendo así el código fuente sincronizado. Almacenar el código fuente en `src/` es una convención común en los proyectos de Python y es una excelente manera de mantener el código fuente separado de los archivos de configuración, facilitando así la sincronización con Ansible.

Ahora podemos instalar todos los paquetes del proyecto Python desde `requirements.txt`. Si la tarea de sincronización funcionó correctamente, ese archivo se ubicará en `/opt/projects/roadtok8s/py/src/requirements.txt`. Podemos usar este archivo junto con el ejecutable `pip`, ubicado en `/opt/venv/bin/pip`, para instalar los paquetes Python en nuestro entorno virtual. El Listado 4.16 muestra la tarea para instalar los paquetes Python.

Listado 4.16 Instalación de Python con Ansible Playbook, parte 7

```
- nombre: Instalar paquetes de Python en el entorno virtual
  ansible.builtin.pip: requisitos: /opt/
    projects/roadtok8s/py/src/requirements.txt ejecutable: /opt/venv/bin/pip
```

Python pip no instalará ni actualizará un paquete ya instalado, por lo que es perfectamente razonable intentar ejecutar esta tarea en cada ejecución de Ansible. Esta es una excelente manera de garantizar que los paquetes de Python estén siempre actualizados.

Hay configuraciones más avanzadas que se pueden realizar con pip, pero este es el enfoque que adoptaremos.

En este punto, nuestro(s) host(s) remoto(s) tendría(n) nuestro proyecto Python instalado y listo para ejecutarse. Solo tenemos que copiar la configuración de NGINX y Supervisor. Una vez copiada, también notificaremos a una función de Ansible llamada " handler" para que ejecute una tarea basada en estos cambios de configuración.

Los controladores de Ansible son básicamente bloques de devolución de llamada que podemos activar desde cualquier lugar de nuestros playbooks mediante el argumento "notify:". El listado 4.17 muestra la tarea para copiar estas configuraciones.

Listado 4.17 Instalación de Python con Ansible Playbook, parte 8

```
- nombre: Configurar gunicorn y uvicorn con supervisor
  copia:
    src: '{{ playbook_dir }}/../conf/supervisor.conf' dest: /etc/supervisor/conf.d/roadtok8s-py.conf
    notificar: recargar supervisor

- nombre: Configurar nginx copia: src:

  '{{ playbook_dir }}/../conf/nginx.conf' dest: /etc/nginx/sites-available/roadtok8s-py

  notificar: reiniciar nginx
```

Al copiar un solo archivo, suelo usar el módulo de copia integrado (consulta el módulo de copia en la documentación de Ansible para más detalles), como se muestra en el listado 4.17. Este módulo es muy similar al módulo de archivo que usamos en el listado 4.16, pero es más específico para copiar archivos. Si el archivo de configuración del supervisor cambia, se usa la declaración de notificación para activar el controlador de recarga del supervisor . Si la configuración de Nginx cambia, se activa el controlador de reinicio de Nginx . Cada controlador se define fuera del bloque " tasks: " y se ejecuta una vez completadas todas las tareas. Casi cualquier tarea o controlador puede activar la ejecución de un controlador. En este caso, es lógico que, si cambia la configuración de Nginx o del Supervisor, se recargue o reinicie cada servicio correspondiente.

Con la nueva configuración de NGINX añadida, eliminaremos la configuración predeterminada de NGINX y la vincularemos al directorio de sitios habilitados . La siguiente lista muestra cómo hacerlo.

Listado 4.18 Instalación de Python con Ansible Playbook, parte 9

```
- nombre: Habilitar sitio nginx
  comando: ln -s /etc/nginx/sites-available/roadtok8s-py /etc/nginx/ sitios-habilitados

  args:
    crea: /etc/nginx/sites-enabled/roadtok8s-py - nombre: Eliminar el sitio nginx
predeterminado
  archivo:
    ruta: "{{ item }}" estado:
    ausente notificar:

  reiniciar nginx con _elementos: - /
etc/nginx/sites-
  enabled/default - /etc/nginx/sites-available/default
```

Eliminar la configuración predeterminada de NGINX es opcional, pero suele ser recomendable, ya que garantiza que solo se utilice la configuración de NGINX. Usamos el módulo de archivo para eliminar la configuración predeterminada y luego volvemos a usar el módulo de comando para crear un enlace simbólico desde nuestra nueva configuración al directorio de sitios habilitados. Esta es una forma común de configurar NGINX y es una excelente manera de garantizar que nuestra configuración de NGINX esté siempre actualizada.

Ahora es el momento de implementar el controlador para cada servicio. La notificación de controladores suele ocurrir cuando una tarea con `notify:` se ejecuta correctamente y el controlador al que se hace referencia existe. En nuestro caso, usamos `notify: restart nginx` y `notify: reload supervisor`. Debemos recordar que el nombre del controlador debe coincidir exactamente con la declaración de `notify:` (por ejemplo, `restart nginx` y `reload supervisor`). El Listado 4.19 muestra el controlador para recargar Supervisor y reiniciar NGINX.

Listado 4.19 Instalación de Python con Ansible Playbook, parte 10

```
manejadores:
  - nombre: recargar supervisor comando:
    "{{ elemento }}" con_elementos: -
      supervisorctl releer
      - supervisorctl actualizar - supervisorctl
      reiniciar roadtok8s-py notificar:
      reiniciar nginx

  - nombre: reiniciar nginx systemd:
    nombre:
      nginx estado:
      reiniciado
```

Los controladores son funciones de devolución de llamada que solo se ejecutan si se les notifica. Solo se notifican si un bloque (tarea u otro controlador) se ejecuta correctamente. Para notificar a un controlador,

Usamos la declaración notify: <handler name> , asumiendo que <handler name> está definido y existe en el libro de estrategias.

En nuestro caso, el supervisor de recarga del controlador tiene una definición de "notify: restart nginx" . Esto significa que si se llama al supervisor de recarga y se ejecuta correctamente, se activará el controlador de reinicio de nginx . Si el controlador de reinicio de nginx tuviera un supervisor de "notify: reload", se produciría un bucle infinito entre estos controladores. Este bucle infinito debe evitarse, pero es algo a tener en cuenta.

Con el playbook casi completo, vamos a enviarlo a Git y subirlo a GitHub. Una vez hecho esto, deberías tener los siguientes archivos relacionados con la ejecución de Ansible:

- devops/ansible/install-nginx.yaml
- devops/ansible/deploy-python.yaml
- .github/workflows/run-ansible.yaml

Como recordará, run-ansible.yaml tiene un paso que llama al archivo install-nginx.yaml con ansible-playbook install-nginx.yaml.

Debemos actualizar esta línea para que lea ansible-playbook deploy-python.yaml . La siguiente lista muestra el paso actualizado en el archivo de flujo de trabajo.

Listado 4.20 Implementación de la aplicación Python con Ansible con GitHub Actions

```
- nombre: Implementar Python a través de Ansible
  directorio de trabajo: ./devops/ansible/ ejecutar: | ansible-
    playbook
    deploy-python.yaml
```

Antes de enviar este playbook y el flujo de trabajo actualizado a nuestro repositorio de GitHub, asegúrese de comparar el suyo con el playbook de referencia completo en el repositorio oficial de mi proyecto de Python en <https://github.com/jmitchel3/roadtok8s-py>. Con el...

Tras el playbook y el flujo de trabajo actualizado de GitHub Actions, es hora de ejecutarlo y ver qué sucede. Al ejecutarlo, debería ver el resultado en la figura 4.14.



The screenshot shows the output of a GitHub Actions workflow named 'Run Ansible Playbooks'. The output is displayed in a monospace font with line numbers on the left. The workflow is running a playbook named 'install-nginx.yaml'. The output shows the following steps:

```

16 ▶ Run ansible-playbook install-nginx.yaml
11
12 PLAY [Install NGINX]
13
14 TASK [Gathering Facts]
15 ok: [***]
16
17 TASK [Install NGINX]
18 ok: [***]
19
20 TASK [Ensure NGINX is running]
21 ok: [***]
22
23 PLAY RECAP
24 ***: ok=3  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
25

```

The output indicates that the playbook was executed successfully, with all tasks completing with an 'ok' status. The 'PLAY RECAP' line shows that 3 tasks were successful, and no errors or warnings were encountered.

Figura 4.14 Resultado de la implementación de Python de Ansible en GitHub Actions

Si se produjeron errores con este flujo de trabajo, es posible que deba usar una sesión SSH manual para diagnosticar todos los problemas que puedan haberse producido, tal como hicimos en el capítulo 3. Si no se produjo ningún error, debería poder abrir su navegador web con su dirección IP y ver la aplicación Python ejecutándose. Si la aplicación Python se ejecuta, la ha implementado correctamente con Ansible y GitHub Actions. ¡Felicidades!

Si tuviste éxito, te recomiendo cambiar el código fuente de Python y ejecutarlo de nuevo. Asegúrate de que los resultados deseados se obtengan correctamente. El siguiente paso es implementar esta automatización de Ansible y GitHub Actions en la aplicación Node.js.

4.2.4 Ansible para nuestra aplicación Node.js

Implementar una aplicación Node.js con Ansible y GitHub Actions es prácticamente idéntico a implementar una aplicación Python, con algunas excepciones. Vamos a crear una nueva máquina virtual para este proceso, aunque podríamos implementar esta aplicación en la misma máquina virtual que el proyecto Python, como vimos en el capítulo 3. Esta sección será mucho más concisa que las anteriores, ya que podemos reutilizar gran parte de nuestro código y configuraciones.

Recapitemos lo que debemos hacer para que nuestra aplicación Node.js se ejecute correctamente en GitHub Actions:

1. Cree nuevas claves SSH: tener nuevas claves ayudará a garantizar seguridad adicional para cada aplicación web aislada.
 - Agregue la nueva clave privada SSH como un secreto de GitHub Actions en nuestro repositorio Node.js.
 - Instale la nueva clave pública SSH en Akamai Linode.
 - Aprovechone una nueva máquina virtual y agregue la nueva dirección IP como un secreto de GitHub Actions.
2. Copie los archivos de configuración de NGINX y Supervisor para Node.js del capítulo 3.
3. Cree un nuevo Ansible Playbook para nuestra aplicación Node.js.
4. Copie y modifique el flujo de trabajo de GitHub Actions para ejecutar Ansible, creado en la sección 4.2.3.

Deberías poder realizar los pasos 1 y 2 fácilmente, así que te lo dejo a ti. Ahora, sigamos adelante y creemos nuestro Playbook de Ansible para nuestra aplicación Node.js.

CREANDO UN NUEVO MANUAL DE ANSIBLE PARA NUESTRA APLICACIÓN NODE.JS

Ya tenemos la mayor parte de la base necesaria para crear un Playbook de Ansible basado en Node.js para nuestro proyecto. El principal

La diferencia entre usar Ansible para instalar Node y lo que hicimos en el capítulo 3 es que vamos a actualizar el paquete apt directamente en lugar de usar el Administrador de versiones de Node.

En su aplicación Node.js, ubicada en ~/dev/roadtok8s/js, cree la carpeta devops y agregue el archivo deploy-js.yaml. A continuación, declare los primeros pasos como se describe en el listado 4.21. Dado que NVM y Ansible no se complementan bien, añadimos un nuevo bloque llamado vars (para variables) que podemos reutilizar en este playbook. En este caso, asignamos a la variable nodejs_v el valor 18 para instalar Node.js v18, como se muestra en el siguiente listado.

Listado 4.21 Instalación de Node.js con Ansible Playbook, parte 1

```
---
- nombre: Implementar la aplicación Node.js
  hosts: todos
  convertirse en: sí
  variables:
    nodejs_v: 18
  tareas:
    - nombre: Sistema de actualización y mejora
      apt:
        upgrade: yes
        update_cache: yes
        cache_valid_time: 3600
    - nombre: Instalar dependencias
      apto:
        paquete: - curl
        - rsync -
        nginx -
        supervisor estado:
        presente etiquetas:
        instalar
```

Una vez más, comenzamos a actualizar apt y varios requisitos del sistema para sentar las bases para la instalación de Node.js.

Lamentablemente, no podemos instalar Node.js simplemente con apt install nodejs, por lo que necesitamos agregar una nueva clave y un nuevo repositorio apt con Ansible usando los bloques que agregan apt_key y apt_repository, respectivamente. Una vez hecho esto, podemos instalar Node.js usando directamente el bloque apt, como se muestra en la siguiente lista.

Listado 4.22 Instalación de Node.js con Ansible Playbook, parte 2

```
---
- nombre: Implementar la aplicación Node.js
  hosts: todos
  convertirse en: sí
  variables:
    nodejs_v: 18 tareas:

    - nombre: Sistema de actualización y mejora
      apt:
        upgrade: yes
        update_cache: yes
        cache_valid_time: 3600 - name:
      Instalar dependencias
      apto:

        paquete: - curl
        - rsync -
        nginx -
        supervisor estado:
      presente etiquetas:
      instalar
```

Una vez que esto esté completo, usaremos el bloque npm para actualizar npm globalmente, como se ve en la siguiente lista.

Listado 4.23 Instalación de Node.js con Ansible Playbook, parte 3

```
- nombre: Actualizar npm globalmente
  npm:
    global: sí nombre:
    "{{ elemento }}" estado: último
    con_elementos:

- npm
```

El bloque npm está integrado en Ansible, pero nuestra máquina host debe tenerlo instalado; de lo contrario, no tendrá éxito.

Antes de continuar, debemos actualizar nuestro archivo supervisor.conf para reflejar el cambio en la forma en que iniciamos nuestra aplicación Node.js. Simplemente puede cambiar el comando de `/root/.nvm/versions/node/v18.16.0/bin/node main.js` a `command=node main.js`. Con el archivo supervisor.conf actualizado, podemos empezar a sincronizar el código con el equipo host, como se muestra en la siguiente lista.

Listado 4.24 Instalación de Node.js con Ansible Playbook, parte 4

```

- nombre: Configurar el directorio de la aplicación
  archivo:
    ruta: "{{ elemento }}" estado:
    directorio modo: '0755'
    con_elementos: - /
    opt/projects/
    roadtok8s/js/src/ - /var/log/supervisor/roadtok8s/js/ -
    nombre: Copiar la aplicación Node.js al servidor remoto
sincronizar: src: '{{ playbook_dir }}/../../src/' dest: /opt/projects/
roadtok8s/js/src/
    recursivo: sí eliminar: sí - nombre: Copiar Package.json al
    servidor remoto sincronizar: src: '{{ playbook_dir }}/../../
    package.json' dest: /
    opt/projects/
roadtok8s/js/package.json recursivo: sí eliminar: sí

```

Este proceso es casi idéntico a lo que hicimos en los listados 4.14 y 4.15, sólo modificado ligeramente para tener en cuenta la aplicación Node.js.

Con el código sincronizado, ahora podemos instalar las dependencias de la aplicación Node.js con el bloque npm , como se ve en el siguiente listado.

Listado 4.25 Instalación de Node.js con Ansible Playbook, parte 5

```
- nombre: Instalar paquetes de Node.js shell: | npm
  install args:
    ejecutable: /bin/
  bash
    chdir: "/opt/projects/roadtok8s/js/"

  notificar: supervisor de recarga
```

El resto de este archivo es el mismo que el de la aplicación Python anterior, como puede verse en la siguiente lista.

Listado 4.26 Instalación de Node.js con Ansible Playbook, parte 6

```

- nombre: Configurar Node.js con supervisor
  copia:
    src: ../../conf/supervisor.conf dest: /etc/supervisor/
    conf.d/roadtok8s-js.conf notificar: recargar supervisor

- nombre: Configurar nginx
  copia:
    src: ../../conf/nginx.conf destino: /etc/nginx/
    sites-available/roadtok8s-js
  Notificar: reiniciar nginx - nombre:
  Habilitar sitio nginx
  comando: ln -s /etc/nginx/sites-available/roadtok8s-js /etc/nginx/ sitios-habilitados

  args:
    crea: /etc/nginx/sites-enabled/roadtok8s-js - nombre: Eliminar el sitio nginx
  predeterminado
  archivo:
    ruta: "{{ item }}" estado:
    ausente notificar:
    reiniciar nginx con _elementos: - /
    etc/nginx/sites-
    enabled/default - /etc/nginx/sites-available/default

manejadores:
- nombre: recargar supervisor comando:
  "{{ elemento }}" con _elementos: -
  supervisorctl releer
  - supervisorctl actualizar - supervisorctl
  reiniciar roadtok8s-js notificar: reiniciar
  nginx

- nombre: reiniciar nginx systemd:
  nombre:
    nginx estado:
    reiniciado

```

Ahora que tenemos nuestro playbook de Ansible completo, actualicemos nuestro flujo de trabajo de GitHub Actions.

FLUJO DE TRABAJO DE ACCIONES DE GITHUB PARA NODE.JS Y ANSIBLE PLAYBOOK

Copia el archivo completo del flujo de trabajo de GitHub Actions del proyecto Python (~/dev/roadtok8s/py/.github/workflows/run-ansible.yaml) al proyecto Node.js (~/dev/roadtok8s/js/.github/workflows/run-ansible.yaml). Después, deberás cambiar una línea para que nuestro nuevo playbook de Ansible funcione: ansible-playbook deploy-python.yaml a ansible-playbook deploy-js.yaml. Todo lo demás debería funcionar igual.

Con este cambio, ahora podemos confirmar nuestros cambios en Git y subirlos a GitHub. Antes de ejecutar el flujo de trabajo, debe verificar los siguientes secretos de GitHub Actions:

- CLAVE PRIVADA SSH
- DIRECCIÓN IP DE LA INSTANCIA DE AKAMAI

Como recordatorio, si aún no lo ha hecho, deberá crear una nueva clave SSH_PRIVATE_KEY con ssh-keygen, instalarla en Akamai Linode y, a continuación, agregar la clave privada a los secretos de GitHub Actions. También deberá crear una nueva máquina virtual en Akamai Linode con la nueva clave SSH para obtener una dirección IP nueva y válida que agregará a los secretos de GitHub Actions como AKAMAI_INSTANCE_IP_ADDRESS.

Una vez hecho esto, puedes ejecutar tu flujo de trabajo de GitHub Actions y verificar que tu aplicación Node.js se esté ejecutando visitando AKAMAI_INSTANCE_IP_ADDRESS una vez finalizado el flujo de trabajo. Te dejo a ti que revises este proceso y lo verifiques.

Resumen

- La integración continua es el proceso de integrar constantemente los cambios de código en una única fuente de verdad, es decir, un repositorio Git.
- La entrega continua es el proceso de entregar constantemente cambios de código a un repositorio de Git en preparación para la implementación.
- La implementación continua es el proceso de implementar constantemente cambios de código en un entorno de producción.
- Implementar entornos consistentes para sus aplicaciones es esencial para garantizar que sus aplicaciones funcionen según lo previsto.
- Los flujos de trabajo de GitHub Actions están diseñados para ser fáciles de entender y usar, y al mismo tiempo son increíblemente flexibles para las distintas herramientas que podamos necesitar usar y automatizar.
- Ansible es una poderosa herramienta que se puede utilizar para automatizar la instalación y configuración de software en máquinas remotas.
- NGINX es un potente servidor web que se ubica delante de las aplicaciones web para brindar seguridad y rendimiento adicionales.
- Supervisor es un potente administrador de procesos que puede utilizarse para garantizar que nuestras aplicaciones estén siempre ejecutándose.
- El formato YAML es un elemento básico de la automatización y la implementación, independientemente del lenguaje de programación en el que escriba sus aplicaciones.
- La ejecución de aplicaciones en máquinas remotas requiere que los sistemas se actualicen y supervisen continuamente para garantizar que se instalen y configuren las dependencias correctas.

- Incluso con un diseño de aplicación simple, el proceso de implementación de aplicaciones puede ser complejo y propenso a errores.
- Los entornos de ejecución de Node.js y Python pueden compartir varias dependencias de producción, como NGINX y Supervisor, pero son lo suficientemente diferentes como para requerir configuraciones específicas, lo que genera más complejidad y más posibilidades de errores.

5 aplicaciones de contenedores

Este capítulo cubre

- Instalación y uso de comandos básicos de Docker
- Ejecución de imágenes de contenedores a través del registro de contenedores
- Hacer que las aplicaciones sean portátiles y que los tiempos de ejecución tengan versiones controladas
- Comprender el alojamiento de contenedores en los registros
- Construir, ejecutar y enviar contenedores usando una máquina local

A finales de la década de 1900, teníamos que insertar discos y cartuchos en nuestros dispositivos para ejecutar software o jugar juegos. Actualizar estas aplicaciones continuamente era prácticamente imposible, por lo que los desarrolladores trabajaron incansablemente para garantizar que estuvieran bloqueadas y lo más libres de errores posible antes de su lanzamiento. Como se puede imaginar, este era un proceso extremadamente lento, especialmente comparado con los estándares actuales.

Estos discos y cartuchos solo se podían jugar o ejecutar en los dispositivos para los que fueron diseñados; Mario se jugaba en dispositivos Nintendo, y las computadoras Windows usaban Microsoft Word. En otras palabras, los dispositivos de almacenamiento que albergaban el software estaban vinculados directamente al sistema operativo que lo ejecutaba. Aún hoy contamos con la versión digital de esto con tiendas de aplicaciones y descargas digitales, con una diferencia importante: internet.

Como saben, hoy en día existen muy pocas aplicaciones que no utilicen internet de alguna forma, independientemente del dispositivo en el que se ejecuten. Esto significa que las aplicaciones que usamos a diario rara vez están vinculadas al sistema operativo en el que se ejecutan. En otras palabras, el hardware y el sistema operativo importan menos que nunca, simplemente porque las aplicaciones utilizan internet y se ejecutan en servidores en otros lugares. Los primeros capítulos de este libro destacaron esto por el simple hecho de que omití los rigurosos requisitos del sistema para ejecutar Python, Node.js, Git, etc. Con el software de finales de los 90, habría tenido que proporcionarles una lista extensa de requisitos de hardware y sistema para ejecutar las aplicaciones e incluso escribir diferentes versiones del libro según el sistema operativo que tuvieran. Ahora, puedo simplemente darles un sistema operativo vagamente, y hay una gran probabilidad de que el software funcione en su máquina.

Eso es, hasta que deja de funcionar. El software de código abierto suele funcionar bien con el proceso de instalación inicial, pero en cuanto empezamos a necesitar dependencias de terceros, nuestras aplicaciones y procesos de CI/CD pueden empezar a fallar, requiriendo mucha atención para estabilizarlos. Aquí es donde entran en juego los contenedores.

Los paquetes de terceros suelen ser esenciales para el desarrollo de aplicaciones modernas. Por ejemplo, podrías necesitar conectar tu aplicación a algún tipo de base de datos. Si bien es probable que exista un paquete nativo de Python o Node.js que puedas usar, estos paquetes suelen asumir también instalaciones a nivel de sistema. Ya sabes lo que dicen sobre las suposiciones, ¿verdad?

Los contenedores agrupan tu código y las dependencias de terceros a nivel de sistema para que puedas tener una aplicación portable. Este paquete se conoce como imagen de contenedor y fue desarrollado y popularizado por Docker. Lo mágico de los contenedores es que si se instala un entorno de ejecución de contenedor, también conocido como entorno de ejecución de Docker, en cualquier...

sistema, existe una gran posibilidad de que su aplicación se ejecute sin instalaciones adicionales.

Los contenedores aún requieren una dependencia a nivel de sistema para ejecutarse, un entorno de ejecución de contenedores, pero el contenido del contenedor puede ser Python, Node.js, Java, Ruby y prácticamente cualquier software que se ejecute en Linux. Los contenedores son la mejor solución moderna para garantizar que su aplicación sea portátil y pueda ejecutarse en cualquier sistema con un entorno de ejecución de contenedores instalado. Los contenedores tienen varias limitaciones que seguiremos explorando en el resto del libro. (Alerta de spoiler: Una de estas limitaciones es la razón de ser de Kubernetes). Comencemos instalando Docker Desktop en su equipo.

5.1 Hola mundo con Docker

En este libro, usaremos Docker para compilar, ejecutar y enviar nuestras imágenes de contenedor. Hay varias maneras de realizar cada una de estas tareas, pero nos ceñiremos a Docker para simplificar las cosas.

Docker se instala más fácilmente en Linux, macOS y el Subsistema de Windows para Linux (WSL) con una herramienta llamada Docker Desktop. Si no usa WSL, necesita Windows Pro para instalar Docker Desktop. Para instalar Docker Desktop, siga estos pasos:

1. Visite la guía de instalación oficial de Docker Desktop (<https://docs.docker.com/desktop/install/mac-install/>).
2. Siga las instrucciones para su sistema operativo.
3. Abra su línea de comandos y verifique que Docker esté instalado con `docker -- versión`.

Una vez que tengas instalado Docker, veremos lo sencillo que es ejecutar aplicaciones de terceros.

5.1.1 Ejecución de su primer contenedor

Docker cuenta con numerosas imágenes de contenedores preconstruidas que podemos ejecutar en cualquier momento. Puedes encontrar muchas imágenes de contenedores de código abierto en Docker Hub (<https://hub.docker.com/>). Más adelante en este capítulo, también usaremos Docker Hub para alojar nuestra imagen de contenedor. Ahora, ejecutemos la imagen de contenedor oficial de Docker para NGINX (https://hub.docker.com/_/nginx), como se muestra en la siguiente lista.

Listado 5.1 Hola mundo con Docker: Ejecución de la imagen del contenedor NGINX

Docker ejecuta nginx

La figura 5.1 muestra el resultado de ejecutar la imagen del contenedor NGINX.

```
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
3ae0c06b4d3a: Pull complete
efe5035ea617: Pull complete
a9b1bd25c37b: Pull complete
f853dda6947e: Pull complete
38f44e054f7b: Pull complete
ed88a19ddb46: Pull complete
495e6ahhed48: Pull complete
Digest: sha256:08bc36ad52474e528cc1ea3426b5e3f4bad8a130318e3140d6cfe29c8892c7ef
Status: Downloaded newer image for nginx:latest
```

Figura 5.1 Salida de la primera ejecución del contenedor

Antes de visitar la instancia en ejecución de esta imagen de contenedor, vemos la salida que contiene "No se puede encontrar la imagen..." y "Extrayendo desde", lo que significa que Docker intenta descargar automáticamente una imagen porque falta en nuestro sistema local. Esta descarga intenta usar Docker Hub, el alojamiento oficial de contenedores de Docker, también conocido como registro de contenedores. Docker usará este registro por defecto a menos que tenga una buena razón para cambiarlo.

Los registros de Docker también son de código abierto, por lo que podría, si

Son súper aventureros y organizan su propio registro, pero eso está fuera del contexto de este libro.

Para las imágenes oficiales de Docker, el comando es `docker run <imageName>`, y para los comandos de terceros (incluidos los nuestros), el comando es `docker run <userRegistryName>/<imageName>`, como `docker run codingforentrepreneurs/try-iac`. Abordaremos esto con más detalle más adelante en este capítulo.

Entonces, ¿dónde podemos acceder a nuestra instancia de NGINX? Pues no podemos. Al menos todavía no. Necesitamos proporcionar a Docker argumentos adicionales, llamados indicadores, para poder acceder a la aplicación que ejecuta el contenedor.

5.1.2 Exposición de puertos

Ejecutar una imagen de contenedor sin argumentos ejecutará el software dentro de la imagen, pero no permitirá ningún tipo de tráfico externo por defecto. Para solucionar esto, necesitamos exponer las imágenes de contenedor a un puerto en nuestro equipo local.

Los puertos permiten la entrada y salida de tráfico de un sistema o, en nuestro caso, de una imagen de contenedor en ejecución. Podemos elegir un número de puerto para el sistema, pero debemos respetar el número de puerto que espera la aplicación dentro del contenedor. Para NGINX, el puerto predeterminado es el 80.

Para nuestro sistema local, usaremos el puerto 8080 para distinguir el comando. La sintaxis de este argumento, también conocido como indicador, es `-p <localPort>:<containerPort>`, que en este ejemplo se traduce como `-p 8080:80`. Aquí hay algunos ejemplos de cómo cambiar los números de puerto:

- `docker run -p 8080:80 nginx`: nuestro localhost:8080 se asigna al puerto 80 del contenedor NGINX. `docker run -p`
- `6543:5432 postgres`: nuestro localhost:6543 se asigna al puerto 5432 del contenedor Postgres

pag

grano

pag

(https://hub.docker.com/_/postgres). `docker run -p 3001:8080`

- codingforentrepreneurs/try-iac— Nuestro localhost:3001 se asigna al puerto del contenedor try-iac 8080.

Suponiendo que `docker run nginx` sigue ejecutándose en tu terminal, presiona Control + C para detener el contenedor en ejecución e iniciar uno nuevo en `docker run -p 8080:80 nginx`. Una vez hecho esto, visita <http://localhost:8080> en tu equipo local y verás el mensaje estándar de NGINX "Hola Mundo", como en la figura 5.2.



Figura 5.2 Visita al puerto 80 en la imagen del contenedor NGINX en localhost:8080

Exponer los puertos como uno de nuestros primeros pasos ayuda a destacar que los contenedores están mayormente aislados del entorno del sistema operativo circundante hasta que agregamos parámetros adicionales para cambiar su funcionamiento. En otras palabras, los contenedores están diseñados para estar aislados del equipo host por defecto. Esto es positivo, ya que significa que podemos ejecutar prácticamente cualquier software en un contenedor sin modificar la configuración de nuestro sistema, salvo para ejecutar Docker. El contenedor tiene su propio mundo dentro.

Bueno. Veamos a qué me refiero con entrar al shell bash del contenedor.

5.1.3 Entrada a un contenedor

Los contenedores tienen su propio shell dentro del contenedor, junto con un sistema de archivos y prácticamente todo lo que ofrece un sistema operativo normal. Por eso, a menudo se les conoce como máquinas virtuales ligeras, ya que están aislados del sistema operativo anfitrión, pero conservan gran parte de la misma funcionalidad.

Veamos el contenido del contenedor NGINX usando otra bandera que nos permite usar la terminal interactiva `-it`. Esta terminal interactiva es muy similar a una conexión Secure Shell (SSH) que hemos usado en capítulos anteriores. Para usar la shell bash del contenedor, usaremos la terminal interactiva. Si necesitamos ejecutar otros comandos (por ejemplo, comandos de entrada de usuario desde una aplicación), también podemos usar la terminal interactiva (por ejemplo, la shell de Python o Node.js basada en contenedor). Para acceder a nuestro contenedor NGINX, ejecute los siguientes comandos:

- `docker run -it nginx /bin/bash`—Ingresa al shell Bash `ls /etc/nginx/conf.d/`
- —Enumera los archivos en el directorio de configuración de NGINX `exit`—Sale del contenedor
-

Debería ver algo parecido a la figura 5.3.

```
% docker run -it nginx /bin/bash
root@718fa4317aa1:/# ls /etc/nginx/conf.d/
default.conf
root@718fa4317aa1:/# exit
exit_
```

Figura 5.3 Ingresar al shell bash de la imagen del contenedor NGINX

Como vemos en este sencillo ejemplo, el entorno de ejecución del contenedor tiene un mundo propio que puede o no existir en nuestro host local. Por eso los contenedores son tan potentes. Podemos ejecutar prácticamente cualquier software en un contenedor y funcionará de la misma manera en cualquier sistema con un entorno de ejecución de contenedor instalado. Es posible que tenga la tentación de tratar un contenedor como una máquina virtual, y puede hacerlo, pero le ofreceré una simple advertencia: los contenedores están diseñados para ser destruidos.

5.1.4 Contenedores sin estado

Las aplicaciones de bases de datos son excelentes ejemplos de software con estado. Si se destruye la máquina que ejecuta la base de datos, es muy probable que se pierdan todos los datos. El estado de los datos debe persistir en algún lugar, por lo que contar con copias de seguridad es esencial para las aplicaciones con estado.

Las máquinas virtuales, al igual que su máquina local, tienen estado de manera predeterminada, pero si la máquina se destruye, el estado o los datos desaparecen para siempre.

Nuestras aplicaciones Python y Node.js son actualmente ejemplos de aplicaciones sin estado, ya que siempre podemos recrear el mismo entorno, independientemente de si la máquina en ejecución se destruye o no. Como probablemente ya sepa, recrear el entorno es posible y, en gran medida, sencillo gracias a los archivos requirements.txt y package.json .

Antes de profundizar en la naturaleza sin estado de los contenedores, veamos un ejemplo práctico: ejecutemos una imagen de contenedor basada en Linux e instalemos NGINX. El propósito de este ejemplo es ilustrar lo que no debemos hacer con los contenedores, pero sí con las máquinas virtuales.

Para este ejemplo, añadiremos la opción `--rm` a nuestro comando `docker run` para eliminar el contenedor inmediatamente después de detenerlo. Esto es lo que haremos:

1. `docker run --rm -it ubuntu /bin/bash`—Descargas ubuntu y entra en el entorno del shell bash
2. `apt-get update`: actualiza el administrador de paquetes dentro del recipiente
3. `apt-get install nginx`: instala NGINX
4. `cat /etc/nginx/sites-available/default`: muestra el archivo de configuración de NGINX
5. `nginx -v`: muestra la versión de NGINX
6. `exit`: sale del contenedor

Este proceso de instalación es muy similar al que se esperaría en un entorno de máquina virtual, ¡porque este contenedor sigue siendo Ubuntu! Así que, llegados a este punto, deberíamos estar listos para usar NGINX, ¿verdad? Bueno, usemos de nuevo el shell bash en este contenedor y veamos qué sucede:

```
docker run --rm -it ubuntu /bin/bash
cat /etc/nginx/sites-available/default nginx -v salir
```

¿Qué? ¡Errores! ¿Por qué? Al repetir este proceso sin instalar NGINX, aparecen los errores «No existe el archivo o directorio» y «Comando no encontrado». Estos errores nos indican claramente que no podemos modificar a largo plazo un contenedor ya existente.

Imagen desde el entorno del contenedor. Esta es la naturaleza sin estado de los contenedores. Entonces, ¿cómo modificamos los contenedores? Este es el propósito de un Dockerfile y lo que haremos en la siguiente sección.

5.2 Diseño de nuevas imágenes de contenedores

Crear una nueva imagen de contenedor requiere al menos dos cosas: un constructor de contenedores y un Dockerfile. El constructor de contenedores es simplemente otro comando disponible al usar Docker Desktop. El Dockerfile es un documento especial que nos permite escribir instrucciones sobre cómo construir nuestra imagen de contenedor.

Recordando la sección anterior, crearemos nuestra primera imagen de contenedor instalando NGINX directamente en la imagen de contenedor de Ubuntu preexistente. Esto nos permitirá modificar la imagen del contenedor y guardarla como una nueva.

5.2.1 Creación de un Dockerfile

La sintaxis de un Dockerfile es exclusiva de Docker, pero normalmente se reduce a usar los siguientes comandos:

- **FROM <image>**—La imagen de contenedor preconstruida de la que podemos heredar.
- **RUN <tu comando>**: Una instrucción para instalar otro software o ejecutar un comando. Suelo pensar en esto como un comando de shell bash que se almacena en caché durante las compilaciones (porque eso es lo que hace). Lo usarás mucho.
- **CMD <su comando>**: una instrucción para ejecutar un comando al iniciar el contenedor. Para NGINX, esto

El comando será `nginx -g 'daemon off;'`. Las instrucciones se dividen en una matriz de cadenas, por lo que el resultado es `CMD ["run", "this", "--app"]` en lugar de simplemente ejecutar esto --

aplicación.

Creemos nuestro primer Dockerfile en nuestra máquina en la ubicación `~/Dev/roadtok8s-hello-docker/Dockerfile` con el contenido de la siguiente lista.

Listado 5.2 Su primer Dockerfile

```
DESDE ubuntu
EJECUTAR apt-get update && apt-get install -y nginx
CMD ["nginx", "-g", "demonio desactivado;"]
```

Antes de usar este Docker, llamemos este contenedor simplemente `rk8s-nginx`. Usaremos este nombre al compilar el contenedor mediante la etiqueta: `-t <nombre de su etiqueta>`. Ahora, debemos navegar al directorio de su Dockerfile y compilar y etiquetar la imagen de nuestro contenedor, como se muestra en el listado 5.3.

Listado 5.3 Construyendo su primera imagen de contenedor desde un Dockerfile

```
cd ~/Dev/roadtok8s-hello-docker#1 docker
build -t rk8s-nginx -f Dockerfile .
```

#1 El directorio donde se encuentra tu
El Dockerfile debe estar ubicado

La figura 5.4 muestra el resultado de ejecutar el comando de compilación. El tiempo para construir esta imagen de contenedor variará de una máquina a otra, pero debería ser relativamente rápido porque nuestro Dockerfile realiza muy pocos cambios en la imagen de contenedor de Ubuntu preconstruida de la que estamos heredando.

```

❏% docker build -t rk8s-nginx -f Dockerfile .
[+] Building 0.0s (6/6) FINISHED                                docker:desktop-linux
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 131B                              0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest 0.0s
=> [1/2] FROM docker.io/library/ubuntu                          0.0s
=> CACHED [2/2] RUN apt-get update && apt-get install -y nginx 0.0s
=> exporting to image                                           0.0s
=> => exporting layers                                           0.0s
=> => writing image sha256:f77b47102f215d4e27878d1d5c232c000f82915fc0f33c0779b 0.0s
=> => naming to docker.io/library/rk8s-nginx                  0.0s

```

Figura 5.4 Creación de su primera imagen de contenedor a partir de un Dockerfile

Si es la primera vez que creas una imagen de contenedor, ¡enhorabuena! Ahora, ejecutémosla y veamos qué sucede.

5.2.2 Ejecución de su primera imagen de contenedor

Ahora que hemos creado nuestra primera imagen de contenedor a partir de un Dockerfile personalizado, ejecutémosla. Es importante recordar que la etiquetamos como rk8s-nginx en nuestra máquina local, así que usaremos ese nombre para ejecutarla. El Listado 5.4 muestra el comando para ejecutar nuestra imagen de contenedor.

Listado 5.4 Ejecución de su primera imagen de contenedor desde un Dockerfile

```
ejecutar docker -p 8080:80 rk8s-nginx
```

Una vez ejecutado esto, debería poder abrir `http://localhost:8080` para volver a ver la página estándar de NGINX "Hola Mundo". Si el puerto está en uso, puede cambiar el número de puerto a cualquier valor que desee o detener la ejecución de la aplicación/contenedor en ese puerto.

Una vez más, verificaremos que NGINX se haya instalado en nuestro contenedor mediante el shell bash. Ejecute los siguientes comandos:

```
1. docker run --rm -it rk8s-nginx /bin/bash 2. cat /etc/nginx/
sites-available/default 3. nginx -v 4. salir
```

Todos estos comandos (cambiamos Ubuntu por rk8s-nginx) deberían estar disponibles en nuestra versión modificada de Ubuntu gracias a nuestro Dockerfile y al contenedor creado. Ahora que conocemos los conceptos básicos, vamos a crear nuestras imágenes de contenedor de Python y Node.js.

5.3 Contenerización de aplicaciones Python

Docker Hub ofrece una gran cantidad de imágenes de contenedor prediseñadas y oficiales, como la imagen de contenedor [//hub.docker.com/_/python\[official Python\]](https://hub.docker.com/_/python[official Python]). Al igual que con la imagen de contenedor https://hub.docker.com/_/ubuntu, las imágenes prediseñadas nos simplifican mucho la vida.

Contenerizar aplicaciones Python es muy accesible gracias al Índice de Paquetes de Python (PyPI; <https://pypi.org/>) que nos permite usar pip para instalar herramientas como FastAPI, Django, solicitudes y gunicorn. Sin PyPI, aún podríamos instalar herramientas, pero sería un proceso mucho más complejo. Con esto en mente, creemos nuestro Dockerfile de Python.

5.3.1 Etiquetas de versión y un Dockerfile de Python

Cuando se trata de proyectos de Python, normalmente empezamos con una versión específica en mente. Anteriormente, mencionamos que debemos...

Use Python 3.8 o superior. En esta sección, usaremos Python 3.11 como nuestra versión preferida, pero ¿cómo?

Por un lado, podríamos usar el bloque RUN <comando> en nuestro Dockerfile para instalar una versión específica. Por otro lado, la imagen del contenedor Python de Docker Hub tiene una gran cantidad de etiquetas (https://hub.docker.com/_/python/tags) que incluyen varias versiones de Python.

Para encontrar la versión que necesitamos, necesitaremos realizar los siguientes pasos:

1. Visite el repositorio oficial de contenedores de Python en https://hub.docker.com/_/python.
2. Haga clic en la pestaña Etiquetas (o navegue directamente a https://hub.docker.com/_/python/tags).
3. Filtra los resultados por 3.11.4 (o cualquier versión que tengas de desear).

La figura 5.5 muestra los resultados de los pasos anteriores con algunas versiones recortadas.

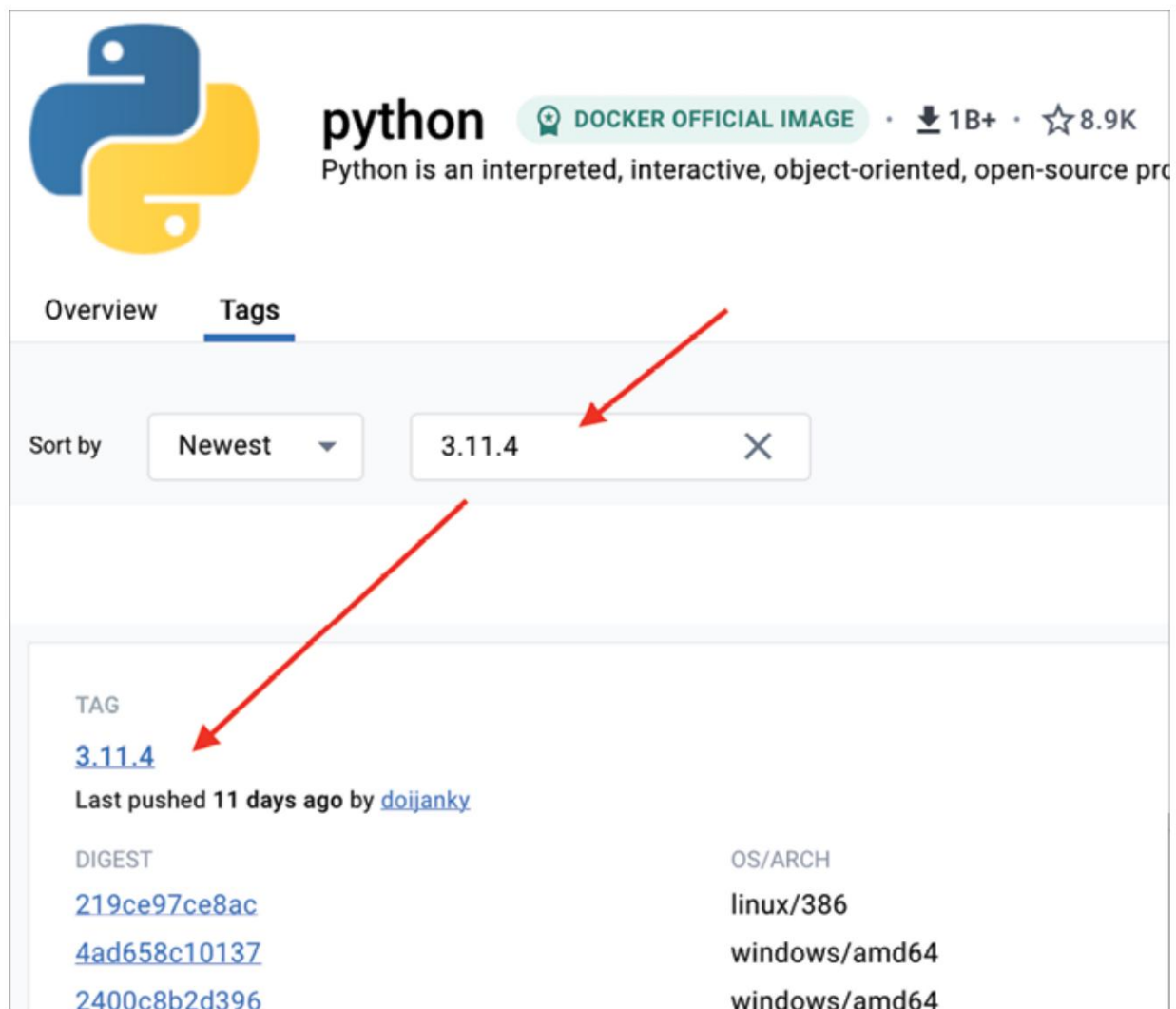


Figura 5.5 Etiquetas Python de Docker Hub

La Figura 5.6 muestra el comando exacto para descargar la imagen a su equipo local. Este proceso, conocido como extracción de una imagen, se realiza automáticamente al ejecutar o compilar una imagen.

Docker Hub mantiene casi todas las versiones de código estable publicadas para los principales proyectos de código abierto. Una gran ventaja de esto es que, a partir de 2035, la versión 3.11.4 probablemente seguirá disponible tal como está aquí. De no ser así, probablemente se deba a algún riesgo de seguridad importante o a un cambio drástico en Docker Hub.

```
docker pull python:3.11.4
```



Figura 5.6 Captura de pantalla de Docker Hub del comando pull

En este caso, el comando `docker pull python:3.11.4` nos muestra la etiqueta de contenedor exacta que necesitamos para usar esta imagen. El formato es `docker pull <nombreContenedor>:<etiqueta>`. Podemos usar este formato en nuestro Dockerfile, en la línea `FROM <imagen>`, como veremos en la siguiente sección.

Antes de continuar, ejecutemos esta imagen de contenedor localmente para ver qué sucede. La siguiente lista muestra el comando para ejecutarla.

Listado 5.5 Ejecución de la imagen del contenedor de Python

```
docker run -it python:3.11.4
```

Al ejecutar esto, es muy probable que el contenedor tenga que descargarse (en mi caso, volví a ver "No se puede encontrar la imagen 'python:3.11.4' localmente"). Una vez que termina la descarga, tengo el shell de Python a mi disposición, ejecutándose a través de Docker. La Figura 5.7 muestra el resultado de ejecutar este comando.

```

❗ % docker run -it python:3.11.4
Unable to find image 'python:3.11.4' locally
3.11.4: Pulling from library/python
42cbebf8bc11: Pull complete
9a0518ec5756: Pull complete
356172c718ac: Pull complete
dddc3ceb998: Pull complete
8e07adfdac1d: Pull complete
dc87587bf469: Pull complete
b7ee9aadd410: Pull complete
47d5195c2be5: Pull complete
Digest: sha256:d73088ce13d5a1eecd05b47736041ae6921d08d2f240035d99642db98bc8d4
Status: Downloaded newer image for python:3.11.4
Python 3.11.4 (main, Jul 4 2023, 21:57:59) [GCC 12.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> █

```

Figura 5.7 Ejecución de la imagen del contenedor de Python

Tómate un momento para apreciar lo rápido que funciona y lo fácil que es usar casi cualquier versión de Python en cuestión de minutos (o incluso segundos). Este es el poder de los contenedores.

Usar casi cualquier versión de casi cualquier lenguaje de programación de código abierto de la última década puede ejecutarse en cuestión de minutos. ¿Recuerdas cuánto tardaste en instalar solo Python? ¿Y luego Node.js? ¿Y si necesitas usar Ruby? ¿O Java? ¿O Go? ¿O Rust? ¿O C++? ¿O C? ¿O PHP? ¿O Perl? O... ya me entiendes. Con Docker instalado, casi cualquier versión de casi cualquier lenguaje de programación de código abierto se puede instalar en minutos.

Con esta capacidad, también podemos ejecutar prácticamente cualquier versión de cualquier iteración de nuestras aplicaciones en cuestión de minutos (o incluso segundos). Este paradigma es lo que me entusiasma de los contenedores. Veámoslo en acción al contenerizar nuestra aplicación Python.

5.3.2 Creación de un Dockerfile de Python

Ahora que podemos ejecutar una versión específica de Python con Docker, comenzaremos el proceso de diseño de nuestra aplicación para nuestro Dockerfile específico.

Los pasos finales que seguiremos son casi idénticos a los que hicimos con Ansible en el capítulo 4, pero modificados para la sintaxis de Dockerfile. Aquí tienes un resumen de lo que debe hacer nuestro Dockerfile:

- Copia nuestro código de proyecto.
- Instale dependencias a nivel de sistema operativo como python3-venv, python3-dev y python3-pip.
- Crear un entorno virtual.
- Actualice la instalación pip predeterminada del entorno virtual.
- Instalar nuestras dependencias del proyecto en el entorno virtual.
- Ejecute nuestra aplicación.

Dos paquetes principales están ausentes a propósito: NGINX y Supervisor. Pronto veremos por qué estos paquetes ya no son necesarios.

Antes de ejecutar nuestro código, crearemos un Dockerfile casi completo para ver qué tenemos disponible por defecto. Esto significa que usaremos los siguientes comandos:

- Cambie nuestro directorio de trabajo a /app (esto quedará claro dentro del Dockerfile).
- Copie nuestra carpeta src/ al directorio /app.
- Ejecute apt-get update && apt-get install -y python3-venv python3-dev python3-pip. Esto garantizará que las dependencias del sistema operativo se instalen en nuestro contenedor.
- Agregue python3 -m venv /opt/venv. Almacenaremos nuestro entorno virtual en la ubicación /opt/ , que es un

Lugar recomendado para almacenar software complementario como entornos virtuales.

- Agregue `/opt/venv/python -m pip install --upgrade pip`. Es Siempre es una buena idea actualizar nuestro entorno virtual Instalación de pip antes de instalar nuestro proyecto requisitos.
- Correr `/opt/venv/python /` -metro `pip` `instalar` `-r`
`app/src/requirements.txt` `a` `instala nuestro proyecto` requisitos.
- Ejecute `/opt/venv/python -m http.server 8080`. Esto ejecutará un Servidor web Python integrado que representará un archivo `index.html`, si existe, o mostrar el contenido de cualquier carpeta dada; tendremos esta última.

Hemos utilizado cada uno de estos comandos unas cuantas veces.

A lo largo de este libro, con la excepción de `python -m http.server`. El objetivo ahora es no introducir demasiados nuevos comandos, pero simplemente replanteamos cómo usamos los comandos que estamos usando. empezando a familiarizarse con.

Con esto en mente, navega hasta la raíz de tu Camino a Proyecto Python de Kubernetes. Si has estado siguiendo... A lo largo del proyecto, se ubicará en `~/dev/roadtok8s/py`. En En este proyecto crearemos un Dockerfile con el contenido del siguiente listado.

Listado 5.6 Archivo Dockerfile de Python para navegar

```
# obtenemos nuestra imagen de
Python DESDE python:3.11.4

# crear y mover al directorio /app # todos los
comandos posteriores se ejecutarán en este directorio
WORKDIR /aplicación

# copia nuestra carpeta src local a /app en el contenedor COPY ./src/ /
app

# Ejecutar actualizaciones a nivel de sistema operativo
EJECUTAR apt-get update &&
    \ apt-get install -y python3-venv python3-dev python3-pip

# Crea nuestro entorno virtual de Python
EJECUTAR python3 -m venv /opt/venv

# Actualizar el entorno virtual pip
EJECUTAR /opt/venv/bin/python -m pip install --upgrade pip

# Instalar las dependencias de nuestro proyecto
# src/requirements.txt es nuestra versión local EJECUTAR /
opt/venv/bin/python -m pip install -r requirements.txt

# Ejecutar un servidor para exponer los archivos
copiados CMD ["/opt/venv/bin/python", "-m", "http.server", "8080"]
```

Ahora construyamos nuestro contenedor con los comandos de la siguiente lista.

Listado 5.7 Creación y ejecución del Dockerfile de Python

```
cd ~/dev/roadtok8s/py cat
Dockerfile # debería generar valores correctos

# ahora construye el contenedor docker
build -t rk8s-py -f Dockerfile . docker run -p 8080:8080 rk8s-      #1
py#2
```

#1 Utilice cualquier etiqueta que considere adecuada.

#2 El puerto 8080 está expuesto en el Dockerfile.

Con esto construido y funcionando, abramos nuestro navegador en <http://localhost:8080>, y deberíamos ver los mismos resultados que en la figura 5.8.

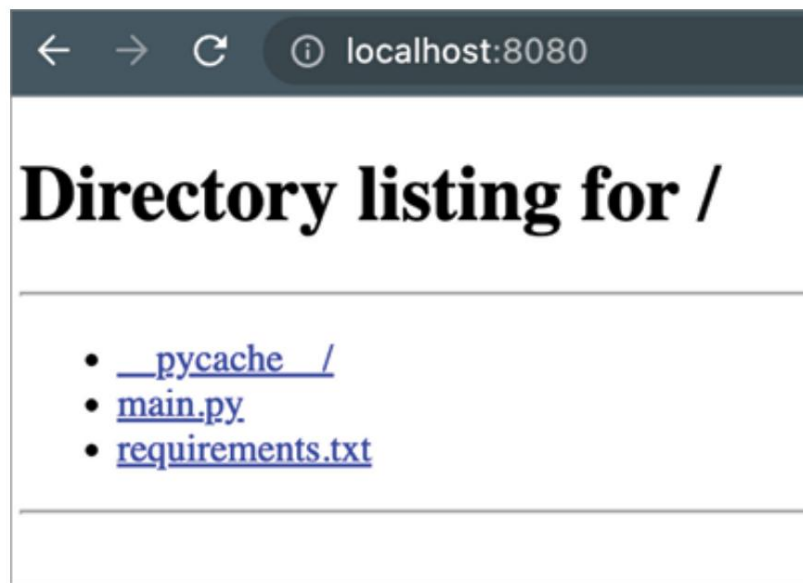


Figura 5.8. Exploración de la salida del Dockerfile de Python

Lo que vemos aquí son algunos datos importantes:

- `__pycache__`/ fue copiado.
- Nuestro directorio de trabajo contiene exactamente lo que está en nuestra carpeta `src/` local.

- El servidor `http.server` integrado de Python se puede utilizar para servir sitios web estáticos si lo necesitamos simplemente agregando un archivo `index.html` a nuestra carpeta `src/`.
- Dado que el servidor `http` se está ejecutando, como muchos desarrolladores de Python ya saben, podemos usar `gunicorn` en su lugar. Podemos, pero lo abordaremos pronto.

Antes de completar este `Dockerfile`, debemos solucionar el problema de los archivos adicionales que no queremos incluir. Aquí es donde resulta útil un archivo `.dockerignore`.

5.3.3 Ignorar archivos no deseados en compilaciones de Docker

Docker tiene dos comandos para agregar archivos o carpetas a una imagen de contenedor durante la fase de compilación: `COPY` y `ADD`. `COPY` es igual que `ADD`, excepto que `ADD` puede gestionar URL remotas para descargar archivos de internet.

Estos comandos copiarán todo en tu imagen de Docker, algo que rara vez deseamos. En su lugar, podemos usar un formato especial casi idéntico al archivo `.gitignore` de Git, llamado `.dockerignore`. Este archivo nos permitirá ignorar la copia de archivos y carpetas en nuestra imagen de contenedor.

En este punto, creemos un `.dockerignore` básico junto a nuestro `Dockerfile` que contiene una sola línea: `**/__pycache__` y veamos si esa carpeta todavía está incluida en nuestras compilaciones.

Después de compilar este archivo, debería ver que la carpeta `__pycache__` ya no existe. Por lo tanto, para ignorar archivos, puede usar un archivo `.dockerignore` teniendo en cuenta las siguientes reglas:

- Utilice rutas absolutas para archivos individuales.

- Utilice rutas relativas para carpetas antepuestas con `**/` para ignorar todos los archivos dentro de esa carpeta.
- Asegúrese de que las carpetas terminen con `/` para ignorar todos los archivos en la carpeta indicada.

Una vez que nuestra fase de compilación se traslade a GitHub Actions, probablemente no necesitemos tanto un archivo `.dockerignore`, ya que usaremos tanto Git como un archivo `.gitignore` para enviar el código. En otras palabras, los archivos y carpetas que normalmente querrías que Docker ignorara ya deberían estar ausentes. Dicho esto, siempre recomiendo usar un archivo `.dockerignore` para que la imagen final del contenedor compilada sea más ligera (por ejemplo, con menos archivos y carpetas), lo que reduce el tiempo de compilación y el de implementación, y la probabilidad de exponer un archivo que no debería exponer (por ejemplo, archivos `.env` o `.secret`). La siguiente lista muestra el resultado de nuestra imagen de contenedor después de agregar un archivo `.dockerignore`.

Listado 5.8 Ejemplo de `.dockerignore` de Python

```
# ignorar archivos en caché de Python **/  
__pycache__/*.[cod]  
  
# ignorar archivos env  
*.env
```

Ahora que tenemos un archivo `.dockerignore`, pasemos al paso final de nuestro Dockerfile: crear un script para iniciar nuestra aplicación.

5.3.4 Creación y ejecución de un script de punto de entrada

Ahora necesitamos ejecutar nuestra aplicación dentro del contenedor Docker. Quizás recuerdes que anteriormente usamos

Supervisor para ejecutar la aplicación. Usaremos básicamente el mismo comando, solo que almacenado en una ubicación diferente.

Antes de continuar, hablemos de la última línea de un Dockerfile: `CMD <su comando>`. Tenemos dos opciones: ejecutar la aplicación directamente o ejecutar un script que la ejecute. Para aplicaciones Python, prefiero usar un script para ejecutar la aplicación. Este script se denomina script de punto de entrada. En cualquier caso, la directiva `CMD` se ejecutará en tiempo de ejecución y no durante la fase de compilación del contenedor; más información al respecto próximamente.

Prefiero ejecutar un script de punto de entrada porque nos permite añadir lógica adicional a nuestra aplicación antes de ejecutarla. Por ejemplo, podríamos añadir un paso para comprobar si una base de datos está disponible antes de ejecutar la aplicación.

Un script de punto de entrada no debe confundirse con la directiva `ENTRYPOINT` en un Dockerfile. `CMD` se puede sustituir por `ENTRYPOINT`, pero por ahora nos quedaremos con `CMD` para simplificar; `ENTRYPOINT` es más adecuado para aplicaciones que no son web.

Nuestro script de punto de entrada se basará en las siguientes condiciones:

- `export RUNTIME_PORT=${PORT:-8080}`—Esto establecerá la variable de entorno `RUNTIME_PORT` en el valor de la variable de entorno `PORT` si existe o 8080 como alternativa.
- `/opt/venv/bin`: la ubicación de nuestros diversos comandos de entorno virtual.
- `/opt/venv/bin/gunicorn`—La ubicación de nuestro ejecutable gunicorn en función del entorno virtual y el paso `EJECUTAR... pip install` de nuestro Dockerfile.

- `--worker-class uvicorn.workers.UvicornWorker main:app -- bind "0.0.0.0:$RUNTIME_PORT`—Los argumentos de gunicorn para ejecutar nuestra aplicación. Este es el mismo comando que usamos en el capítulo 4, pero con algunos cambios: ya no necesitamos un ID de proceso; la
 - imagen de Docker en ejecución será nuestro proceso.
 - El puerto gunicorn ahora está vinculado a una variable de entorno, lo que ofrece flexibilidad fuera del contenedor en sí.

Como recordatorio, las variables de entorno son donde se almacena información confidencial del entorno de ejecución, como contraseñas y claves API. También se pueden usar para modificar la configuración del entorno de ejecución de la aplicación (p. ej., `PORT`), por lo que nuestro punto de entrada tiene una variable de entorno `PORT` como respaldo. Las variables de entorno confidenciales nunca deben almacenarse en el código base, añadirse a un `Dockerfile` ni incluirse en la compilación de un contenedor (p. ej., usar `.dockerignore` para ignorar archivos de variables de entorno comunes como `.env` o `.secrets`). La mayoría de los entornos de ejecución de contenedores, como Docker o Kubernetes, facilitan la inyección de variables de entorno al ejecutar el contenedor. Con esto en mente, crearemos un nuevo archivo en nuestra carpeta `src/` llamado `conf/entrypoint.sh` con el contenido que se muestra a continuación.

Listado 5.9 Script de punto de entrada de Python

```
#!/bin/bash export
PUERTO_DE_TIEMPO_DE_EJECUCIÓN=${PUERTO:-8080}

/opt/venv/bin/gunicorn \
    --worker-class uvicorn.workers.UvicornWorker \ main:app \ --bind

"0.0.0.0:$RUNTIME_PORT"
```

Una característica clave de la ubicación de este punto de entrada es que no se almacena en `src/`, sino en `conf/`, junto a `nginx.conf` y `supervisor.conf`. Prefiero conservar toda mi configuración.

archivos en una sola ubicación; esto no es diferente, pero también nos da la oportunidad de volver a utilizar el comando COPIAR para un archivo específico.

Este script de punto de entrada será el último comando que ejecutemos, lo que significa que el script debe tener los permisos correctos para ejecutarse (p. ej., `chmod +x`); de lo contrario, obtendremos un error de permisos después de compilar este contenedor al intentar ejecutarlo. La siguiente lista muestra las líneas actualizadas de nuestro Dockerfile.

Listado 5.10 Archivo Docker final de Python

```
...
COPIA ./src/ /app

# copia nuestro conf/entrypoint.sh local a /app en el contenedor COPY ./
conf/entrypoint.sh /app/entrypoint.sh
...
# Hacer que nuestro script de punto de entrada sea
ejecutable RUN chmod +x /app/entrypoint.sh
...
# Ejecutar nuestro script de punto de
entrada CMD ["/app/entrypoint.sh"]
```

Sigamos adelante y probemos nuestro nuevo Dockerfile compilándolo y ejecutándolo:

- Construyalo : `docker build -t rk8s-py -f Dockerfile`.
- Ejecútelo : `docker run -p 8080:8080 rk8s-py`.
- Ejecútelo con una nueva variable de entorno PORT : `docker run -p 8080:8081 -e PORT=8081 rk8s-py`. Observe que el puerto interno del contenedor (`-p <puerto-local>:<puerto-contenedor>`) coincide con el valor de la variable de entorno `port` declarada con `-e`.
PUERTO 80801.
- Abra otra terminal y ejecútela nuevamente con otro nuevo valor de PUERTO : `docker run -p 8081:8082 -e PORT=8082`

rk8s-py.

Dependiendo del alojamiento de producción elegido para nuestras imágenes de contenedor, la variable PORT suele estar configurada automáticamente, por lo que debemos asegurarnos de que nuestra aplicación sea lo suficientemente flexible para gestionarla. Ahora que hemos creado nuestra imagen de contenedor de Python, pasemos a nuestra imagen de contenedor de Node.js.

5.4 Contenerización de aplicaciones Node.js

No es sorprendente que el proceso para contenerizar una aplicación Node.js sea prácticamente idéntico al de una aplicación Python, por diseño. En nuestro caso, vamos a contenerizar nuestra aplicación Express.js, que tiene un entorno de ejecución estándar de Node.js. Es importante tener en cuenta que no todas las aplicaciones web de Node.js utilizan dicho entorno como servidor web. Por ejemplo, las aplicaciones renderizadas del lado del servidor (SSR) no siempre utilizan dicho entorno como servidor web (p. ej., React.js puede ser una aplicación SSR). El proceso es el siguiente:

1. Busque la etiqueta de versión de Node.js que necesitamos en Node.js Docker repositorio de imágenes en Hub (http://hub.docker.com/_/node).
2. Utilice la directiva FROM para utilizar esta imagen y etiqueta de versión.
3. COPIA nuestro código en nuestro WORKDIR.
4. EJECUTE el comando npm install para actualizar npm e instalar las dependencias de nuestro proyecto package.json . (Le recordamos que existen otros gestores de paquetes para Node.js, pero en este libro nos centraremos en npm).
5. Escriba un script de punto de entrada para administrar el tiempo de ejecución de nuestra aplicación y el PUERTO predeterminado.

6. Cree la directiva CMD para ejecutar nuestro script de punto de entrada.

Para encontrar la versión que necesitamos, necesitaremos realizar los siguientes pasos:

1. Visite el repositorio oficial de contenedores de Node en https://hub.docker.com/_/node.
2. Haga clic en la pestaña Etiquetas (o navegue directamente a https://hub.docker.com/_/node/tags).
3. Filtra los resultados por 18.17.0 (o cualquier versión que desees).
4. Ordenar los resultados por AZ.

La figura 5.9 muestra los resultados de estos pasos.

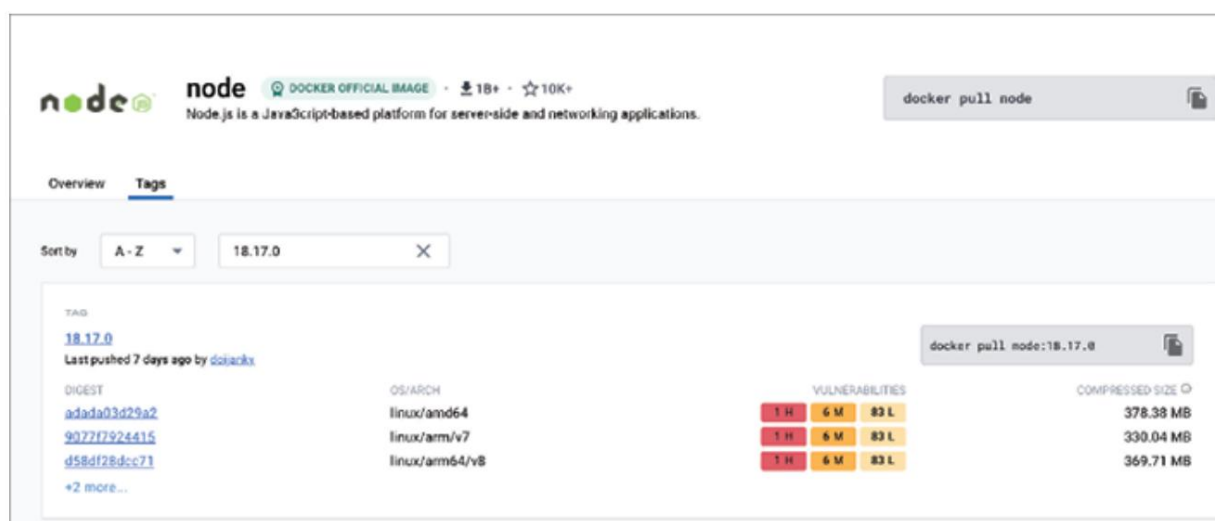


Figura 5.9 Etiquetas Node.js de Docker Hub

La Figura 5.10 muestra una captura de pantalla de la dirección del comando desde el repositorio de Node.js para extraer la imagen y la etiqueta específica de Docker Hub. Es importante saber qué buscar para asegurarse de usar la etiqueta correcta para cada imagen de contenedor.

```
docker pull node:18.17.0
```



Figura 5.10 Comando de extracción del nodo Docker Hub

Una vez más, `docker pull node:18.17.0` nos muestra la etiqueta de contenedor exacta que necesitamos usar para esta imagen.

Ahora que tenemos la imagen del contenedor y la etiqueta de imagen que necesitamos, creemos el script de punto de entrada que necesitamos antes de crear nuestro Dockerfile.

5.4.1 El script de punto de entrada de Node.js y el archivo `.dockerignore`

Como recordarán de capítulos anteriores, ejecutamos nuestra aplicación Express.js directamente con Node.js mediante el comando `node main.js`. En nuestro archivo `main.js`, también tenemos el código para un valor de puerto basado en una variable de entorno (p. ej., `const port = process.env.PORT || 3000;`). Esto significa que nuestro script de punto de entrada será bastante simple:

- Determinar la ubicación de `main.js`.
- Ejecute `node path/to/main.js`. Al igual que con la aplicación Python, usaremos la carpeta `/app`. A diferencia del proceso Python, también usaremos la carpeta `/src` para mantener las rutas relativas.

La siguiente lista muestra el contenido de nuestro script de punto de entrada ubicado en `conf/entrypoint.sh` en nuestra aplicación Node.js (ubicado en `~/Dev/roadtok8s/js` si ha estado siguiendo).

Listado 5.11 Script de punto de entrada de Node.js

```
#!/bin/bash  
  
nodo /app/src/main.js
```

Como podemos ver, el script de punto de entrada de Node.js es increíblemente simple pero también muy consistente con la forma en que contenedorizamos nuestra aplicación Python.

Nuestro último paso previo al Dockerfile es crear un archivo `.dockerignore` que incluya todos los archivos y carpetas que queremos ignorar. La siguiente lista muestra el contenido de nuestro archivo `.dockerignore`, ubicado en `~/Dev/roadtok8s/js/.dockerignore`.

Listado 5.12 Node.js Dockerignore

```
# ignorar módulos de nodo **/  
node_modules/  
  
# ignorar archivos env  
*.env
```

Ahora que tenemos todo el código que necesitamos, creemos nuestro Dockerfile.

5.4.2 El Dockerfile de Node.js

Como primer paso para crear otro Dockerfile, reexaminemos las distintas acciones que necesitaremos realizar para construir nuestro contenedor:

- Crea un directorio de trabajo (WORKDIR) para nuestro proyecto; usaremos el comúnmente usado `/app`.
- Copiar todo el código de `src/` a `app/src/`. Con nuestra aplicación Node.js, queremos mantener las rutas relativas a la raíz de nuestro proyecto debido a que `node_modules/` será...

Instalado en la raíz de nuestro proyecto. Mover `node_modules` no es tan sencillo como mover un entorno virtual de Python, por lo que no se recomienda.

- Copie los archivos `package.json` y `package-lock.json`. `package-lock.json` ayudará a garantizar compilaciones consistentes desde el entorno local hasta el contenedor.
- Copie `conf/entrypoint.sh` en `app/entrypoint.sh`.
- Ejecute `npm install` para instalar las dependencias de nuestro proyecto.
- Ejecute `chmod +x entrypoint.sh` para garantizar que nuestro script de punto de entrada pueda ejecutarse.
- Escriba la directiva `CMD` para ejecutar nuestro script de punto de entrada.

Todos estos pasos deberían resultar repetitivos por una buena razón: estamos repitiendo prácticamente el mismo proceso. Conseguir un entorno consistente a menudo implica repetirnos constantemente. La siguiente lista muestra el `Dockerfile` final de nuestra aplicación Node.js.

Listado 5.13 Dockerfile de Node.js

```
# Usando la imagen oficial de Node.js para la versión LTS DESDE node:18.17.0

# Establezca el directorio de trabajo de nuestra aplicación
WORKDIR /aplicación

# copiar el código fuente principal COPY ./src /
app/src/

# copia package.json y package-lock.json para # asegurarnos de que
usamos las mismas dependencias COPY package.json /
app/ COPY package-lock.json /app/

# copia el entrypoint.sh para ejecutar la aplicación COPY conf/
entrypoint.sh /app/

# instalar dependencias
EJECUTAR npm install

# hacer que entrypoint.sh sea ejecutable RUN chmod
+x entrypoint.sh

# ejecuta nuestra
aplicación CMD ["/entrypoint.sh"]
```

Ahora que tenemos este Dockerfile, ejecutémoslo y construyámoslo con los siguientes pasos:

1. Construya la imagen con `docker build -t rk8s-js -f Archivo Docker`.
2. Ejecute la imagen con `docker run -p 3000:3000 rk8s-js`. Si recuerda, configuramos el puerto predeterminado en nuestra aplicación Express.js en 3000.
3. Pruebe un nuevo puerto: `docker run -p 3001:3000 -e PORT=3001 rk8s-js`.

4. Agregue una nueva variable de entorno: `docker run -p 3002:3000 -e MENSAJE="Hola Mundo" rk8s-js`.

El resultado de estos comandos será casi idéntico al del proceso de contenedorización de Python, así que deje que lo verifique. Ahora que sabemos cómo extraer, crear y ejecutar imágenes de contenedor, aprendamos a subirlas a Docker Hub con el comando `push` integrado.

5.5 Enviar imágenes de contenedores a Docker Hub

Antes de empezar a distribuir las imágenes de nuestros contenedores, necesitamos alojarlas en un registro de Docker. En el resto del libro, usaremos Docker Hub como nuestro registro predilecto para simplificar al máximo las cosas.

Los registros de Docker almacenan y alojan imágenes de contenedores, de forma similar a como los repositorios de Git almacenan y alojan código. Los registros de Docker son de código abierto, por lo que podría crear su propio registro para alojar sus contenedores, pero hacerlo es mucho más complejo que lo que explicamos en este libro. Docker Hub es el registro oficial de Docker y es el que la herramienta de línea de comandos de Docker utiliza por defecto. Estos son los pasos para usar Docker Hub:

1. Crea una cuenta en <http://hub.docker.com> y consigue un nombre de usuario; el mío es `codingforentrepreneurs`.
2. Inicie sesión en Docker Hub desde la línea de comandos con `docker` acceso.
3. Los pasos para construir y empujar, que cubriremos más detalladamente en un momento:
 - Construye `docker build -t <tu-nombre-de-usuario>/<tu-nombre-de-contenedor>:<tu-etiqueta> -f <tu-archivo-docker>`.

- Push `docker push <su-nombre-de-usuario>/<su-nombre-de-contenedor>: <su-etiqueta>`.
- Si realiza un envío a su cuenta de Docker Hub, se creará un nuevo registro de contenedor público por defecto. También puede crear uno manualmente para elegir el estado del contenedor: público o privado.

Con esto en mente, procedamos a un proceso de compilación a envío para nuestra aplicación Python.

5.5.1 Su primer envío a Docker Hub

Subir a Docker Hub es casi tan sencillo como compilar en una máquina local. La siguiente lista ofrece una visión completa de cómo navegar, compilar y subir nuestro contenedor a Docker Hub.

Listado 5.14 Envío de nuestra imagen de contenedor de Python a Docker Hub

```
cd ~/dev/roadtok8s/py

# si aún no lo has hecho # inicia
sesión en Docker Hub inicio
de sesión en Docker

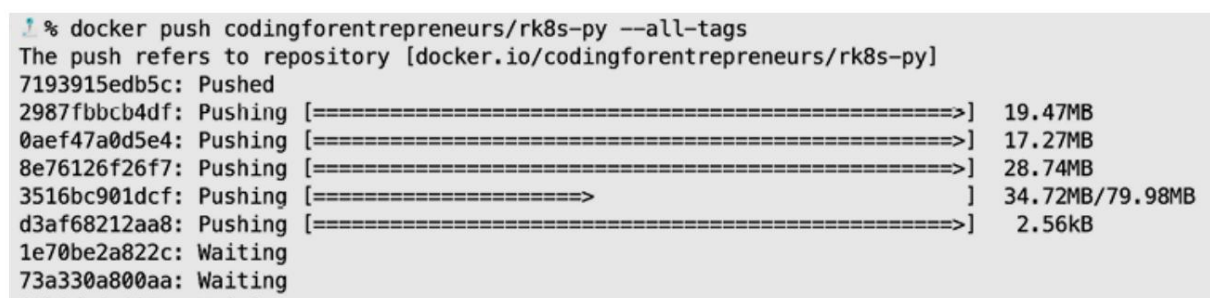
# compila y etiqueta dos
veces docker build -f Dockerfile \ -t
    codingforentrepreneurs/rk8s-py:0.0.1 \ -t
    codingforentrepreneurs/rk8s-py:latest \
    .

# empujar una etiqueta
especifica docker push codingforentrepreneurs/rk8s-py:0.0.1

# enviar todas las versiones
etiquetadas docker push codingforentrepreneurs/rk8s-py --all-tags
```

Antes de revisar el resultado, quiero señalar que compilé la imagen una vez, pero la etiqueté dos veces con la opción `-t`. Esto se debe a una función estándar de Docker. Si recuerdan, cuando ejecutamos `docker run nginx`, en realidad estábamos ejecutando `docker run nginx:latest`; la etiqueta `latest` se usa por defecto si no se especifica ninguna. Etiquetar la compilación de mi contenedor con la etiqueta `latest` y una versión específica me da la flexibilidad de usar cualquiera de las dos etiquetas mientras almaceno iteraciones de cualquier versión.

Este escenario pretende destacar la naturaleza del control de versiones al etiquetar, compilar y enviar imágenes a Docker Hub. La Figura 5.11 muestra el resultado de nuestro comando `push`.



```
% docker push codingforentrepreneurs/rk8s-py --all-tags
The push refers to repository [docker.io/codingforentrepreneurs/rk8s-py]
7193915edb5c: Pushed
2987fbbcb4df: Pushing [=====>] 19.47MB
0aef47a0d5e4: Pushing [=====>] 17.27MB
8e76126f26f7: Pushing [=====>] 28.74MB
3516bc901dcf: Pushing [=====>] 34.72MB/79.98MB
d3af68212aa8: Pushing [=====>] 2.56kB
1e70be2a822c: Waiting
73a330a800aa: Waiting
```

Figura 5.11. Subiendo nuestra imagen de contenedor de Python a Docker Hub

Esta salida muestra todas las capas que componen el contenedor compilado que se envía a Docker Hub. Estas mismas capas deberían ser similares a los comandos `docker run` o `docker pull` cuando usamos una imagen que no está en nuestro equipo local.

Las capas que vemos en la figura 5.11 corresponden aproximadamente a cada instrucción (por ejemplo, `FROM`, `COPY`, `RUN`, etc.) de nuestro Dockerfile. Cada instrucción se almacena en caché y se reutiliza al crear nuestra imagen de contenedor. Este almacenamiento en caché también explica por qué nuestra primera compilación puede tardar un poco, pero las posteriores se aceleran. Por eso, los registros de Docker pueden alojar imágenes de contenedor masivas sin ocupar mucho espacio ni tardar mucho en cargarse o descargarse. Con la compilación completada y enviada a Docker Hub, revisemos nuestro repositorio en Docker Hub. La Figura 5.12 muestra la página del repositorio de nuestra imagen de contenedor de Python.

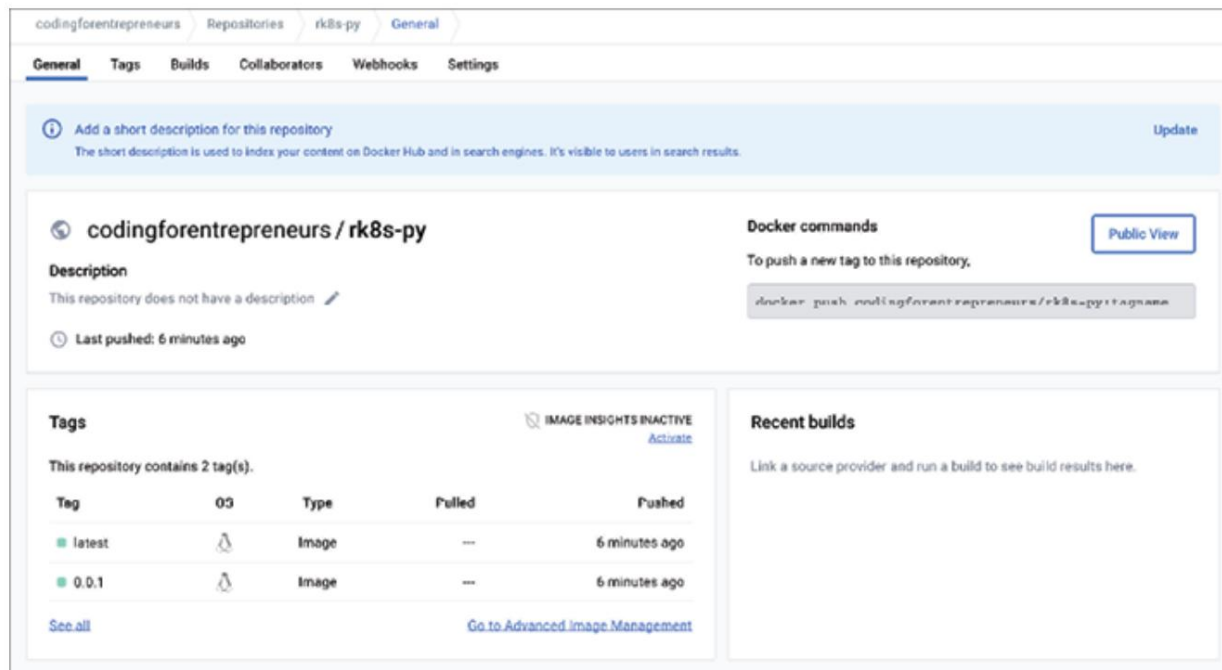


Figura 5.12 Imagen del contenedor Python de Docker Hub rk8s

Esta página nos muestra algunos datos importantes:

- Nuestra imagen de contenedor es pública por defecto (esto es fácil de cambiar). Si quisiéramos que fuera privada antes de la primera publicación, primero crearíamos el repositorio.
- Podemos ver las dos etiquetas que enviamos a Docker Hub (por ejemplo, latest y 0.0.1).
- Si vamos a la pestaña Etiquetas, podemos ver el tamaño del contenedor (no aparece en la imagen, pero es de unos 400 MB).

Si trabajas con Apple Silicon o una Raspberry Pi basada en ARM, necesitarás añadir una bandera adicional para tener en cuenta la plataforma. Este proceso no supone un problema hasta que lo sea, así que asegurémonos de que no lo sea. Si no utilizas estas plataformas, puedes omitir la siguiente sección.

LA PLATAFORMA DE COMPILACIÓN DOCKER

Si utiliza Apple Silicon (por ejemplo, chip M1, chip M2) o Raspberry Pi, eso significa que la arquitectura de su sistema es ARM, mientras que la mayoría de los servidores Linux basados en la nube tienen una arquitectura x86.

Cuando ejecuta Docker Builder, se compilará de forma nativa para la arquitectura del sistema que está utilizando, lo que puede causar problemas cuando intenta ejecutar este contenedor en otras plataformas (por ejemplo, una máquina virtual Linux).

La solución es simple; solo necesitas incluir `--platform=linux/amd64` en tu comando de compilación, como por ejemplo usando `docker build -f Dockerfile --platform=linux/amd64 <your-username>/<your-repo>:<your-tag>`.

Hay otra herramienta Docker más nueva llamada `buildx`, que permite compilaciones multiplataforma y multiarquitectura; esto está fuera del alcance de este libro porque es un poco más complejo de lo que necesitamos.

En lugar de usar compilaciones multiplataforma, compilaremos nuestras imágenes de contenedor en GitHub Actions. Esto nos permitirá compilarlas en una máquina virtual Linux y subirlas a Docker Hub. Esto también nos permitirá compilarlas en cada confirmación de nuestra rama principal, lo que garantizará que estén siempre actualizadas.

En este punto, hemos hecho todo lo necesario para crear y ejecutar imágenes de contenedor en nuestra máquina local. Es recomendable seguir experimentando con estos contenedores locales antes de empezar a automatizar el proceso de creación y envío a Docker Hub. Quiero...

Enfatiza que Docker es mucho más que lo que cubrimos aquí, pero comprender este capítulo será crucial para implementar una aplicación en contenedores en producción.

Resumen

- Los contenedores son mini-máquinas Linux y tenemos mucha flexibilidad en cómo podemos usarlos.
- Los contenedores permiten la portabilidad de casi cualquier tipo de aplicación, especialmente las de código abierto.
- El patrón de instrucciones para contenedores utiliza un método lógico de copiar e instalar el software que cualquier aplicación pueda necesitar.
- Los contenedores se almacenan en un registro de contenedores como Docker Hub para que luego puedan distribuirse a los sistemas de producción.
- La construcción de contenedores es esencialmente idéntica independientemente de la aplicación, excepto por lo que va en el Dockerfile.
- El contenedor es responsable de ejecutar la aplicación o aplicaciones en contenedores a través de lo declarado en el Dockerfile (o como un argumento adicional).
- Los contenedores imponen coherencia en el modo en que empaquetamos nuestra aplicación final.
- Una aplicación en contenedores puede ejecutarse de manera diferente según las variables de entorno que se pasen en el tiempo de ejecución.

6 contenedores en acción

Este capítulo cubre

- Uso de GitHub Actions para crear y enviar contenedores
- Adjuntar volúmenes de almacenamiento y mantener el estado
- Uso de variables de entorno basadas en archivos
- Ejecutar y administrar múltiples contenedores con Docker Compose
- Aprender los conceptos básicos de comunicación y redes entre contenedores

Imagina estar entre el público y CodeOprah dice: "¡Un contenedor, un contenedor, un contenedor, todos reciben un contenedor!". La multitud enloquece. Pero tú no, al menos todavía. Estás ahí, haciéndote algunas preguntas fundamentales: ¿Cómo ejecuto una base de datos dentro de un contenedor? ¿Cómo se comunican mis dos contenedores? ¿Cómo gestiono varios contenedores a la vez? Antes de empezar a repartir contenedores como si fueran coches gratis, necesitamos configurar nuestra canalización de CI/CD de extremo a extremo para que GitHub Actions se encargue de compilar y enviar nuestros contenedores a Docker Hub.

Una vez automatizado nuestro proceso de compilación, necesitamos comprender cómo inyectar estado en los contenedores mediante volúmenes. Los volúmenes permiten conservar datos directamente en un contenedor, independientemente de su estado actual. Pueden existir incluso si el contenedor no se está ejecutando o está completamente destruido. El uso de volúmenes es la forma de obtener contenedores con estado, o lo contrario de los sin estado.

Los volúmenes no son la única forma de habilitar el estado en los contenedores. Las variables de entorno pueden tener claves API o credenciales secretas para servicios de bases de datos externos, lo que permite una forma de estado de la aplicación.

Adjuntar volúmenes y usar variables de entorno con el comando docker puede ser fácil de usar, pero difícil de recordar y administrar. Veamos un par de ejemplos:

- `estibador` `correr` `-v`
`ruta/al/volumen/local:/ruta/al/volumen/del/contenedor ENV_VAR=VALOR -mi`
`--env-file ./ruta/al/env/.env -p 8000:8000 -d mi-imagen-del-contenedor-a`
- `docker run -v ruta/al/vol/local-b:/ruta/al/contenedor/vol-b -e ENV_ VAR=VALOR -e`
`PUERTO=9090 -e DEBUG=1 --env-file ./ruta/al/env/.dev-file -p 8000:8000 -d mi-imagen-del-`
`contenedor-b`

¿Te imaginas intentar ejecutar esto para más de un proyecto? Será un auténtico desastre. Descubre Docker Compose. Con Docker Compose, definimos todas las opciones necesarias para el entorno de ejecución de contenedores en un archivo YAML y luego usamos un simple comando para ejecutarlo todo. Docker Compose puede compilar, ejecutar y desactivar contenedores fácilmente; adjuntar volúmenes; inyectar variables de entorno; y mucho más. Docker Compose es la primera y más sencilla herramienta de orquestación de contenedores que usaremos. Trabajar con Docker Compose es un paso importante en nuestro camino hacia el uso de Kubernetes para la orquestación de contenedores. Comencemos a automatizar las compilaciones y los envíos de contenedores con GitHub Actions.

6.1 Creación y envío de contenedores con GitHub Actions

La creación de imágenes de contenedores para nuestras aplicaciones debe ser lo más automatizada posible; tenemos varias opciones para lograrlo. Primero, podríamos usar Docker Hub para crear imágenes automáticamente mediante una integración con GitHub, o podríamos usar GitHub Actions directamente para crear nuestras imágenes.

Si bien es excelente que Docker Hub pueda gestionar nuestras compilaciones automatizadas, esto causa un problema clave que queremos evitar: otra ubicación donde almacenar secretos, como claves o contraseñas. Después de compilar un contenedor, queremos poder actualizar los sistemas que usan estas imágenes de contenedor.

Si bien esto es posible con Docker Hub, es mucho más desafiante que simplemente usar GitHub Actions o la canalización CI/CD junto a donde se encuentra nuestro código.

El flujo de trabajo para GitHub Actions nos presentará una nueva función que aún no hemos visto: usar una GitHub Action de terceros. Docker cuenta con una GitHub Action oficialmente compatible que ayuda a garantizar que la herramienta de línea de comandos de Docker esté disponible automáticamente en nuestros flujos de trabajo de GitHub Actions. La herramienta se llama `setup-buildx-action` y es muy fácil de implementar (consulta la documentación de GitHub en <https://github.com/docker/setup-buildx-action>). Antes de continuar, veamos qué son las GitHub Actions de terceros.

6.1.1 Inicio de sesión en Docker y herramientas de acción de GitHub de terceros

GitHub Actions es un término que se refiere tanto a los flujos de trabajo de la canalización de CI/CD como a lo que yo llamo plugins. Ya hemos visto estos plugins en flujos de trabajo anteriores con `actions/checkout` y `actions/setup-python`. Estos son plugins propios de GitHub Actions, ya que GitHub los mantiene. También puedes usar muchos plugins de terceros, que implementaremos en esta sección.

Tenga en cuenta que no es obligatorio usar complementos de GitHub Action de terceros, pero puede ayudar a reducir el tiempo de ejecución de cualquier flujo de trabajo. Estos complementos también conllevan un riesgo inherente, ya que pueden exponer sus flujos de trabajo a terceros no confiables.

Ahora usaremos el complemento de GitHub Actions, gestionado por Docker y llamado login_action (consulta <https://github.com/marketplace/actions/docker-login>). Este complemento simplifica el proceso de inicio de sesión y nos permite usar secretos de GitHub Actions para ocultar nuestro nombre de usuario de Docker Hub y la contraseña de nuestro token de acceso personal.

Antes de poder usar este complemento, debemos crear un token de acceso personal en Docker Hub y agregarlo a nuestros secretos de GitHub

Actions. Aquí está el proceso paso a paso:

1. Inicie sesión en Docker Hub en <http://hub.docker.com/login>.
2. Vaya a Seguridad en <http://hub.docker.com/settings/security> o de la <http://hub.docker.com/account> Cuenta.
 - Haz clic en el icono de tu perfil en la parte superior derecha.
 - Haga clic en Configuración de la cuenta en el menú desplegable.
 - Haga clic en Seguridad en el lado izquierdo.
3. Haga clic en Nuevo token de acceso:
 - Dale un nombre a tu token (por ejemplo, Acciones de GitHub de RK8).
 - Seleccione Leer y escribir para el alcance (no necesitamos permiso de eliminación).
 - Haga clic en Crear.
 - Copie el token para agregarlo como secreto de Acciones de GitHub para Python y JavaScript.
4. Navega a cada uno de tus repositorios de GitHub Action (los míos son <https://github.com/jmitchel3/roadtok8s-py> y <https://github.com/jmitchel3/roadtok8s-js>): Navega a Configuración en la parte superior derecha.
 -

- Vaya a Secretos y variables y haga clic en Acciones en el lado izquierdo.
- Haga clic en Nuevo secreto de repositorio.
- Agregue DOCKERHUB_USERNAME como nombre, con su nombre de usuario como secreto.
- Agregue DOCKERHUB_PASSWORD como nombre, con el token recién creado como secreto.
- Añade DOCKERHUB_REPO como nombre, con el valor del nombre del repositorio que quieras usar. En mi caso, usaré rk8s-py y rk8s-js como valores.

Estos pasos nos permiten, primero, iniciar sesión en Docker Hub de forma segura y, segundo, etiquetar nuestras imágenes según los secretos (es decir, sin que la etiqueta se exponga) en un flujo de trabajo de GitHub Actions usando `docker build -t ${{ secrets.DOCKERHUB_USERNAME }}/${{ secrets.DOCKERHUB_REPO }}:0.0.1` o la etiqueta que elijamos. Ahora, construyamos el flujo de trabajo para ambas aplicaciones (es prácticamente el mismo para ambas).

6.1.2 Un flujo de trabajo de GitHub Actions para crear contenedores

Los comandos de compilación y envío de Docker son básicamente los mismos para todos los proyectos, con variaciones relacionadas con la forma en que etiquetamos la imagen del contenedor. Esto significa que nuestro archivo de flujo de trabajo de GitHub Action puede reutilizarse una y otra vez. Usaremos las siguientes etiquetas para etiquetar nuestras imágenes:

- `${{ secrets.DOCKERHUB_USERNAME }}/${{ secrets.DOCKERHUB_REPO }}:latest` (como predeterminado) `{{{ }}/$`
- `{{{ secrets.DOCKERHUB_USERNAME }}/${{ secrets.DOCKERHUB_REPO }}:${{ github.sha }}`. `github.sha` es un secreto integrado que se relaciona con el SHA de la confirmación específica sobre el cual puede leer en la documentación.

La siguiente lista muestra exactamente lo sencillo que puede ser contener una aplicación de esta manera.

Listado 6.1 Creación de contenedores con GitHub Actions

```

nombre: Construir y enviar contenedor
en:
  flujo de trabajo_despacho:

trabajos:
  build-push: se
    ejecuta en: ubuntu-latest
    entorno:
      DH_USER: ${{ secrets.DOCKERHUB_USERNAME }}
      REPO: ${{ secrets.DOCKERHUB_REPO }} pasos: -
    usos:
      acciones/checkout@v3 - usos: docker/login-
      action@v2
    con:
      nombre de usuario: ${{ secrets.DOCKERHUB_USERNAME }} contraseña:
        ${{ secrets.DOCKERHUB_TOKEN }} - nombre: Construya el
      contenedor
    ejecutar:
      | docker build -f Dockerfile \
        -t "$DH_USER/$REPO:último" \ -t
        "$DH_USER/$REPO:${{ github.sha }}" \
        .
      - nombre: Empujar las imágenes del contenedor
    ejecutar:
      | docker push "$DH_USER/$REPO" --all-tags
  
```

Para facilitar la lectura, utilizamos las variables env para DH_USER y REPO para representar nuestro secreto de almacenamiento para USER y REPO, respectivamente. Esto significa que "\$DH_USER/\$REPO" se traduce a \${{ secrets.DOCKERHUB_USERNAME }}/\${{ secrets.DOCKERHUB_REPO }}. y codingforentrepreneurs/rk8s-js según el repositorio.

Confirma este archivo en tu repositorio de Git, súbelo a GitHub y Pruébalo para ver si puedes compilar ambas aplicaciones correctamente. Si encuentras errores, asegúrate de usar el repositorio de Python de referencia (<https://github.com/jmitchel3/roadtok8s-py>), y el repositorio de Node.js (<https://github.com/jmitchel3/roadtok8s-js>) para este libro. para comparar tu código con el código funcional. Ahora que tenemos nuestras imágenes de contenedor compiladas y alojadas en Docker Hub, comencemos a aprender sobre otra gran función de Docker: Docker Compose.

6.2 Administrar un contenedor con Docker Compose

Docker Compose es nuestro primer paso hacia la orquestación de nuestros contenedores. Para mí, orquestar significa simplemente garantizar que nuestros contenedores funcionen exactamente como queremos. Esto puede incluir la compilación, diferentes variables de entorno y el repositorio del contenedor. A medida que mejoremos la orquestación, aprenderemos que también implica garantizar el estado del contenedor. Por ahora, simplificaremos nuestra orquestación: compilar y ejecutar contenedores.

Estos son dos de los comandos Docker más simples que debes memorizar:

- `docker build -f Dockerfile -t su-etiqueta .`
- `docker run -p 4321:4321 -e PORT=4321 su-etiqueta`

Si tiene problemas con estos dos comandos, revise el capítulo 5. Si bien es importante memorizar su funcionamiento, no es necesario memorizar sus parámetros. En otras palabras, cada proyecto...

Es diferente, y probablemente tengas parámetros distintos para cada uno. Aquí es donde entra Docker Compose.

Docker Compose nos ayudará a recordar los parámetros necesarios para compilar y ejecutar nuestros contenedores correctamente. Incluso si nunca usa Docker Compose directamente, el archivo de configuración de Docker Compose, conocido como `compose.yaml` (también conocido como `docker-compose.yaml`), puede ser un excelente lugar para almacenar todos los parámetros necesarios para compilar y ejecutar sus contenedores.

6.2.1 Por qué necesitamos Docker Compose

¿Recuerdas en qué puerto quieres que se ejecute tu contenedor de Python? ¿Y el de Node.js? ¿Es el 4321? ¿Otro? ¿Y las variables de entorno? ¿Recuerdas cuáles necesitas configurar?

El uso de Docker implica abstraer una serie de elementos de tiempo de ejecución que necesitará conocer para responder estas preguntas.

También notarás que tus comandos de ejecución de Docker son cada vez más largos, por lo que esa podría ser la configuración que necesitas para ejecutar tu aplicación. Intenta hacer lo mismo con otros tres tipos de contenedores y te resultará un fastidio recordarlo.

```
docker run -v ruta/al/vol/local-b:/ruta/al/contenedor/vol-b -e ENV_  
  
    VAR=VALOR -e PUERTO=9090 -e DEBUG=1 -env-file ./ruta/al/env/.dev-file  
-pag  
    8000:8000 -d mi-imagen-de-contenedor-b
```

Presentamos Docker Compose. Docker Compose puede automatizar todos los comandos necesarios para compilar y ejecutar nuestros contenedores con un solo comando: `docker compose up --build`. Antes de explorar los comandos para usar Docker Compose, repasemos un...

Archivo de configuración de ejemplo conocido como `compose.yaml` o `docker-compose.yaml` (ambos nombres son compatibles). La siguiente lista es un ejemplo que podría usarse con una aplicación web que ya tenga un `Dockerfile`.

Listado 6.2 Su primer archivo de Docker Compose

```
servicios:
  web:
    construir: .
    puertos:
      - "3001:3001"
```

Este código se traduce a

```
docker build --tag <nombre-directorio-carpeta>-web .
docker run --publish 3001:3001 <nombre-directorio-carpeta>-web
```

¿Te suenan? Espero que sí. Este archivo `compose.yaml` es solo una pequeña parte, e incluso le falta la declaración que reemplaza el argumento `-f Dockerfile` que usamos antes. El objetivo de ver esto ahora es darnos una idea de lo que Docker Compose puede hacer por nosotros. Ahora que conocemos Docker Compose, vamos a ponerlo en práctica con nuestra aplicación Python.

6.2.2 Docker Compose para nuestra aplicación Python

Nuestra aplicación Python está más que lista para usar Docker Compose para compilar nuestro contenedor, ya que contamos con una aplicación funcional, un `Dockerfile` y variables de entorno. Para compilar y ejecutar este contenedor, necesitamos hacer lo siguiente:

1. Navegue a la raíz de nuestro proyecto con `cd ~/dev/roadtok8s/py`.

2. Ejecute nuestro comando de compilación con `docker build -f Dockerfile -t rk8s-py .`
3. Ejecute nuestro contenedor con `docker run -e PORT=8080 -p 8080:8080 rk8s-py`

Este proceso es prácticamente idéntico a Docker Compose, con la ausencia de un aspecto clave: los argumentos de tiempo de ejecución. En esta etapa del uso de Docker Compose, nos centraremos en eliminar todos los argumentos de tiempo de ejecución. Para ello, primero comprendamos los elementos clave de una configuración para Docker Compose:

- **Servicios:** este bloque definirá cualquier aplicación en contenedores que queramos administrar con Docker Compose.
Se llaman servicios porque Docker Compose puede administrar múltiples contenedores a la vez y al mismo tiempo proporcionar una red entre ellos.
- **Web:** Asignaremos a nuestra primera aplicación contenedorizada el nombre de servicio "web". El nombre del servicio es ligeramente diferente al nombre de la imagen y el nombre de la etiqueta, pero no por mucho. Los nombres de los servicios se aclararán a medida que avancemos en este capítulo.
build:context: La ruta a la ubicación desde la
- que queremos ejecutar nuestro comando de compilación. Podríamos usar un directorio diferente si fuera necesario o usar el directorio actual (por ejemplo, el que está junto a `compose.yaml`). **build:dockerfile:** La ruta al Dockerfile que queremos usar en relación con la ruta
- **build:context. ports:** El mismo que el argumento `-p` que usamos en nuestro comando `docker run`.
-
- **entorno:** el mismo que el argumento `-e` que usamos en nuestro comando `docker run`.

Docker Compose tiene muchas más opciones de configuración disponibles, pero estas se encuentran entre las más comunes.

y, en nuestro caso, los únicos necesarios para nuestra aplicación Python.

Navegue a la raíz de su proyecto Python (`~/dev/roadtok8s/py`) y cree `compose.yaml` con el contenido de la siguiente lista.

Listado 6.3 Docker Compose para nuestra aplicación Python

```
servicios:
  web:
    compilación:
      contexto: .
      dockerfile: Dockerfile
    Puertos:
      - Entorno
    "8080:8080":
      - PUERTO=8080
```

Para construir, ejecutar y detener nuestro contenedor, usaremos los siguientes comandos:

- `docker compose build` construirá nuestro contenedor.
- `docker compose up` ejecutará nuestro contenedor.
- `docker compose down` detendrá nuestro contenedor.

Pruebe estos comandos. La Figura 6.1 muestra un fragmento del comando de compilación (`docker compose build`).

```
% docker compose build
[+] Building 6.9s (16/16) FINISHED
=> [web internal] load .dockerignore                                0.0s
=> => transferring context: 92B                                     0.0s
=> [web internal] load build definition from Dockerfile            0.0s
=> => transferring dockerfile: 798B                                 0.0s
=> [web internal] load metadata for docker.io/library/python:3.11.4 2.1s
=> [web auth] library/python:pull token for registry-1.docker.io  0.0s
=> [web internal] load build context                               0.0s
```

Figura 6.1 Construcción de un contenedor de Python con Docker Compose

La figura 6.2 muestra un recorte del comando de ejecución (docker compose up).

```
% docker compose up
[+] Running 2/0
✓ Network roadtok8s-py_default Cre... 0.0s
✓ Container roadtok8s-py-web-1 Cre... 0.0s
Attaching to roadtok8s-py-web-1
roadtok8s-py-web-1 | [2023-08-04 00:11:49 +0000] [7] [INFO] Starting gunico
rn 21.2.0
roadtok8s-py-web-1 | [2023-08-04 00:11:49 +0000] [7] [INFO] Listening at: h
ttp://0.0.0.0:8080 (7)
```

Figura 6.2 Ejecución de un contenedor de Python con Docker Compose

Con Docker Compose en ejecución (después de usar docker compose up), abra una nueva ventana de terminal, navegue hasta su proyecto y ejecute docker compose down para detener el contenedor.

La figura 6.4 muestra un recorte del comando de detención (docker compose down).

```
% docker compose down
[+] Running 2/2
✓ Container roadtok8s-py-web-1 Re... 10.1s
✓ Network roadtok8s-py_default Re... 0.1s
% █
```

Figura 6.3 Detención de un contenedor de Python con Docker Compose

Docker Compose se puede detener de dos maneras:

- Utilizando Control + C, como muchos otros procesos.
- Usando Docker Compose desde otra ventana de terminal.

Este comando debería ser tu mejor opción para detener contenedores, ya que Docker Compose lo gestiona y se limpia mejor. Volveremos a este comando en el futuro.

¿FALTA DOCKER COMPOSE?

Si está en una máquina Linux y Docker Compose no está instalado, considere usar los siguientes comandos:

```
sudo apt-get update  
sudo apt-get install docker-compose-plugin
```

Si esto falla, es posible que tengas que usar una versión anterior de Docker Compose. Puedes instalar esta versión mediante el paquete de Python de Docker Compose (<https://pypi.org/project/docker-compose/>) con `python3 -m pip install docker-compose --upgrade`.

Una vez que esté instalado, utilizará `docker-compose` (observe el uso de un guion -) en lugar de `docker compose` siempre que necesite usar Docker Compose.

Esta versión de Docker Compose basada en Python puede requerir cambios adicionales en la configuración. Consulta la documentación oficial de Docker Compose (<https://docs.docker.com/compose/install/>) para obtener más información. _____

Ahora que tenemos nuestro primer vistazo a Docker Compose con nuestra aplicación Python, cambiemos de tema para nuestra aplicación Node.js; alerta de spoiler: será casi idéntica.

6.2.3 Docker Compose para nuestra aplicación Node.js

En este punto, te reto a crear tu propio archivo `compose.yaml` para tu aplicación Node.js, ya que es casi idéntico a lo que ya hemos visto. Como resumen rápido

En resumen, estos son los pasos que debemos seguir para construir y ejecutar nuestro contenedor Node.js:

1. Navegue a la raíz de nuestro proyecto con `cd ~/dev/roadtok8s/js`.
2. Ejecute nuestro comando de compilación con `docker build -f Dockerfile -t rk8s-js .`
3. Ejecute nuestro contenedor con `docker run -p 3001:3000 -e PORT=3001 rk8s-js`.

El proceso de compilación y ejecución, salvo por varios argumentos, es idéntico independientemente del contenedor que se utilice. Esta es la ventaja de Docker Compose: presenta esos argumentos en un formato fácil de repetir y muy portable. Ahora, dirígete a la raíz de tu proyecto Node.js (p. ej., `~/dev/roadtok8s/js`) y crea `compose.yaml` con el contenido de la siguiente lista.

Listado 6.4 Docker Compose para nuestra aplicación Node.js

```
servicios:
  web:
    construir:
      contexto: .
      dockerfile: Dockerfile
    Puertos:
      - Entorno
    "3000:3000":
      - PUERTO=3000
```

Una vez más, podemos construir, ejecutar y detener nuestro contenedor, respectivamente, con lo siguiente:

- compilación de docker compose
- Docker Compose
- Docker Compose Down

Te dejaré explorar esto en tu propio proyecto para verificar su correcto funcionamiento. Ahora que comprendemos los fundamentos de la gestión de contenedores con Docker Compose, comencemos a explorar otros tipos de aplicaciones que tus proyectos podrían necesitar: aplicaciones con estado .

6.3 Contenedores con estado y volúmenes

Los contenedores se pueden crear y destruir fácilmente; están diseñados para ser efímeros. Al pensar en archivos dentro de un contenedor puro, debería pensar en archivos temporales. Pero ¿qué ocurre si desea que los cambios persistan dentro de un contenedor? Aquí es donde entran en juego los volúmenes.

El tiempo de ejecución de un contenedor no tiene estado. Para que tenga estado, se agrega un volumen. Un volumen es básicamente una carpeta de su equipo local o sistema operativo, que se monta en el contenedor durante la ejecución. El volumen persistirá incluso si el contenedor se destruye, se reconstruye, se detiene, se inicia, etc. en.

La mejor manera de familiarizarse con los volúmenes es usándolos para ver lo simples que son. Empezaremos usando volúmenes con comandos Docker.

6.3.1 Uso de volúmenes con Docker

Para usar un volumen con la CLI de Docker, incluiremos el indicador `-v` junto con el argumento `<carpeta-local>:<carpeta-contenedor>`. En este caso, `<carpeta-local>` es un marcador de posición para la ruta a una carpeta en nuestra máquina local que queremos adjuntar a nuestro contenedor. `<carpeta-contenedor>` es un marcador de posición para la ruta.

Queremos montar esta carpeta en nuestro contenedor. Aquí hay un par de ejemplos:

- `docker run -v ~/dev/roadtok8s/py/data:/app/data/...`—Esto montará la carpeta `~/dev/roadtok8s/py/data/` en el contenedor en `/app/data/`. El `~/dev/roadtok8s/py/data/` será el mismo en cualquier lugar de la máquina. `docker run -v data:/app/data/ ..`—Esto montará la carpeta `data` ubicada relativamente en el contenedor en `/app/data/`. Esta carpeta `data` será relativa a la carpeta donde se ejecuta el comando (por ejemplo, usar `docker run -v data:... en path/to/some/folder` asumirá que `data` existe en `path/to/some/folder/data`).

Puedes adjuntar un volumen a cualquier contenedor. Desvincularlo es tan sencillo como detener el contenedor y volver a ejecutarlo sin el parámetro `-v`.

Antes de usar volúmenes, recapitulemos los distintos argumentos que necesita nuestro comando `docker run`:

- `docker run` es el comando para ejecutar un contenedor.
- `-v` es el indicador para adjuntar un volumen. `--`
- `name` es el indicador para nombrar nuestro contenedor en ejecución (esto hace que sea más fácil detener o ingresar al
- `shell`). `<image-name>` es el argumento final que usaremos para especificar nuestra imagen previamente construida (por ejemplo, con `docker build -t <image-name> -f Dockerfile`).

Usar la opción `--name <my-custom-name>` nos permite ejecutar comandos como `docker stop <my-custom-name>` para detener un contenedor y `docker exec -it my-custom-name /bin/bash` para acceder a la shell `bash`. Haremos ambas cosas en breve. La siguiente lista muestra cómo usar un volumen con nuestra aplicación Python.

Listado 6.5 Uso de volúmenes con Docker

```
docker run -v ~/dev/roadtok8s/py/data:/app/data/ --name my-r k8s-py rk8s-py
```

El nombre de este entorno de ejecución es my-rk8s-py, mientras que el de la imagen es rk8s-py. Estos dos nombres me confundieron durante mucho tiempo, así que es importante que comprenda la diferencia: usar --name simplemente le da un nombre arbitrario al entorno de ejecución de su contenedor, mientras que <image-name> debe ser una imagen válida. En otras palabras, el nombre del entorno de ejecución es una variable; el nombre de la imagen es fijo.

Con este contenedor en ejecución, probemos nuestro volumen con los siguientes comandos:

```
echo "hola mundo" >> ~/dev/roadtok8s/py/data/sample.txt docker exec  
my-rk8s-py cat /app/data/sample.txt
```

Si se hizo correctamente, debería ver el resultado "hola mundo" del comando cat . Esto significa que el volumen funciona correctamente.

Registraremos las fechas de una vista mediante el comando "date" en un archivo y veremos si podemos leerlas. Ejecute los siguientes comandos:

```
fecha >> ~/dev/roadtok8s/py/data/sample.txt fecha >>  
~/dev/roadtok8s/py/data/sample.txt fecha >> ~/dev/  
roadtok8s/py/data/sample.txt docker exec my-rk8s-py  
cat /app/data/sample.txt
```

NOTA: El comando date muestra la fecha y hora actuales en la terminal. Puedes usar cualquier comando, pero date es fácil de usar y puedes seguir su funcionamiento.

Si se hace correctamente, debería ver el resultado que se muestra en la figura 6.4.

```

% docker exec my-rk8s-py cat /app/data/sample.txt
hello world
Mon Aug 14 15:59:38 MDT 2023
Mon Aug 14 15:59:41 MDT 2023
Mon Aug 14 15:59:41 MDT 2023

```

Figura 6.4. Muestras de salida de volumen con Docker y una máquina local

También podemos agregar contenido al archivo desde el contenedor.

Ejecute los siguientes comandos:

```

docker exec my-rk8s-py bash -c "echo \"\n\" >> /app/data/sample.txt" docker exec my-
rk8s-py bash -c "echo \"desde docker\" >> /app/data/sample.tx
"t"
docker exec my-rk8s-py bash -c "fecha >> /app/data/sample.txt" docker exec
my-rk8s-py bash -c "fecha >> /app/data/sample.txt" cat ~/dev/roadtok8s/py/data/
sample.txt

```

La figura 6.5 muestra la salida de estos comandos.

```

% cat ~/dev/roadtok8s/py/data/sample.txt
hello world
Mon Aug 14 15:59:38 MDT 2023
Mon Aug 14 15:59:41 MDT 2023
Mon Aug 14 15:59:41 MDT 2023

from docker
Mon Aug 14 22:08:26 UTC 2023
Mon Aug 14 22:08:30 UTC 2023

```

Figura 6.5 Muestras de salida de volumen con Docker y una máquina local

Este sencillo ejemplo nos muestra algunos elementos clave sobre Volúmenes de Docker:

- Los volúmenes son una forma de conservar datos fuera de un contenedor.
- Se puede acceder a los volúmenes desde el contenedor o la máquina local.
- Los cambios en los archivos ocurren instantáneamente (por ejemplo, los archivos no se sincronizan ni se copian como en una compilación de Docker).
- Los volúmenes se utilizan en tiempo de ejecución.
- Aunque no siempre se recomienda debido a posibles daños en los archivos o problemas de lectura y escritura, los volúmenes se pueden usar para compartir datos entre contenedores.

Ahora que tenemos una comprensión básica de los volúmenes, usémoslos con Docker Compose.

6.3.2 Uso de volúmenes con Docker Compose

Siguiendo con nuestro volumen local ubicado en `~/dev/roadtok8s/py/data/`, ahora usaremos Docker Compose para montarlo en nuestro contenedor en la ruta `/app/data`. La siguiente lista muestra cómo usar un volumen con nuestra aplicación Python.

Listado 6.6 Configuración de Docker Compose con volúmenes

```
servicios: web:

  construir:
    contexto: .
    dockerfile: Dockerfile

  Puertos:
    - Entorno "8080:8080":

    - PUERTO=8080

  volúmenes:
    - ~/dev/roadtok8s/py/data:/app/data
```

Ahora podemos ejecutar lo siguiente:

- `docker compose up`: Para iniciar el servicio Docker Compose de nuestra aplicación web. Asegúrese de detener cualquier otro contenedor que se esté ejecutando en el puerto 8080. `docker compose`
- `exec web ls /app/data`: Para listar todos los archivos del volumen montados en el contenedor.
- `estibador compose exec web /app/ intento -do "fecha >> data/sample.txt`—Para agregar una nueva fecha a nuestra muestra. `docker compose`
- `exec web cat /app/data/sample.txt`—Para ver la salida de nuestro archivo de muestra.

Como vemos con estos comandos, Docker Compose ha reducido la capacidad de usar volúmenes con nuestros contenedores. Ahora podemos usar el comando `docker compose exec` para ejecutar comandos dentro de nuestro contenedor en lugar de necesitar conocer el nombre del contenedor o, para un uso más avanzado de Docker, su ID.

Para detener nuestros servicios de Docker Compose e intentar eliminar volúmenes, podemos ejecutar `docker compose down -v` o `docker compose down --volumes`. El motivo de este intento es...